تیروتیر
آموزشگاه هوش مصنوعی

برنامه‌نویسی برای همه

جزوه کاربردی پایتون



آموزش پایتون از صفر؛
برنامه‌نویسی متنی و گرافیکی با هم

دکتر داریوش علیپور
دکتر مینوش حیدری



نکته
یادآوری
هشدار
راهنمایی
کوییز

کنسول
(متنی)
+
Tkinter
(گرافیکی)
=
یادگیری عمیق‌تر
و کاربردی‌تر

پروژه‌های واقعی
و کاربردی

پیوست‌های کاربردی
و برگه تقلب جامع

کوییز و تمرین‌های
کلاسی و خانگی

مثال‌های کامل
و خروجی نمونه

برنامه‌نویسی را فقط با خواندن نمی‌توان یاد گرفت. روزی ۳۰ دقیقه تمرین از هفته‌ای یک بار چند ساعت نشستن موثرتر است.

چاپ اول — آبان ۱۴۰۵

مجوز انتشار
Creative Commons
Attribution–NonCommercial 4.0 International
(CC BY-NC 4.0)

نسخه‌های به‌رور:
@AI_and_Philosophy
https://github.com/worker2025/python-jozveh
tirotir.ir

آموزشگاه هوش مصنوعی

Tirotir.ir

جزوه کاربردی پایتون

آموزش پایتون از صفر؛ برنامه‌نویسی متنی و گرافیکی با هم

نویسندگان

دکتر داریوش علیپور

دکتر مینوش حیدری

چاپ اول — آبان ۱۴۰۵

شناسنامه کتاب

عنوان	**جزوه کاربردی پایتون**
نویسندگان	دکتر داریوش علیپور دکتر مینوش حیدری
سال انتشار	چاپ اول — آبان ۱۴۰۵
نوبت چاپ و ویرایش	اول — آبان ۱۴۰۵ https://doi.org/10.5281/zenodo.23833424
نسخه‌های به‌روز بعدی	کانال تلگرام «فلسفه هوش مصنوعی»: @AI_and_Philosophy مخزن گیت‌هاب: https://github.com/worker2025/python-jozveh وب‌سایت: tirotir.ir

مجوز انتشار

این کتاب با مجوز زیر منتشر شده است.

Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)

بر اساس این مجوز:

- مطالعه، نسخه‌برداری، چاپ، اشتراک‌گذاری و توزیع کتاب آزاد است.
- ترجمه، ویرایش، اقتباس و تولید آثار مشتق از کتاب، با ذکر نام نویسنده و صرفاً برای اهداف غیرتجاری، مجاز است.
- استفاده‌ی تجاری، از جمله فروش کتاب (چاپی یا الکترونیکی) یا فروش آثار مشتق، بدون اجازه‌ی کتبی نویسنده مجاز نیست.
- در تمامی نسخه‌های منتشرشده، نام نویسنده و لینک نسخه‌ی به‌روزرسانی باید حفظ شود:

@AI_and_Philosophy
https://github.com/worker2025/python-jozveh
tirotir.ir

متن کامل مجوز:

https://creativecommons.org/licenses/by-nc/4.0

پیشگفتار

پایتون امروز یکی از ساده‌ترین و در عین حال پرکاربردترین زبان‌های برنامه‌نویسی است. از ساخت یک بازی کوچک تا تحلیل داده و هوش مصنوعی، همه در همین زبان نوشته می‌شوند. اما بسیاری از آموزش‌های پایتون یا برای کسانی نوشته شده‌اند که قبلاً برنامه‌نویسی می‌دانند، یا آن‌قدر ساده‌اند که خواننده را بعد از چند جلسه بدون مهارت عملی رها می‌کنند.

این جزوه برای **کسی که از صفر شروع می‌کند** نوشته شده است: دانش‌آموز، دانشجو، معلم یا هر علاقه‌مندی که می‌خواهد با دست خودش برنامه بنویسد و ببیند کار می‌کند.

ایده این کتاب: متنی و گرافیکی با هم

در بیشتر کتاب‌ها اول چند فصل با پنجره سیاه کنسول کار می‌کنید و ساخت برنامه‌ای با پنجره و دکمه به فصل‌های آخر می‌رسد. این کتاب این ترتیب را عوض کرده است. در **هر درس** هر مفهوم دو بار دیده می‌شود: یک بار در کنسول (با `print` و `input`) و یک بار در یک پنجره گرافیکی (با `tkinter`). دلیلش ساده است:

- مفهوم‌های انتزاعی مثل متغیر و حلقه وقتی نتیجه‌شان را روی صفحه ببینید، ملموس‌تر می‌شوند.
- از همان جلسه اول برنامه‌ای می‌سازید که شبیه نرم‌افزارهای واقعی است و این انگیزه را بالا نگه می‌دارد.
- مقایسه دو نسخه نشان می‌دهد که منطق برنامه یکی است و فقط شیوه ارتباط با کاربر فرق می‌کند.

ساختار کتاب

کتاب ۱۲ درس دارد. هر درس با هدف‌ها شروع می‌شود، مثال‌های کامل و خروجی نمونه می‌دهد و با کوییز، تمرین کلاسی و تمرین در خانه پایان می‌یابد. پاسخ همه کوییزها و حل تمرین‌های خانه در پاسخنامه آمده است. در پایان کتاب پیوست‌ها قرار دارند: معرفی ریپوهای آموزشی، برگه تقلب جامع، و موضوع‌های تکمیلی مثل ماژول‌ها، اشکال‌زدایی، چیدمان در Tkinter، نصب و عیب‌یابی، شیوه‌نامه کدنویسی، راهنمای مدرس، واژه‌نامه، منابع و نمایه.

جعبه‌های رنگی در متن کمک می‌کنند سریع‌تر بیاموزید: **نکته**، **یادآوری**، **هشدار**، **راهنمایی** و **کوییز**.

چطور از این کتاب استفاده کنیم

۱. هر درس را در یک جلسه بخوانید و **همه مثال‌ها را خودتان تایپ و اجرا کنید**. دیدن کد با نوشتن آن فرق دارد.
۲. نتیجه را پیش از اجرا حدس بزنید. اگر حدستان غلط بود، دقیقاً همان‌جا چیز تازه‌ای یاد گرفته‌اید.
۳. به خطاها نترسید. پیام خطا به شما می‌گوید مشکل کجاست (پیوست اشکال‌زدایی را ببینید).
۴. کوییز و تمرین‌ها را قبل از دیدن پاسخ‌ها انجام دهید.
۵. کدهای کامل کتاب در ریپوی همراه آن در دسترس است تا لازم نباشد همه چیز را از نو تایپ کنید.

برای مدرسان

کتاب برای تدریس در ۱۲ جلسه طراحی شده است. پیشنهاد زمان‌بندی، نکات تدریس هر درس و دو نمونه آزمون با پاسخ در پیوست «راهنمای مدرس» آمده است.

نسخه‌های به‌روز و بازخورد

کتاب زنده است و با پیشنهاد خوانندگان بهتر می‌شود. نسخه‌های به‌روز را از کانال تلگرام «فلسفه هوش مصنوعی» (`@AI_and_Philosophy`)، مخزن گیت‌هاب کتاب و وب‌سایت tirotir.ir دنبال کنید. اگر اشتباهی دیدید یا پیشنهادی دارید، خوشحال می‌شویم با کمک شما اشتباه را رفع کرده و پیشنهاد را اعمال کنیم.

> **نکته:** برنامه‌نویسی را فقط با خواندن نمی‌توان یاد گرفت. روزی ۳۰ دقیقه تمرین از هفته‌ای یک بار چند ساعت نشستن مؤثرتر است.

با آرزوی موفقیت

داریوش علیپور و مینوش حیدری

آبان ۱۴۰۵

فهرست مطالب

پیشگفتار **۴**

پیش از شروع **۱**

درس ۱: اولین برنامه و اولین پنجره **۴**

درس ۲: متغیر و دریافت ورودی **۱۱**

درس ۳: عملیات ریاضی **۱۸**

درس ۴: رشته‌ها و پیام‌ها **۲۶**

درس ۵: تصمیم‌گیری با if **۳۳**

درس ۶: حلقه‌ها **۴۱**

درس ۷: لیست‌ها **۴۸**

درس ۸: تابع‌ها **۵۶**

درس ۹: دیکشنری، تاپل و انتخاب‌ها **۶۵**

درس ۱۰: کار با فایل **۷۴**

درس ۱۱: مدیریت خطا، منو و لغزنده **۸۱**

درس ۱۲: پروژه‌های پایانی **۸۹**

پاسخنامه و راهنمای حل تمرین‌ها **۹۹**

پاسخ‌های درس ۱: اولین برنامه و اولین پنجره ۹۹

پاسخ‌های درس ۲: متغیر و دریافت ورودی ۱۰۰

پاسخ‌های درس ۳: عملیات ریاضی ۱۰۲

پاسخ‌های درس ۴: رشته‌ها و پیام‌ها ۱۰۳

پاسخ‌های درس ۵: تصمیم‌گیری با if ۱۰۵

پاسخ‌های درس ۶: حلقه‌ها ۱۰۶

پاسخ‌های درس ۷: لیست‌ها ۱۰۸

پاسخ‌های درس ۸: تابع‌ها ۱۱۰

پاسخ‌های درس ۹: دیکشنری، تاپل و انتخاب‌ها ۱۱۲

پاسخ‌های درس ۱۰: کار با فایل ۱۱۴

پاسخ‌های درس ۱۱: مدیریت خطا، منو و لغزنده ۱۱۶

پاسخ‌های درس ۱۲: پروژه‌های پایانی ۱۱۹

پیوست الف: ریپوهای مرتبط و فایل‌های مهم **۱۲۳**

۱. ریپوی کدهای همین کتاب: python-jozveh ۱۲۳
۲. ریپوی Python-۱ (ترم اول پایتون) ۱۲۴
۳. نقشه درس‌های کتاب به فایل‌های ریپوی Python-۱ ۱۲۴
۴. فراتر از کتاب: کلاس‌ها (OOP) ۱۲۷
۵. ریپوی PythonKids_Lessons_ENG ۱۲۷
۶. اکستنشن‌های VS Code ۱۲۷
۷. مسیر پیشنهادی یادگیری با این منابع ۱۲۸

پیوست ب: برگه تقلب جامع پایتون ۱۲۹

۱. پایه‌ها ۱۲۹
۲. رشته‌ها ۱۳۰
۳. لیست، تاپل، مجموعه، دیکشنری ۱۳۰
۴. کنترل جریان ۱۳۱
۵. تابع‌ها ۱۳۲
۶. خطاها ۱۳۲
۷. کلاس‌ها ۱۳۳
۸. فایل، مسیر و داده ۱۳۴
۹. کتابخانه‌های استاندارد پرکاربرد ۱۳۵
۱۰. محیط مجازی و pip ۱۳۶
۱۱. Tkinter: خلاصه ابزارها ۱۳۶
۱۲. Turtle: نقاشی با لاک‌پشت ۱۳۷
۱۳. رنگی کردن متن و گفتار ۱۳۷
۱۴. ساخت فایل اجرایی با PyInstaller ۱۳۷
۱۵. دکتر خطا: معنی پیام‌های رایج ۱۳۸
۱۶. میانبرهای VS Code ۱۳۸
۱۷. ۵۳ کتابخانه معروف پایتون ۱۳۹
۱۸. ۲۰ ترفند داغ ۱۴۱
۱۹. ایده‌های بعدی برای یادگیری ۱۴۲

پیوست پ: ماژول‌ها و دستور import ۱۴۳

۱. ماژول چیست؟ ۱۴۳
۲. چهار شکل import ۱۴۳
۳. دیدن محتوای ماژول: dir و help ۱۴۴
۴. ماژول‌های استاندارد پرکاربرد ۱۴۴
۵. ساختن ماژول خودمان ۱۴۶
۶. شرط '__main__' == __name__ if' ۱۴۷
۷. بسته (package) ۱۴۷
۸. ماژول‌های نصب‌شدنی و خطای ModuleNotFoundError ۱۴۷

پیوست ت: اشکال‌زدایی (Debugging) ۱۵۱

۱. سه نوع خطا ۱۵۱

۲. خواندن پیام خطا (Traceback) ۱۵۱

۳. روش اول: ردیابی با print ۱۵۲

۴. روش دوم: اشکال‌زدای VS Code ۱۵۲

۵. روش سوم: assert و logging ۱۵۴

۶. استراتژی ده‌مرحله‌ای رفع اشکال ۱۵۵

۷. چطور سؤال خوب بپرسیم؟ ۱۵۵

۸. خطاهای رایج مبتدی‌ها: چک‌لیست ۱۵۵

۹. برنامه‌های معیوب برای تمرین ۱۵۶

پیوست ث: چیدمان در Tkinter و نقاشی با Canvas ۱۶۰

۱. سه مدیر چیدمان ۱۶۰

۲. pack: گزینه‌ها ۱۶۰

۳. grid: سطر و ستون ۱۶۱

۴. place: مختصات ۱۶۲

۵. Frame و LabelFrame: ظرف‌ها ۱۶۲

۶. ماشین‌حساب با grid ۱۶۳

۷. Canvas: بوم نقاشی ۱۶۵

۸. انیمیشن با after و move ۱۶۶

پیوست ج: نصب و عیب‌یابی ۱۷۱

۱. نصب پایتون ۱۷۱

۲. ساخت و اجرای اولین فایل ۱۷۱

۳. مشکل: 'python' is not recognized ۱۷۲

۴. مشکل: VS Code پایتون را پیدا نمی‌کند ۱۷۲

۵. مشکل: pip شناخته نمی‌شود یا کتابخانه نصب نمی‌شود ۱۷۲

۶. مشکل: 'No module named 'tkinter ۱۷۲

۷. مشکل: نمایش فارسی ۱۷۳

۸. مشکل: فایل پیدا نمی‌شود (FileNotFoundError) ۱۷۴

۹. مشکل: پنجره گرافیکی باز نمی‌شود یا می‌بندد ۱۷۴

۱۰. مشکل: فعال‌سازی محیط مجازی در PowerShell ۱۷۴

۱۱. مشکل: ساخت exe با PyInstaller ۱۷۴

۱۲. جدول خلاصه: خطا، علت، راه‌حل ۱۷۵

پیوست چ: شیوه‌نامه کدنویسی (PEP ۸) ۱۷۷

۱. نام‌گذاری ۱۷۷

۲. فاصله‌گذاری و ایندنت ۱۷۷

۳. import ها ۱۷۷

۴. توضیح و docstring ۱۷۸
۵. مقایسه: قبل و بعد ۱۷۸
۶. چند قاعده مهم دیگر ۱۷۸
۷. ابزارهای خودکار ۱۷۹

پیوست ح: راهنمای مدرس و آزمون‌ها ۱۸۱
۱. ساختار پیشنهادی هر جلسه (۹۰ دقیقه) ۱۸۱
۲. برنامه ۱۲ جلسه‌ای ۱۸۱
۳. نکات تدریس و اشتباه‌های رایج هر درس ۱۸۲
۴. پیشنهاد ارزیابی ۱۸۲
۵. آزمون میان‌ترم (دروس ۱ تا ۶) ۱۸۳
۶. آزمون پایان‌ترم (کل کتاب) ۱۸۶
۷. معیار ارزیابی پروژه نهایی (۲۰ نمره) ۱۸۹
۸. پاسخنامه آزمون‌ها ۱۸۹

پیوست خ: واژه‌نامه فارسی ـ انگلیسی ۱۹۳
اصطلاح‌های هم‌ریشه و تفاوت‌های ظریف ۱۹۷

پیوست د: منابع و مراجع ۱۹۹
۱. مستندات رسمی ۱۹۹
۲. کتاب‌های پیشنهادی (انگلیسی) ۱۹۹
۳. وب‌سایت‌های آموزشی ۱۹۹
۴. منابع آموزشگاه Tirotir ۲۰۰
۵. مسیرهای بعدی یادگیری ۲۰۰

پیوست ذ: نمایه ۲۰۱

پیش از شروع

روش کار این جزوه: در هر درس یک مفهوم پایتون را دو بار می‌بینید: یک بار در کنسول (ورودی و خروجی متنی) و یک بار در پنجره گرافیکی با Tkinter. این‌طور مفهوم هم درست یاد گرفته می‌شود و هم همان لحظه نتیجه‌اش را روی صفحه می‌بینید.

ساختار هر درس

هر درس از این بخش‌ها ساخته شده است:

- **هدف‌ها:** بعد از درس چه کارهایی را می‌توانید انجام دهید.
- **مفهوم و مثال‌ها:** توضیح کوتاه و چند مثال، هرکدام با نسخه متنی و گرافیکی.
- **جعبه‌های رنگی:** نکته، یادآوری، هشدار و راهنمایی که در ادامه معرفی می‌شوند.
- **کوییز:** چند سؤال برای سنجش یادگیری. پاسخ‌ها در پایان جزوه است.
- **تمرین کلاسی:** در کلاس با راهنمایی مدرس انجام می‌شود.
- **تمرین در خانه:** مسئله‌های بیشتر برای تمرین مستقل. حل آنها در پایان جزوه آمده است.

راهنمای جعبه‌های رنگی

نکته: نکته‌ها اطلاعات مفیدی هستند که کار شما را سریع‌تر یا تمیزتر می‌کنند.

یادآوری: یادآوری‌ها مطلبی را که قبلاً یاد گرفته‌اید دوباره به شما می‌گویند.

هشدار: هشدارها خطاهای رایج و اشتباه‌هایی هستند که بیشتر مبتدی‌ها مرتکب می‌شوند.

راهنمایی: راهنمایی‌ها سرنخ حل تمرین‌ها هستند، بدون اینکه جواب را بدهند.

کوییز: جعبه بنفش یعنی وقت سنجش یادگیری است.

آماده‌سازی محیط

۱. پایتون را از سایت رسمی python.org دریافت و نصب کنید. هنگام نصب حتماً گزینه Add Python to PATH را تیک بزنید.

۲. VS Code را نصب کنید.

۳. در VS Code با Ctrl+Shift+X پنجره افزونه‌ها را باز کنید و اکستنشن‌های **izeh**، **Python** و **Tirotir** را نصب کنید.

۴. یک پوشه برای دوره بسازید، مثلاً `python-course`، و با File و سپس Open Folder آن را باز کنید.

۵. برای هر درس یک فایل جدا بسازید: `lesson1.py`، `lesson2.py` و همین‌طور تا آخر.

۶. برای اجرا روی دکمه مثلثی (Run Python File) بالای ویرایشگر بزنید.

> **نکته:** اگر پایتون نصب شده است، در ترمینال VS Code دستور `python --version` را بزنید. باید شماره نسخه نمایش داده شود. در بعضی سیستم‌ها باید `py --version` بزنید.

> **هشدار:** اسم فایل‌هایتان را `tkinter.py` یا `random.py` نگذارید. پایتون به جای کتابخانه اصلی فایل شما را بارگذاری می‌کند و برنامه خطا می‌دهد.

اسنیپت‌ها: کدنویسی سریع‌تر

اسنیپت یک تکه کد آماده است. نام آن را بنویسید و Tab بزنید تا کد کامل درج شود.

- **izeh** (اسنیپت‌های پایتون): `izeh`، `izehadd`، `izehconcat`، `izehreadfile`، `izehwritefile`، `izehfuncdef`، `izehlistops`
- **Tirotir** (اسنیپت‌های Tkinter): `tir` پنجره، `tirB` دکمه، `tirL` برچسب، `tirE` ورودی، `tirF` قاب، `tirM` منو، `tirMsg` پیام، `tirC` بوم، `tirCB` چک‌باکس، `tirRB` رادیو، `tirS` لغزنده، `tirLB` لیست، `tirT` متن

> **یادآوری:** اسنیپت‌ها فقط وقتی کار می‌کنند که فایل شما پسوند `.py` داشته باشد و اکستنشن مربوط نصب و فعال باشد.

برنامه پیشنهادی مطالعه

- هر درس را در یک جلسه بخوانید و **همه مثال‌ها را خودتان تایپ و اجرا کنید**. فقط خواندن کافی نیست.
- پیش از شروع درس بعد، کوییز و تمرین‌های درس قبل را انجام دهید.
- اگر روی تمرینی بیش از ۲۰ دقیقه گیر کردید، راهنمایی را بخوانید و بعد پاسخ را ببینید.

- هر هفته درس‌های قبلی را مرور کنید.

> **نکته:** برنامه‌نویسی با دیدن یاد گرفته نمی‌شود، با نوشتن و خطا گرفتن یاد گرفته می‌شود. از پیام‌های خطا نترسید: آنها به شما می‌گویند مشکل کجاست.

درس ۱: اولین برنامه و اولین پنجره

هدف‌ها

- اجرای یک برنامه پایتون در VS Code
- نمایش متن با `print` و آشنایی با `sep`، `end` و کاراکترهای ویژه
- نوشتن توضیح (کامنت) در کد
- ساخت یک پنجره با `tkinter`، تنظیم عنوان، اندازه و رنگ
- گذاشتن برچسب و دکمه در پنجره

مفهوم: برنامه چیست؟

برنامه فهرستی از دستورهاست که کامپیوتر به ترتیب و خط‌به‌خط اجرا می‌کند. پایتون زبانی است که دستورهایش به انگلیسی ساده نزدیک است، برای همین برای شروع عالی است.

برنامه‌ها دو شکل دارند. در **برنامه متنی** ارتباط با کاربر از راه تایپ و نوشته در پنجره سیاه کنسول است. در **برنامه گرافیکی** (GUI) کاربر با پنجره، دکمه و کادر متنی کار می‌کند، مثل برنامه‌هایی که هر روز استفاده می‌کنید. در این دوره هر دو را با هم یاد می‌گیریم.

> **یادآوری:** کتابخانه `tkinter` همراه خود پایتون نصب می‌شود و نیازی به نصب جداگانه ندارد.

مثال ۱: اولین دستور print

دستور `print` هر چیزی را که داخل پرانتز بنویسید در کنسول نشان می‌دهد. متن را باید داخل نقل‌قول (تک یا جفت) بگذارید.

```
print('سلام دنیا!')
```

خروجی:

```
سلام دنیا!
```

> **نکته:** بین نقل‌قول تک `'` و جفت `"` فرقی نیست، فقط باید ابتدا و انتهای متن یکی باشد. اگر داخل متن نقل‌قول دارید، از نوع دیگر برای دور آن استفاده کنید.

مثال ۲: چند خط خروجی

هر `print` خروجی را در یک خط جدید می‌نویسد. می‌توان عدد و متن را با کاما کنار هم چاپ کرد.

```
print('سلام')
print('نام من علی است')
print('سن من:', 15)
print("It's a nice day")
print('او گفت: "سلام"')
```

خروجی:

```
سلام
نام من علی است
سن من: 15
It's a nice day
او گفت: "سلام"
```

مثال ۳: جداکننده و پایان خط

دستور `print` دو گزینه دارد: `sep` (چیزی که بین مقدارها قرار می‌گیرد) و `end` (چیزی که در انتها چاپ می‌شود، پیش‌فرض رفتن به خط جدید).

```
print('الف', 'ب', 'ج', sep='-')
print('بدون خط جدید', end=' ')
print('ادامه همان خط')
print('خط اول\nخط دوم')
print('ستون۱\tستون۲')
```

خروجی:

```
الف-ب-ج
بدون خط جدید ادامه همان خط
خط اول
خط دوم
ستون۱	ستون۲
```

> **نکته:** `\n` یعنی خط جدید و `\t` یعنی یک فاصله بلند (Tab). به اینها کاراکترهای ویژه می‌گویند.

مثال ۴: کامنت

متنی که بعد از `#` نوشته شود را پایتون نادیده می‌گیرد. کامنت برای توضیح دادن کد به آدم‌هاست.

```
# این برنامه یک پیام چاپ می‌کند
print('سلام')   # این هم یک کامنت است


# print('این خط اجرا نمی‌شود')
```

خروجی:

```
سلام
```

> **نکته:** کامنت‌گذاری یک خط به شکل موقت را «غیرفعال کردن کد» می‌گویند. در VS Code با انتخاب خط و زدن Ctrl+/ این کار را انجام دهید.

مثال ۵: کارت معرفی متنی

با تکرار کاراکتر می‌توان کادر کشید. عملگر `*` یک متن را چند بار تکرار می‌کند.

```
print('*' * 24)
print('*  علی رضایی')
print('*  دانش‌آموز پایتون')
print('*  Tirotir.ir')
print('*' * 24)
```

خروجی:

```
************************
*  علی رضایی
*  دانش‌آموز پایتون
*  Tirotir.ir
************************
```

مثال ۶: اولین پنجره

حالا همان سلام را در یک پنجره بسازیم. سه خط اصلی هر برنامه گرافیکی: ساختن پنجره، چیدن اجزا، و `mainloop` که پنجره را باز نگه می‌دارد.

```
import tkinter as tk

win = tk.Tk()
win.title('اولین پنجره')
win.geometry('300x150')
tk.Label(win, text='سلام دنیا!', font=('Tahoma', 16)).pack(pady=40)
win.mainloop()
```

> **نکته:** `'300x150'` یعنی عرض ۳۰۰ و ارتفاع ۱۵۰ پیکسل. بین دو عدد حرف x انگلیسی بدون فاصله است.

> **هشدار:** اگر `win.mainloop()` را فراموش کنید، پنجره یک لحظه باز می‌شود و فوراً بسته می‌شود.

مثال ۷: کارت معرفی گرافیکی

چند برچسب با رنگ و فونت متفاوت و رنگ پس‌زمینه.

```
import tkinter as tk

win = tk.Tk()
win.title('کارت معرفی')
win.geometry('320x220')
win.config(bg='lightyellow')

tk.Label(win, text='علی رضایی', font=('Tahoma', 18, 'bold'),
         bg='lightyellow').pack(pady=10)
tk.Label(win, text='دانش‌آموز پایتون', fg='blue',
         bg='lightyellow').pack()
tk.Label(win, text='Tirotir.ir', fg='gray',
         bg='lightyellow').pack(pady=20)
win.mainloop()
```

مثال ۸: دکمه خروج

دکمه با `Button` ساخته می‌شود و `command` مشخص می‌کند با کلیک چه شود. `win.destroy` پنجره را می‌بندد.

```
import tkinter as tk

win = tk.Tk()
win.title('خروج')
win.geometry('260x140')
tk.Label(win, text='برای بستن دکمه را بزنید').pack(pady=15)
tk.Button(win, text='خروج', command=win.destroy).pack()
win.mainloop()
```

> **یادآوری:** در `command=win.destroy` بعد از نام تابع پرانتز نمی‌گذاریم. اگر بگذارید، تابع همان لحظه اجرا می‌شود و پنجره بلافاصله بسته می‌شود.

مثال ۹: پنجره‌ای با اندازه ثابت

```
import tkinter as tk

win = tk.Tk()
win.title('پنجره ثابت')
win.geometry('360x200')
win.resizable(False, False)
tk.Label(win, text='اندازه این پنجره تغییر نمی‌کند',
         font=('Tahoma', 12)).pack(expand=True)
win.mainloop()
```

خطاهای رایج درس اول

- **`NameError: name 'Print' is not defined`**: پایتون بین حروف بزرگ و کوچک فرق می‌گذارد. `print` را با حرف کوچک بنویسید.
- **`SyntaxError`**: معمولاً یک پرانتز یا نقل‌قول را نبسته‌اید.
- **`ModuleNotFoundError: No module named 'Tkinter'`**: نام کتابخانه با حرف کوچک است: `tkinter`.
- **پنجره باز نمی‌شود:** فایل را ذخیره کرده و دوباره اجرا کنید. احتمالاً `mainloop` را ننوشته‌اید.

> **کوییز:** به کوییز زیر پاسخ دهید و بعد پاسخ‌ها را در پایان جزوه بخوانید.

کوییز

س۱. کدام دستور متن را در کنسول نمایش می‌دهد؟

الف) `show('سلام')`

ب) `print('سلام')`

ج) `display 'سلام'`

د) `write.print('سلام')`

س۲. خروجی `print('ب', 'الف', sep='+')` چیست؟

الف) `الف ب`

ب) `الف‌ب`

ج) `الف+ب`

د) `الف+ب+`

س۳. نماد شروع کامنت در پایتون کدام است؟

الف) `//`

ب) `#`

ج) `--`

د) `/*`

س۴. در `win.geometry('400x300')` عدد ۴۰۰ چه چیزی است؟

الف) ارتفاع پنجره

ب) عرض پنجره

ج) مکان پنجره روی صفحه

د) فاصله از لبه

س۵. کدام خط پنجره را باز نگه می‌دارد؟

الف) `win.title()`

ب) `win.destroy()`

ج) `win.mainloop()`

د) `win.pack()`

تمرین کلاسی

۱. با `print` سه خط بنویسید: نام، شهر و رشته مورد علاقه‌تان.

۲. پنجره‌ای به اندازه ۳۵۰ در ۲۰۰ بسازید که عنوان آن «پنجره من» باشد و داخلش نام شما نوشته شود.

۳. در همان پنجره یک دکمه «بستن» بگذارید.

> **راهنمایی:** برای تمرین ۲ از مثال ۶ شروع کنید و فقط `geometry`، `title` و متن را عوض کنید. برای تمرین ۳ مثال ۸ را ببینید.

تمرین در خانه

۱. با `print` و کاراکتر `*` یک مثلث پنج خطی بکشید.

۲. برنامه متنی بنویسید که یک کارت ویزیت شامل نام، تلفن و ایمیل را در کادر ستاره‌ای چاپ کند.

۳. پنجره‌ای بسازید که رنگ پس‌زمینه آن دلخواه باشد و سه برچسب با سه رنگ متفاوت داشته باشد.

۴. پنجره‌ای با اندازه ثابت ۴۰۰×۳۰۰ و دو دکمه بسازید: دکمه اول «خروج» و دکمه دوم هر کاری که دوست دارید (فعلاً فقط ظاهر).

۵. **چالش:** با `print` اسم خودتان را به شکل بزرگ و هنری (با حروف و علامت‌ها) بکشید.

راهنمایی: برای مثلث از `print(' ' * 4 + '*')` شروع کنید و در هر خط یک فاصله کمتر و دو ستاره بیشتر بگذارید.

درس ۲: متغیر و دریافت ورودی

هدف‌ها

- ساختن متغیر و دادن نام درست به آن
- شناخت انواع `str`، `int`، `float` و `bool`
- گرفتن ورودی در کنسول با `input` و در پنجره با `Entry`
- تبدیل نوع داده با `int()` و `float()` و `str()`
- نمایش نتیجه در `Label`

مفهوم: متغیر

متغیر مثل یک جعبه برچسب‌دار است: اسمی دارد و مقداری در آن نگه‌داری می‌شود. با علامت `=` مقدار را در متغیر می‌گذاریم. این علامت یعنی «بگذار»، نه برابر ریاضی.

هر مقدار نوعی دارد: `str` (متن)، `int` (عدد صحیح مثل ۱۵)، `float` (عدد اعشاری مثل ۳٫۵) و `bool` (فقط `True` یا `False`). با تابع `type` می‌توان نوع را دید.

مثال ۱: ساختن و چاپ متغیر

```
name = 'سارا'
age = 15
height = 1.62
is_student = True

print(name)
print(age)
print(height)
print(is_student)
```

خروجی:

```
سارا
15
1.62
True
```

مثال ۲: دیدن نوع داده

```
print(type('سلام'))
print(type(15))
print(type(1.5))
print(type(True))
```

خروجی:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

مثال ۳: تغییر مقدار

مقدار متغیر را هر زمان می‌توان عوض کرد. می‌توان از مقدار قبلی خودش هم استفاده کرد.

```
score = 10
print(score)
score = 15
print(score)
score = score + 5
print(score)
```

خروجی:

```
10
15
20
```

> **نکته:** `score += 5` همان `score = score + 5` است. برای `-`، `*` و `/` هم همین‌طور است.

قانون‌های نام‌گذاری

- نام فقط از حرف انگلیسی، عدد و زیرخط `_` ساخته می‌شود.
- نام نباید با عدد شروع شود: `1name` غلط است ولی `name1` درست است.
- فاصله ندارد: `first name` غلط است، `first_name` درست است.
- حروف بزرگ و کوچک فرق دارند: `Age` و `age` دو متغیر جدا هستند.
- از نام‌هایی مثل `print` و `input` برای متغیر استفاده نکنید.

> **نکته:** نام‌های معنادار بگذارید. `age` از `a` خواناتر است و بعد از یک هفته هم می‌فهمید چه بود.

مثال ۴: گرفتن ورودی در کنسول

تابع `input` پیامی نشان می‌دهد و منتظر می‌ماند کاربر چیزی تایپ کند و Enter بزند. نتیجه همیشه `str` است.

```
name = input('نام شما؟ ')
print('سلام', name)
```

خروجی:

```
نام شما؟ علی
سلام علی
```

مثال ۵: ورودی عددی

چون `input` همیشه متن می‌دهد، برای محاسبه باید با `int()` یا `float()` عدد کرد.

```
name = input('نام شما؟ ')
age = int(input('سن شما؟ '))
print('سلام', name, '- سال بعد', age + 1, 'ساله می‌شوید')
```

خروجی:

```
نام شما؟ علی
سن شما؟ 15
سلام علی - سال بعد 16 ساله می‌شوید
```

هشدار: اگر `int` را فراموش کنید، `age + 1` خطای `TypeError` می‌دهد، چون نمی‌شود متن را با عدد جمع کرد.

مثال ۶: تبدیل نوع

```
print(int('25') + 5)
print(float('2.5') * 2)
print(str(100) + ' تومان')
print(int(3.9))
```

خروجی:

```
30
5.0
100 تومان
3
```

یادآوری: `int(3.9)` عدد را گرد نمی‌کند، فقط قسمت اعشار را حذف می‌کند و ۳ می‌شود.

مثال ۷: جابه‌جایی دو متغیر

```
a = 'قرمز'
b = 'آبی'
a, b = b, a
print(a, b)
```

خروجی:

```
آبی قرمز
```

مثال ۸: اولین ورودی گرافیکی

`Entry` کادر تایپ است. با `.get()` مقدارش را می‌خوانیم و با `Label.config(text=...)` متن برچسب را عوض می‌کنیم.

```
import tkinter as tk

win = tk.Tk()
win.title('سلام')
win.geometry('280x180')

tk.Label(win, text='نـام شمـا:').pack(pady=5)
entry = tk.Entry(win)
entry.pack()
result = tk.Label(win, text='', font=('Tahoma', 12))
result.pack(pady=10)

def greet():
    result.config(text='سلام ' + entry.get())

tk.Button(win, text='سلام کـن', command=greet).pack()
win.mainloop()
```

> **نکته:** تابعی که به `command` می‌دهیم باید قبل از ساختن دکمه تعریف شده باشد. پس `def greet()` را بالاتر از `Button` بنویسید.

مثال ۹: دو ورودی و جمله ساختن

```
import tkinter as tk


win = tk.Tk()
win.title('معرفی')
win.geometry('300x240')


tk.Label(win, text='نام:').pack()
e_name = tk.Entry(win)
e_name.pack()
tk.Label(win, text='شهر:').pack()
e_city = tk.Entry(win)
e_city.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def show():
    out.config(text=e_name.get() + ' اهل ' + e_city.get() + ' است')


tk.Button(win, text='نمایش', command=show).pack()
win.mainloop()
```

مثال ۱۰: محاسبه سن سال بعد در پنجره

```
import tkinter as tk


win = tk.Tk()
win.geometry('280x180')
tk.Label(win, text='سن شما:').pack(pady=5)
e_age = tk.Entry(win)
e_age.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def next_year():
    age = int(e_age.get())
    out.config(text=f'سال بعد {age + 1} سالِه می‌شوید')


tk.Button(win, text='محاسبه', command=next_year).pack()
win.mainloop()
```

هشدار: اگر در کادر سن چیزی غیر از عدد بنویسید برنامه خطای `ValueError` می‌دهد. در درس یازدهم یاد می‌گیریم چطور جلوی این خطا را بگیریم.

> **کوییز:** وقت سنجش یادگیری است.

کوییز

س۱. نوع مقدار `3.5` چیست؟

الف) `int`

ب) `str`

ج) `float`

د) `bool`

س۲. کدام نام متغیر **غلط** است؟

الف) `user_name`

ب) `age2`

ج) `2age`

د) `_score`

س۳. خروجی `input('عدد : ')` از چه نوعی است؟

الف) همیشه `int`

ب) همیشه `str`

ج) بستگی به ورودی دارد

د) `float`

س۴. بعد از اجرای `x = 5` و `x = x * 2` مقدار `x` چند است؟

الف) ۵

ب) ۷

ج) ۱۰

د) ۲۵

س۵. برای خواندن متن کادر `Entry` به نام `e` از کدام استفاده می‌کنیم؟

الف) `e.text`

ب) `e.read()`

ج) `e.get()`

د) `e.value`

تمرین کلاسی

۱. نام و نام خانوادگی را جدا بگیرید و با فاصله چاپ کنید.

۲. سن را بگیرید و بگویید ده سال دیگر چند ساله می‌شوید (متنی و گرافیکی).

۳. در پنجره دو کادر و یک دکمه بسازید و با کلیک، دو متن را با خط تیره به هم بچسبانید.

> **راهنمایی:** برای تمرین ۳ مثال ۹ را ببینید و به جای `' ا هل '` از `' - '` استفاده کنید.

تمرین در خانه

۱. برنامه متنی بنویسید که سال تولد را بگیرد و سن را حساب کند (سال جاری را عدد ثابت بگیرید).

۲. برنامه‌ای بنویسید که طول و عرض مستطیل را بگیرد و مساحت و محیط را چاپ کند.

۳. نسخه گرافیکی تمرین ۲ را بسازید.

۴. در پنجره نام کاربر را بگیرید و با دکمه «سلام» پیام خوشامدگویی نشان دهید. دکمه دوم «پاک کردن» کادر و پیام را خالی کند.

۵. **چالش:** دما را به سانتی‌گراد بگیرید و به فارنهایت تبدیل کنید. فرمول: `F = C * 9 / 5 + 32`.

> **راهنمایی:** برای پاک کردن کادر از `entry.delete(0, tk.END)` استفاده کنید و متن برچسب را با `config(text='')` خالی کنید.

درس ۳: عملیات ریاضی

هدف‌ها

- استفاده از عملگرهای `+`، `-`، `*`، `/`، `//`، `%` و `**`
- شناخت اولویت عملگرها و استفاده از پرانتز
- استفاده از `round`، `abs` و کتابخانه `math`
- ساخت ماشین‌حساب متنی و گرافیکی (اسنیپت `izehadd`)

مفهوم: عملگرهای ریاضی

پایتون مثل یک ماشین‌حساب قوی است. عملگرهای اصلی:

- `+` جمع، `-` تفریق، `*` ضرب، `/` تقسیم (نتیجه همیشه اعشاری)
- `//` تقسیم صحیح (فقط خارج‌قسمت)
- `%` باقی‌مانده تقسیم
- `**` توان

مثال ۱: عملیات پایه

```
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(2 ** 10)
```

خروجی:

```
10
4
21
3.5
3
1
1024
```

> **یادآوری:** `7 / 2` برابر `3.5` است ولی `7 // 2` برابر `3` است. اولی دقیق و دومی فقط خارج‌قسمت صحیح را می‌دهد.

مثال ۲: اولویت عملگرها

پایتون مثل ریاضی ابتدا توان، بعد ضرب و تقسیم و در آخر جمع و تفریق را حساب می‌کند. پرانتز بالاترین اولویت را دارد.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 * 2)
print(10 - 4 - 3)
```

خروجی:

```
14
20
16
3
```

> **نکته:** اگر شک دارید، پرانتز بگذارید. هم نتیجه امن‌تر است و هم کد خواناتر.

مثال ۳: زوج و فرد با باقی‌مانده

اگر باقی‌مانده تقسیم عدد بر ۲ صفر باشد، عدد زوج است. این ترفند در برنامه‌نویسی خیلی پرکاربرد است.

```
n = int(input('یک عدد: '))
print('باقی‌مانده بر ۲:', n % 2)
```

خروجی:

```
13: یک عدد
1: باقی‌مانده بر ۲
```

مثال ۴: تقسیم کالا: خارج‌قسمت و باقی‌مانده

۲۳ شکلات را بین ۵ نفر تقسیم می‌کنیم.

```
total = 23
people = 5
print('به هر نفر', total // people, 'تا می‌رسد')
print('باقی‌مانده:', total % people)
```

خروجی:

```
به هر نفر 4 تا می‌رسد
3: باقی‌مانده
```

مثال ۵: گرد کردن و قدر مطلق

```
print(round(3.14159, 2))
print(round(2.5))
print(abs(-12))
print(max(4, 9, 2))
print(min(4, 9, 2))
```

خروجی:

```
3.14
2
12
9
2
```

هشدار: `round(2.5)` برابر `2` می‌شود، نه ۳. پایتون عدد ممیزدار نیمه را به نزدیک‌ترین عدد زوج گرد می‌کند.

مثال ۶: کتابخانه math

برای ریشه دوم، عدد پی و ... باید کتابخانه `math` را وارد کنیم.

```
import math


print(math.sqrt(49))
print(math.pi)
print(math.ceil(4.1))
print(math.floor(4.9))
```

خروجی:

```
7.0
3.141592653589793
5
4
```

مثال ۷: ماشین‌حساب متنی

```
a = float(input('عدد اول: '))
b = float(input('عدد دوم: '))
print('جمع:', a + b)
print('تفاضل:', a - b)
print('ضرب:', a * b)
print('تقسیم:', a / b)
```

خروجی:

```
10: عدد اول
4: عدد دوم
14.0: جمع
6.0: تفاضل
40.0: ضرب
2.5: تقسیم
```

هشدار: اگر عدد دوم صفر باشد، برنامه خطای `ZeroDivisionError` می‌دهد. جلوی آن را در درس ۵ و ۱۱ می‌گیریم.

مثال ۸: ماشین‌حساب گرافیکی با چهار دکمه

یک تابع جدا برای هر عمل می‌نویسیم و هر دکمه یکی را صدا می‌زند.

```
import tkinter as tk

win = tk.Tk()
win.title('ماشین‌حساب')
win.geometry('300x300')

tk.Label(win, text='عدد اول:').pack()
e1 = tk.Entry(win)
e1.pack()
tk.Label(win, text='عدد دوم:').pack()
e2 = tk.Entry(win)
e2.pack()
out = tk.Label(win, text='نتیجه', font=('Tahoma', 14))
out.pack(pady=10)

def add():
    out.config(text=float(e1.get()) + float(e2.get()))

def sub():
    out.config(text=float(e1.get()) - float(e2.get()))

def mul():
    out.config(text=float(e1.get()) * float(e2.get()))

def div():
    out.config(text=float(e1.get()) / float(e2.get()))

tk.Button(win, text='+', width=4, command=add).pack(side='left', padx=8)
tk.Button(win, text='-', width=4, command=sub).pack(side='left', padx=8)
tk.Button(win, text='×', width=4, command=mul).pack(side='left', padx=8)
```

```
tk.Button(win, text='÷', width=4, command=div).pack(side='left', padx=8)
win.mainloop()
```

مثال ۹: میانگین سه نمره

```
a = float(input('نمره اول: '))
b = float(input('نمره دوم: '))
c = float(input('نمره سوم: '))
avg = (a + b + c) / 3
print('میانگین:', round(avg, 2))
```

خروجی:

```
18: نمره اول
15: نمره دوم
17: نمره سوم
16.67: میانگین
```

مثال ۱۰: محاسبه تخفیف در پنجره

قیمت و درصد تخفیف را می‌گیریم و قیمت نهایی را نشان می‌دهیم.

```
import tkinter as tk


win = tk.Tk()
win.title('تخفیف')
win.geometry('300x260')
tk.Label(win, text='قیمت (تومان):').pack()
e_price = tk.Entry(win)
e_price.pack()
tk.Label(win, text='درصد تخفیف:').pack()
e_off = tk.Entry(win)
e_off.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def calc():
    price = float(e_price.get())
    off = float(e_off.get())
    final = price - price * off / 100
    out.config(text=f'قیمت نهایی: {final:.0f} تومان')


tk.Button(win, text='محاسبه', command=calc).pack()
win.mainloop()
```

نکته: در `f'{final:.0f}'` عبارت `:.0f` یعنی عدد بدون رقم اعشار نمایش داده شود. `:.2f` دو رقم اعشار می‌دهد.

مثال ۱۱: شاخص توده بدنی (BMI)

```
w = float(input('وزن (کیلوگرم): '))
h = float(input('قد (متر): '))
bmi = w / h ** 2
print(f'BMI شما: {bmi:.1f}')
```

خروجی:

```
وزن (کیلوگرم): 70
قد (متر): 1.75
BMI شما: 22.9
```

> **کوییز:** وقت سنجش یادگیری است.

کوییز

س۱. حاصل `17 // 5` چیست؟

الف) ۳/۴

ب) ۳

ج) ۲

د) ۴

س۲. حاصل `17 % 5` چیست؟

الف) ۳

ب) ۲

ج) ۳/۴

د) ۱۲

س۳. حاصل `2 + 3 * 2` چیست؟

الف) ۱۰

ب) ۸

ج) ۷

د) ۱۲

س۴. حاصل `5 ** 2` چیست؟

الف) ۱۰

ب) ۷

ج) ۲۵

د) ۳۲

س۵. کدام دستور ریشه دوم ۱۶ را می‌دهد؟

الف) `sqrt(16)`

ب) `math.sqrt(16)`

ج) `16 ** math`

د) `math.root(16)`

تمرین کلاسی

۱. دو عدد بگیرید و حاصل جمع، ضرب و باقی‌مانده تقسیم اولی بر دومی را چاپ کنید.

۲. در ماشین‌حساب گرافیکی دکمه «توان» اضافه کنید.

۳. برنامه‌ای بنویسید که تعداد ثانیه را بگیرد و بگوید چند دقیقه و چند ثانیه است.

> **راهنمایی:** برای تمرین ۳ از `//` و `%` با عدد ۶۰ استفاده کنید.

تمرین در خانه

۱. برنامه متنی بنویسید که شعاع دایره را بگیرد و مساحت و محیط را چاپ کند (`math.pi`).

۲. برنامه‌ای که مبلغ صورتحساب و درصد انعام را بگیرد و جمع کل را نشان دهد.

۳. تعداد روز را بگیرید و بگویید چند هفته و چند روز است.

۴. نسخه گرافیکی تمرین ۱ را بسازید.

۵. در پنجره، دو ضلع قائم یک مثلث قائم‌الزاویه را بگیرید و وتر را با `math.sqrt` حساب کنید.

۶. **چالش:** مبلغی را به تومان بگیرید و بگویید با اسکناس‌های ۵۰۰۰۰، ۱۰۰۰۰ و ۵۰۰۰ چند عدد لازم است (از `//` و `%` استفاده کنید).

راهنمایی: برای چالش اول با `//` تعداد اسکناس بزرگ را حساب کنید، بعد با `%` باقی‌مانده را بردارید و همین کار را برای اسکناس بعدی تکرار کنید.

درس ۴: رشته‌ها و پیام‌ها

هدف‌ها

- چسباندن و تکرار متن با `+` و `*`
- اندیس‌گذاری و برش رشته (`[0]`، `[-1]`، `[1:4]`)
- استفاده از `len`، `upper`، `lower`، `strip`، `replace`، `find`، `count`، `split` و `join`
- ساختن متن با `f-string`
- نمایش پیام در پنجره با `messagebox` (اسنیپت `tirMsg`)

مفهوم: رشته

هر متن در پایتون یک **رشته** (`str`) است، یعنی دنباله‌ای از کاراکترها. هر کاراکتر شماره‌ای دارد به نام **اندیس** که از صفر شروع می‌شود. اندیس منفی از آخر شمارش می‌کند: `-1` آخرین کاراکتر است.

مثال ۱: چسباندن و تکرار (اسنیپت izehconcat)

```
first = 'علی'
last = 'رضایی'
full = first + ' ' + last
print(full)
print('-' * 10)
print('ها' * 3)
```

خروجی:

```
علی رضایی
----------
ها ها ها
```

> **هشدار:** نمی‌شود متن را با عدد جمع کرد: `'سن: ' + 15` خطا می‌دهد. باید بنویسید `'سن: ' + str(15)` یا از `f-string` استفاده کنید.

مثال ۲: اندیس و برش

```
s = 'Python'
print(s[0])
print(s[-1])
print(s[1:4])
print(s[:2])
print(s[2:])
print(s[::-1])
```

خروجی:

```
P
n
yth
Py
thon
nohtyP
```

> **یادآوری:** در برش `s[1:4]` شروع شامل می‌شود ولی پایان نه، یعنی کاراکترهای اندیس ۱ تا ۳. `[::-1]` رشته را برعکس می‌کند.

مثال ۳: طول و حروف بزرگ و کوچک

```
text = 'Hello World'
print(len(text))
print(text.upper())
print(text.lower())
print(text.title())
print(text.swapcase())
```

خروجی:

```
11
HELLO WORLD
hello world
Hello World
hELLO wORLD
```

مثال ۴: پاک کردن فاصله‌های اضافی

متد `strip` فاصله‌های ابتدا و انتهای متن را حذف می‌کند. برای پاکسازی ورودی کاربر بسیار مفید است.

```
name = '   سارا   '
print('[' + name + ']')
print('[' + name.strip() + ']')
```

خروجی:

```
[   سارا   ]
[سارا]
```

مثال ۵: جستجو و جایگزینی

```
s = 'من پایتون را دوست دارم، پایتون عالی است'
print(s.count('پایتون'))
print(s.find('دوست'))
print(s.replace('پایتون', 'برنامه‌نویسی'))
print('عالی' in s)
print(s.startswith('من'))
```

خروجی:

```
2
13
من برنامه‌نویسی را دوست دارم، برنامه‌نویسی عالی است
True
True
```

> **نکته:** `find` اگر متن را پیدا نکند `-1` برمی‌گرداند. `in` جواب `True` یا `False` می‌دهد.

مثال ۶: جدا کردن و دوباره ساختن

`split` یک رشته را با جداکننده به لیستی از قطعه‌ها تبدیل می‌کند و `join` برعکس عمل می‌کند.

```
sentence = 'امروز هوا خیلی خوب است'
words = sentence.split()
print(words)
print(len(words))
print('-'.join(words))
```

خروجی:

```
['امروز', 'هوا', 'خیلی', 'خوب', 'است']
5
امروز-هوا-خیلی-خوب-است
```

مثال ۷: f-string

با گذاشتن `f` قبل از نقل‌قول، می‌توانیم متغیرها و عبارت‌ها را داخل `{}` بنویسیم.

```
name = 'علی'
score = 18.5
print(f'{name} نمره {score} گرفت')
print(f'دو برابر نمره: {score * 2}')
print(f'نمره با یک اعشار: {score:.1f}')
```

خروجی:

```
علی نمره 18.5 گرفت
دو برابر نمره: 37.0
نمره با یک اعشار: 18.5
```

مثال ۸: متن چندخطی

```
poem = '''خط اول
خط دوم
خط سوم'''
print(poem)
print(len(poem.split('\n')), 'خط')
```

خروجی:

```
خط اول
خط دوم
خط سوم
خط 3
```

مثال ۹: پیام ساده در پنجره

ماژول `messagebox` پنجره‌های پیام آماده دارد: `showinfo` اطلاعات، `showwarning` هشدار و `showerror` خطا.

```
import tkinter as tk
from tkinter import messagebox

win = tk.Tk()
win.geometry('280x200')
entry = tk.Entry(win)
entry.pack(pady=15)

def show():
    t = entry.get()
    messagebox.showinfo('نتیجه', f'{t.upper()} - طول: {len(t)}')

tk.Button(win, text='بررسی', command=show).pack()
win.mainloop()
```

مثال ۱۰: سه نوع پیام

```
import tkinter as tk
from tkinter import messagebox

win = tk.Tk()
win.geometry('260x200')
tk.Button(win, text='اطلاعات',
          command=lambda: messagebox.showinfo('اطلاعات', 'ذخیره
شد')).pack(pady=8)
tk.Button(win, text='هشدار',
          command=lambda: messagebox.showwarning('هشدار', 'فضا کم
است')).pack(pady=8)
tk.Button(win, text='خطا',
          command=lambda: messagebox.showerror('خطا', 'فایل پیدا
نشد')).pack(pady=8)
win.mainloop()
```

نکته: `lambda:` یک تابع کوتاه بی‌نام می‌سازد. وقتی فقط یک خط کار داریم به‌جای `def` از آن استفاده می‌کنیم.

مثال ۱۱: شمارنده زنده حروف

با `bind` می‌توان کاری کرد که با هر بار تایپ، تابع اجرا شود.

```
import tkinter as tk

win = tk.Tk()
win.title('شمارنده')
win.geometry('300x160')
entry = tk.Entry(win, width=30)
entry.pack(pady=15)
lbl = tk.Label(win, text='تعداد حروف: 0')
lbl.pack()

def count(event):
    lbl.config(text='تعداد حروف: ' + str(len(entry.get())))

entry.bind('<KeyRelease>', count)
win.mainloop()
```

مثال ۱۲: برعکس کردن متن

```
import tkinter as tk

win = tk.Tk()
win.geometry('300x180')
entry = tk.Entry(win, width=30)
entry.pack(pady=10)
out = tk.Label(win, text='', font=('Tahoma', 14))
out.pack(pady=10)

def reverse():
    out.config(text=entry.get()[::-1])

tk.Button(win, text='برعکس کن', command=reverse).pack()
win.mainloop()
```

> **کوییز:** وقت سنجش یادگیری است.

کوییز

س۱. خروجی `'Python'[1]` چیست؟

الف) `P`

ب) `y`

ج) `t`

د) `n`

س۲. خروجی `'abcdef'[2:4]` چیست؟

الف) `cd`

ب) `bc`

ج) `cde`

د) `bcd`

س۳. کدام متد فاصله‌های ابتدا و انتهای رشته را حذف می‌کند؟

الف) `trim()`

ب) `cut()`

ج) `strip()`

د) `clean()`

س۴. خروجی `'a-b-c'.split('-')` چیست؟

الف) `'abc'`

ب) `['a', 'b', 'c']`

ج) `'a b c'`

د) `3`

س۵. کدام پنجره پیام خطا را نشان می‌دهد؟

الف) `messagebox.showinfo`

ب) `messagebox.showwarning`

ج) `messagebox.showerror`

د) `messagebox.showmessage`

تمرین کلاسی

۱. نام کامل را بگیرید و با حروف بزرگ و با طول آن چاپ کنید.

۲. جمله‌ای بگیرید و تعداد کلمه‌های آن را بشمارید.

۳. در پنجره، متن را بگیرید و با دکمه «چاپ حروف کوچک» نتیجه را در `messagebox` نشان دهید.

> **راهنمایی:** برای تمرین ۲ از `split` و `len` با هم استفاده کنید.

تمرین در خانه

۱. برنامه‌ای بنویسید که نام و نام خانوادگی را جدا بگیرد و آنها را به شکل «خانوادگی، نام» چاپ کند.

۲. کلمه‌ای بگیرید و اولین و آخرین حرف آن را چاپ کنید.

۳. جمله‌ای بگیرید و همه فاصله‌های آن را با خط تیره عوض کنید.

۴. در پنجره، دو کادر برای نام و نام خانوادگی بسازید و با دکمه، پیام «سلام نام نام‌خانوادگی عزیز» را در `messagebox` نشان دهید.

۵. شمارنده زنده را طوری تغییر دهید که تعداد کلمه‌ها را هم نشان دهد.

۶. **چالش:** برنامه‌ای که یک کلمه بگیرد و نشان دهد کلمه معکوس آن با خودش برابر است یا نه (مثل «ملم»). نتیجه را با `==` مقایسه کنید و `True` یا `False` چاپ کنید.

> **راهنمایی:** برای چالش از `word[::-1]` و مقایسه `word == word[::-1]` استفاده کنید.

درس ۵: تصمیم‌گیری با if

هدف‌ها

- شناخت مقدارهای `True` و `False` و عملگرهای مقایسه
- نوشتن `if`، `elif` و `else`
- ترکیب شرط‌ها با `and`، `or` و `not`
- رعایت فرورفتگی (ایندنت)
- ساختن فرم ورود و بررسی نمره در پنجره

مفهوم: شرط

برنامه‌ها باید بتوانند تصمیم بگیرند. با `if` می‌گوییم «اگر این شرط درست بود، این کار را انجام بده». شرط یک عبارت است که جواب آن `True` (درست) یا `False` (غلط) می‌شود.

عملگرهای مقایسه: `==` مساوی، `!=` نامساوی، `>` بزرگ‌تر، `<` کوچک‌تر، `>=` بزرگ‌تر یا مساوی، `<=` کوچک‌تر یا مساوی.

> **هشدار:** برای مقایسه از دو مساوی `==` استفاده کنید. یک مساوی `=` برای دادن مقدار به متغیر است و در شرط خطای `SyntaxError` می‌دهد.

مثال ۱: مقایسه و مقدار بولی

```
print(5 > 3)
print(5 == 5)
print(5 != 5)
print('a' == 'A')
print(10 <= 9)
```

خروجی:

```
True
True
False
False
False
```

مثال ۲: اولین if

```
age = int(input('سن شما؟ '))
if age >= 18:
    print('شما بزرگسال هستید')
print('پایان برنامه')
```

خروجی:

```
سن شما؟ 20
شما بزرگسال هستید
پایان برنامه
```

> **هشدار:** بعد از شرط حتماً دونقطه `:` بگذارید و خط‌های داخل `if` را با ۴ فاصله (یا یک Tab) جلوتر بنویسید. خط `print('پایان برنامه')` چون جلو نرفته، همیشه اجرا می‌شود.

مثال ۳: if و else

```
n = int(input('یک عدد: '))
if n % 2 == 0:
    print('زوج')
else:
    print('فرد')
```

خروجی:

```
7: یک عدد
فرد
```

مثال ۴: elif برای چند حالت

شرط‌ها از بالا به پایین بررسی می‌شوند و با اولین شرط درست، بقیه نادیده گرفته می‌شوند.

```
score = float(input('نمره: '))
if score >= 17:
    print('عالی')
elif score >= 14:
    print('خوب')
elif score >= 10:
    print('قبول')
else:
    print('مردود')
```

خروجی:

```
15: نمره
خوب
```

مثال ۵: ترکیب شرط‌ها با and و or

- `and`: وقتی هر دو شرط درست باشند، درست است.
- `or`: اگر حداقل یکی درست باشد، درست است.
- `not`: جواب را برعکس می‌کند.

```
age = int(input('سن: '))
has_ticket = input('بلیت دارید؟ (خیر/بله) ') == 'بله'


if age >= 12 and has_ticket:
    print('وارد شوید')
elif not has_ticket:
    print('ابتدا بلیت بگیرید')
else:
    print('این فیلم برای شما مناسب نیست')
```

خروجی:

```
15: سن
بلیت دارید؟ )بله/خیر (بله
وارد شوید
```

مثال ۶: بزرگ‌ترین از سه عدد

```
a = float(input('عدد اول: '))
b = float(input('عدد دوم: '))
c = float(input('عدد سوم: '))


if a >= b and a >= c:
    print('بزرگترین:', a)
elif b >= c:
    print('بزرگترین:', b)
else:
    print('بزرگترین:', c)
```

خروجی:

```
4: عدد اول
9: عدد دوم
7: عدد سوم
9.0: بزرگترین
```

مثال ۷: سال کبیسه (شرط تو در تو)

سال میلادی کبیسه است اگر بر ۴ بخش‌پذیر باشد، مگر اینکه بر ۱۰۰ بخش‌پذیر باشد و بر ۴۰۰ نباشد.

```
year = int(input('سال: '))
if year % 400 == 0:
    print('کبیسه')
elif year % 100 == 0:
    print('غیر کبیسه')
elif year % 4 == 0:
    print('کبیسه')
else:
    print('غیر کبیسه')
```

خروجی:

```
2024: سال
کبیسه
```

مثال ۸: رمز عبور

```
password = '1234'
entered = input('رمز را وارد کنید: ')
if entered == password:
    print('خوش آمدید')
else:
    print('رمز اشتباه است')
```

خروجی:

```
1234: رمز را وارد کنید
خوش آمدید
```

مثال ۹: نمره در پنجره با رنگ

```
import tkinter as tk


win = tk.Tk()
win.title('نمره')
win.geometry('260x200')
entry = tk.Entry(win)
entry.pack(pady=10)
lbl = tk.Label(win, text='', font=('Tahoma', 16))
lbl.pack(pady=5)


def check():
    score = float(entry.get())
    if score >= 10:
        lbl.config(text='قبول', fg='green')
    else:
        lbl.config(text='مردود', fg='red')


tk.Button(win, text='بررسی', command=check).pack()
win.mainloop()
```

مثال ۱۰: کارنامه با چند حالت

```
import tkinter as tk


win = tk.Tk()
win.title('کارنامه')
win.geometry('260x200')
entry = tk.Entry(win)
entry.pack(pady=10)
lbl = tk.Label(win, text='', font=('Tahoma', 16))
lbl.pack(pady=5)


def check():
    s = float(entry.get())
    if s >= 17:
        lbl.config(text='عالی', fg='darkgreen')
    elif s >= 14:
        lbl.config(text='خوب', fg='blue')
    elif s >= 10:
        lbl.config(text='قبول', fg='orange')
    else:
        lbl.config(text='مردود', fg='red')


tk.Button(win, text='بررسی', command=check).pack()
win.mainloop()
```

مثال ۱۱: فرم ورود

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.title('ورود')
win.geometry('260x220')
tk.Label(win, text='نام کاربری:').pack()
e_user = tk.Entry(win)
e_user.pack()
tk.Label(win, text='رمز عبور:').pack()
e_pass = tk.Entry(win, show='*')
e_pass.pack()


def login():
    if e_user.get() == 'ali' and e_pass.get() == '1234':
        messagebox.showinfo('ورود', 'خوش آمدید')
    else:
        messagebox.showerror('خطا', 'نام کاربری یا رمز اشتباه است')


tk.Button(win, text='ورود', command=login).pack(pady=12)
win.mainloop()
```

نکته: `show='*'` باعث می‌شود حروف رمز به شکل ستاره نمایش داده شوند.

مثال ۱۲: ساعت و پیام روز

```
hour = int(input('ساعت (۰ تا ۲۳): '))
if hour < 12:
    print('صبح بخیر')
elif hour < 18:
    print('ظهر بخیر')
else:
    print('شب بخیر')
```

خروجی:

```
ساعت (۰ تا ۲۳): 9
صبح بخیر
```

یادآوری: ورودی `input` رشته است. برای مقایسه با عدد حتماً `int()` یا `float()` بزنید.

کوییز: وقت سنجش یادگیری است.

کوییز

س۱. خروجی `print(7 != 7)` چیست؟

الف) `True`

ب) `False`

ج) `7`

د) خطا

س۲. کدام شرط درست نوشته شده است؟

الف) `if x = 5:`

ب) `if x == 5`

ج) `if x == 5:`

د) `if (x == 5)`

س۳. اگر `x = 8` باشد، `x > 5 and x < 7` چه می‌شود؟

الف) `True`

ب) `False`

ج) `8`

د) خطا

س۴. در یک ساختار `if / elif / else` اگر شرط اول درست باشد چه می‌شود؟

الف) همه شرط‌ها بررسی می‌شوند

ب) فقط `else` اجرا می‌شود

ج) بقیه شرط‌ها بررسی نمی‌شوند

د) برنامه خطا می‌دهد

س۵. `not (3 > 5)` چیست؟

الف) `True`

ب) `False`

ج) `3`

د) `5`

تمرین کلاسی

۱. عددی بگیرید و بگویید مثبت، منفی یا صفر است.

۲. سن را بگیرید: زیر ۱۲ «کودک»، ۱۲ تا ۱۷ «نوجوان»، ۱۸ به بالا «بزرگسال».

۳. فرم ورود را طوری تغییر دهید که نام کاربری و رمز دلخواه شما باشد.

> **راهنمایی:** برای تمرین ۲ از `elif age < 18` استفاده کنید. لازم نیست `age >= 12` را دوباره بنویسید، چون اگر به آن شرط برسد حتماً بزرگ‌تر از ۱۲ است.

تمرین در خانه

۱. برنامه‌ای که دو عدد بگیرد و بگوید کدام بزرگ‌تر است یا مساوی‌اند.

۲. برنامه‌ای که عددی بگیرد و بگوید بر ۳ بخش‌پذیر است یا نه.

۳. برنامه‌ای که سه ضلع بگیرد و بگوید مثلث متساوی‌الاضلاع، متساوی‌الساقین یا مختلف‌الاضلاع است.

۴. نسخه گرافیکی تمرین ۲ را با نمایش جواب در برچسب بسازید.

۵. فرم ورود را طوری تغییر دهید که اگر کادر نام کاربری خالی بود پیام هشدار بدهد.

۶. **چالش:** بازی «حدس عدد»: عدد مخفی ۷ است. از کاربر عدد بگیرید و بگویید «بزرگ‌تر»، «کوچک‌تر» یا «درست».

راهنمایی: برای تمرین ۵ شرط `if e_user.get() == '':` را به اولین شرط اضافه کنید و با `showwarning` پیام دهید.

درس ۶: حلقه‌ها

هدف‌ها

- تکرار کار با `for` و `range`
- تکرار تا برقراری شرط با `while`
- استفاده از `break` و `continue`
- جمع‌زدن و شمارش با حلقه
- نمایش نتیجه حلقه در `Listbox` (اسنیپت `tirLB`)

مفهوم: حلقه

فرض کنید باید «سلام» را صد بار چاپ کنید. نوشتن صد خط `print` کار درستی نیست. حلقه به ما اجازه می‌دهد یک کار را هر چند بار که بخواهیم تکرار کنیم.

دو نوع حلقه داریم: `for` وقتی تعداد تکرار مشخص است و `while` وقتی تا برقرار بودن یک شرط تکرار می‌کنیم.

مثال ۱: اولین for

```
for i in range(5):
    print('سلام', i)
```

خروجی:

```
0 سلام
1 سلام
2 سلام
3 سلام
4 سلام
```

> **یادآوری:** `range(5)` اعداد ۰ تا ۴ را می‌سازد، یعنی پنج عدد که از صفر شروع می‌شوند و به ۵ نمی‌رسند.

مثال ۲: شکل‌های مختلف range

```
for i in range(1, 6):
    print(i, end=' ')
print()
for i in range(2, 11, 2):
    print(i, end=' ')
print()
for i in range(5, 0, -1):
    print(i, end=' ')
```

خروجی:

```
1 2 3 4 5
2 4 6 8 10
5 4 3 2 1
```

> **نکته:** `range(شروع, پایان, گام)` پایان هیچ‌وقت شامل نمی‌شود. گام منفی عددها را شمارش معکوس می‌کند.

مثال ۳: جدول ضرب

```
n = int(input('جدول ضرب چه عددی؟ '))
for i in range(1, 11):
    print(f'{n} x {i} = {n * i}')
```

خروجی:

```
جدول ضرب چه عددی؟ 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

مثال ۴: جمع اعداد ۱ تا ۱۰۰

یک متغیر **انباشتگر** (مثلاً `total`) می‌سازیم و در هر دور چیزی به آن اضافه می‌کنیم.

```
total = 0
for i in range(1, 101):
    total += i
print('جمع:', total)
```

خروجی:

```
جمع: 5050
```

مثال ۵: حلقه روی رشته

```
for ch in 'Python':
    print(ch)
```

خروجی:

```
P
y
t
h
o
n
```

مثال ۶: مثلث ستاره با حلقه

مثلثی را که در درس اول دستی می‌کشیدیم، حالا با حلقه می‌کشیم.

```
for i in range(1, 6):
    print(' ' * (5 - i) + '*' * (2 * i - 1))
```

خروجی:

```
    *
   ***
  *****
 *******
*********
```

مثال ۷: اولین while

```
count = 1
while count <= 3:
    print('دور', count)
    count += 1
print('تمام شد')
```

خروجی:

```
دور 1
دور 2
دور 3
تمام شد
```

هشدار: اگر داخل `while` مقدار شرط را تغییر ندهید (مثلاً `count += 1` را فراموش کنید)، حلقه بی‌نهایت می‌شود. با Ctrl+C در ترمینال آن را متوقف کنید.

مثال ۸: تکرار تا رمز درست

```
password = '1234'
entered = ''
while entered != password:
    entered = input('رمز: ')
print('وارد شدید')
```

خروجی:

```
رمز: 1111
رمز: 1234
وارد شدید
```

مثال ۹: break و continue

`break` حلقه را کاملاً قطع می‌کند و `continue` فقط دور جاری را رد می‌کند و به دور بعد می‌رود.

```
for i in range(1, 8):
    if i == 3:
        continue
    if i == 6:
        break
    print(i)
```

خروجی:

```
1
2
4
5
```

مثال ۱۰: منوی تکراری

```
while True:
    cmd = input('سلام یا خروج؟ ')
    if cmd == 'سلام':
        print('سلام به شما!')
    elif cmd == 'خروج':
        print('خداحافظ')
        break
    else:
        print('دستور نامعتبر')
```

خروجی:

```
سلام یا خروج؟ سلام
سلام به شما!
سلام یا خروج؟ خروج
خداحافظ
```

نکته: `while True:` یک حلقه بی‌پایان می‌سازد که فقط با `break` تمام می‌شود. در برنامه‌های منو بسیار رایج است.

مثال ۱۱: جدول ضرب در Listbox

```
import tkinter as tk


win = tk.Tk()
win.title('جدول ضرب')
win.geometry('240x300')
lb = tk.Listbox(win, width=22, height=11)
lb.pack(padx=20, pady=20)
n = 7
for i in range(1, 11):
    lb.insert(tk.END, f'{n} x {i} = {n * i}')
win.mainloop()
```

مثال ۱۲: جدول ضرب با عدد دلخواه

```
import tkinter as tk


win = tk.Tk()
win.title('جدول ضرب')
win.geometry('260x380')
entry = tk.Entry(win, width=10)
entry.pack(pady=10)
lb = tk.Listbox(win, width=22, height=11)
lb.pack(padx=20, pady=10)


def make():
    lb.delete(0, tk.END)
    n = int(entry.get())
    for i in range(1, 11):
        lb.insert(tk.END, f'{n} x {i} = {n * i}')


tk.Button(win, text='بساز', command=make).pack()
win.mainloop()
```

مثال ۱۳: شمارش معکوس

```
import tkinter as tk


win = tk.Tk()
win.geometry('240x180')
lbl = tk.Label(win, text='10', font=('Tahoma', 40))
lbl.pack(pady=20)
n = 10


def tick():
    global n
    n -= 1
    lbl.config(text=n)
    if n > 0:
        win.after(1000, tick)


win.after(1000, tick)
win.mainloop()
```

نکته: در پنجره نباید از `while` با `input` یا `time.sleep` استفاده کنید، چون پنجره قفل می‌شود. به‌جای آن از `win.after(میلی‌ثانیه, تابع)` استفاده می‌کنیم تا تابع بعد از مدتی دوباره صدا زده شود.

کوییز: وقت سنجش یادگیری است.

کوییز

س۱. `range(2, 6)` چه عددهایی را می‌سازد؟

الف) ۲ تا ۶

ب) ۲، ۳، ۴، ۵

ج) ۳ تا ۶

د) ۲، ۳، ۴

س۲. `for i in range(3): print(i)` چند بار چاپ می‌کند؟

الف) ۲

ب) ۳

ج) ۴

د) بی‌نهایت

س۳. کدام دستور حلقه را کاملاً قطع می‌کند؟

الف) `continue`

ب) `stop`

ج) `exit`

د) `break`

س۴. کدام حلقه بی‌پایان است؟

الف) `for i in range(5):`

ب) `while True:` (بدون `break`)

ج) `for ch in 'abc':`

د) `while count < 3:` (با `count += 1`)

س۵. در `for i in range(10, 0, -2)` اولین و آخرین عدد کدام‌اند؟

الف) ۱۰ و ۱

ب) ۱۰ و ۲

ج) ۹ و ۱

د) ۱۰ و ۰

تمرین کلاسی

۱. اعداد ۱ تا ۲۰ را چاپ کنید.

۲. فقط اعداد زوج ۱ تا ۵۰ را چاپ کنید.

۳. جدول ضرب را به‌جای ۱۰ تا ۱۲ ردیف نشان دهید.

> **راهنمایی:** برای تمرین ۲ می‌توانید از `range(2, 51, 2)` یا از `if i % 2 == 0` در حلقه استفاده کنید.

تمرین در خانه

۱. برنامه‌ای که عددی بگیرد و جمع اعداد ۱ تا آن عدد را چاپ کند.

۲. برنامه‌ای که فاکتوریل یک عدد را حساب کند (مثلاً `5! = 120`).

۳. برنامه‌ای که ۵ نمره بگیرد (با حلقه) و میانگین را نشان دهد.

۴. حلقه `while` که تا وارد کردن عدد منفی، عددها را بگیرد و جمعشان را چاپ کند.

۵. در پنجره، فاکتوریل را حساب کنید و در برچسب نشان دهید.

۶. **چالش:** همه اعداد اول کوچک‌تر از ۵۰ را پیدا کنید و در یک `Listbox` نشان دهید.

> **راهنمایی:** عدد `n` اول است اگر هیچ عددی از ۲ تا `n-1` آن را بدون باقی‌مانده تقسیم نکند. از یک متغیر `is_prime = True` و `break` استفاده کنید.

درس ۷: لیست‌ها

هدف‌ها

- ساختن لیست و دسترسی به عضوها با اندیس و برش
- افزودن، حذف و مرتب‌سازی با `append`، `insert`، `remove`، `pop` و `sort`
- پیمایش لیست با `for` و استفاده از `sum`، `min`، `max`، `len`
- کار با `random` برای انتخاب تصادفی
- مدیریت لیست در `Listbox` (اسنیپت‌های `izehlistops` و `tirLB`)

مفهوم: لیست

لیست یک مجموعه مرتب از مقدارهاست که در یک متغیر نگه‌داری می‌شود. با کروشه `[ ]` ساخته می‌شود و اعضا با کاما جدا می‌شوند. اندیس عضوها مثل رشته از صفر شروع می‌شود. برخلاف رشته، لیست را می‌توان تغییر داد.

مثال ۱: ساختن و خواندن لیست

```
fruits = ['سیب', 'موز', 'انار']
print(fruits)
print(fruits[0])
print(fruits[-1])
print(len(fruits))
```

خروجی:

```
['انار', 'موز', 'سیب']
سیب
انار
3
```

مثال ۲: تغییر عضو و برش

```
nums = [10, 20, 30, 40, 50]
nums[1] = 25
print(nums)
print(nums[1:4])
print(nums[:2])
print(nums[::-1])
```

خروجی:

```
[10, 25, 30, 40, 50]
[25, 30, 40]
[10, 25]
[50, 40, 30, 25, 10]
```

مثال ۳: افزودن عضو (اسنیپت izehlistops)

```
fruits = ['سیب', 'موز']
fruits.append('انار')
fruits.insert(0, 'گیلاس')
print(fruits)
fruits.extend(['هلو', 'کیوی'])
print(fruits)
```

خروجی:

```
['گیلاس', 'سیب', 'موز', 'انار']
['گیلاس', 'سیب', 'موز', 'انار', 'هلو', 'کیوی']
```

> **نکته:** append عضو را به انتها اضافه می‌کند، insert(اندیس, مقدار) در جای دلخواه می‌گذارد و extend چند عضو را با هم اضافه می‌کند.

مثال ۴: حذف عضو

```
colors = ['قرمز', 'آبی', 'سبز', 'زرد']
colors.remove('آبی')
print(colors)
last = colors.pop()
print(last, colors)
del colors[0]
print(colors)
```

خروجی:

```
['زرد', 'سبز', 'قرمز']
'] سبز', 'قرمز['  زرد
['سبز']
```

> **هشدار:** اگر با remove عضوی را که در لیست نیست حذف کنید، برنامه خطای ValueError می‌دهد. اول با in بررسی کنید.

مثال ۵: مرتب‌سازی

```
nums = [5, 2, 9, 1, 7]
nums.sort()
print(nums)
nums.sort(reverse=True)
print(nums)
nums.reverse()
print(nums)
print(sorted([3, 1, 2]))
```

خروجی:

```
[1, 2, 5, 7, 9]
[9, 7, 5, 2, 1]
[1, 2, 5, 7, 9]
[1, 2, 3]
```

> **یادآوری:** `sort()` خود لیست را تغییر می‌دهد، ولی `sorted()` یک لیست مرتب جدید می‌سازد و لیست اصلی را دست نمی‌زند.

مثال ۶: پیمایش لیست

```
fruits = ['سیب', 'موز', 'انار']
for f in fruits:
    print(f)
for i, f in enumerate(fruits, 1):
    print(i, f)
```

خروجی:

```
سیب
موز
انار
1 سیب
2 موز
3 انار
```

مثال ۷: آمار لیست

```
grades = [18, 15, 20, 12, 17]
print('جمع:', sum(grades))
print('بیشترین:', max(grades))
print('کمترین:', min(grades))
print('میانگین:', sum(grades) / len(grades))
```

خروجی:

```
82: جمع
20: بیشترین
12: کمترین
16.4: میانگین
```

مثال ۸: جستجو در لیست

```
names = ['علی', 'سارا', 'رضا']
name = input('نام: ')
if name in names:
    print('در لیست هست، جایگاه:', names.index(name))
else:
    print('در لیست نیست')
```

خروجی:

```
نام سارا:
در لیست هست، جایگاه 1:
```

مثال ۹: ساخت لیست با ورودی کاربر

```
nums = []
for i in range(3):
    nums.append(int(input('عدد: ')))
print(nums)
print('جمع:', sum(nums))
```

خروجی:

```
عدد 4:
عدد 8:
عدد 6:
[4, 8, 6]
جمع 18:
```

مثال ۱۰: انتخاب تصادفی

ماژول `random` به ما انتخاب و درهم‌ریختن تصادفی را می‌دهد.

```
import random


names = ['علی', 'سارا', 'رضا', 'مریم']
print(random.choice(names))
random.shuffle(names)
print(names)
print(random.randint(1, 6))
```

خروجی:

```
مریم
['سارا', 'مریم', 'علی', 'رضا']
4
```

> **نکته:** خروجی این مثال هر بار فرق می‌کند چون تصادفی است.

مثال ۱۱: لیست خرید در پنجره (افزودن و حذف)

```
import tkinter as tk


win = tk.Tk()
win.title('لیست خرید')
win.geometry('300x340')
entry = tk.Entry(win)
entry.pack(pady=8)
lb = tk.Listbox(win)
lb.pack(pady=5)


def add():
    if entry.get() != '':
        lb.insert(tk.END, entry.get())
        entry.delete(0, tk.END)


def remove():
    sel = lb.curselection()
    if sel:
        lb.delete(sel[0])


tk.Button(win, text='افزودن', command=add).pack(pady=3)
tk.Button(win, text='حذف انتخاب‌شده', command=remove).pack(pady=3)
win.mainloop()
```

نکته: `lb.curselection()` اندیس مورد انتخاب‌شده را برمی‌گرداند. اگر کاربر چیزی انتخاب نکرده باشد، نتیجه خالی است و شرط `if sel:` از خطا جلوگیری می‌کند.

مثال ۱۲: مرتب‌سازی و شمارش

```
import tkinter as tk


win = tk.Tk()
win.geometry('300x380')
entry = tk.Entry(win)
entry.pack(pady=8)
lb = tk.Listbox(win)
lb.pack(pady=5)
info = tk.Label(win, text='تعداد: 0')
info.pack()


def add():
    lb.insert(tk.END, entry.get())
    entry.delete(0, tk.END)
    info.config(text=f'تعداد: {lb.size()}')


def sort_list():
    items = sorted(lb.get(0, tk.END))
    lb.delete(0, tk.END)
    for it in items:
        lb.insert(tk.END, it)


tk.Button(win, text='افزودن', command=add).pack(pady=3)
tk.Button(win, text='مرتب‌سازی', command=sort_list).pack(pady=3)
win.mainloop()
```

مثال ۱۳: قرعه‌کشی

```
import random
import tkinter as tk


win = tk.Tk()
win.title('قرعه‌کشی')
win.geometry('280x240')
names = ['علی', 'سارا', 'رضا', 'مریم', 'نگین']
lbl = tk.Label(win, text='؟', font=('Tahoma', 28))
lbl.pack(pady=30)


def draw():
    lbl.config(text=random.choice(names))


tk.Button(win, text='قرعه بکش', command=draw).pack()
win.mainloop()
```

کوییز: وقت سنجش یادگیری است.

کوییز

س۱. اگر `a = [4, 5, 6]` باشد، `a[1]` چیست؟

الف) ۴

ب) ۵

ج) ۶

د) خطا

س۲. کدام متد عضو را به انتهای لیست اضافه می‌کند؟

الف) `add()`

ب) `push()`

ج) `append()`

د) `insert()`

س۳. بعد از `nums = [3, 1, 2]` و `nums.sort()` مقدار `nums` چیست؟

الف) `[3, 1, 2]`

ب) `[1, 2, 3]`

ج) `[2, 1, 3]`

د) `None`

س۴. `len([1, 2, [3, 4]])` چند است؟

الف) ۳

ب) ۴

ج) ۲

د) ۱

س۵. کدام دستور آخرین عضو لیست را حذف و برمی‌گرداند؟

الف) `remove()`

ب) `pop()`

ج) `del()`

د) `cut()`

تمرین کلاسی

۱. لیستی از پنج نام بسازید و آن را به ترتیب شماره‌دار چاپ کنید.

۲. پنج عدد از کاربر بگیرید و بزرگ‌ترین و کوچک‌ترین را چاپ کنید.

۳. لیست خرید (مثال ۱۱) را طوری تغییر دهید که تعداد اقلام را هم در یک برچسب نشان دهد.

> **راهنمایی:** برای تمرین ۳ مثال ۱۲ را ببینید. تعداد اقلام را با `lb.size()` می‌گیرید.

تمرین در خانه

۱. لیست اعداد زوج بین ۱ تا ۲۰ را بسازید و جمع آنها را چاپ کنید.

۲. برنامه‌ای که از کاربر چند عدد بگیرد (تا وارد کردن ۰) و آنها را مرتب چاپ کند.

۳. برنامه‌ای که لیستی از نمره‌ها را بگیرد و بگوید چند نفر بالای میانگین هستند.

۴. برنامه‌ای که لیست را بدون عضو تکراری کند (راهنمایی: حلقه و `in`).

۵. در پنجره، دکمه «حذف همه» به لیست خرید اضافه کنید.

۶. **چالش:** در پنجره دو لیست بسازید و با دو دکمه، عضو انتخاب‌شده را از یک لیست به دیگری منتقل کنید.

> **راهنمایی:** برای تمرین ۵ از `lb.delete(0, tk.END)` استفاده کنید. برای چالش، مقدار را با `lb.get(sel[0])` بخوانید، به لیست دیگر `insert` کنید و بعد از اولی `delete` کنید.

درس ۸: تابع‌ها

هدف‌ها

- تعریف و صدا زدن تابع با `def` (اسنیپت `izehfuncdef`)
- استفاده از پارامتر، آرگومان و مقدار پیش‌فرض
- برگرداندن نتیجه با `return`
- شناخت متغیر محلی و سراسری
- نوشتن یک تابع و استفاده از آن در برنامه متنی و گرافیکی

مفهوم: تابع

تابع یک تکه کد نام‌دار است که یک بار می‌نویسیم و هر چند بار بخواهیم صدا می‌زنیم. تابع‌هایی مثل `print`، `len` و `input` را قبلاً استفاده کرده‌ایم. حالا یاد می‌گیریم تابع خودمان را بنویسیم.

دلیل استفاده از تابع: کد تکراری ننویسیم، برنامه را مرتب کنیم و اگر اشتباهی بود فقط یک جا آن را درست کنیم.

مثال ۱: تابع ساده بدون ورودی

```
def say_hello():
    print('سلام!')
    print('به دنیای پایتون خوش آمدید')

say_hello()
say_hello()
```

خروجی:

```
سلام!
به دنیای پایتون خوش آمدید
سلام!
به دنیای پایتون خوش آمدید
```

> **هشدار:** فقط نوشتن `def` تابع را اجرا نمی‌کند. باید تابع را با نامش و پرانتز صدا بزنید: `say_hello()`.

مثال ۲: تابع با پارامتر

```
def greet(name):
    print('سلام', name)

greet('علی')
greet('سارا')
```

خروجی:

```
سلام علی
سلام سارا
```

> **یادآوری:** `name` پارامتر است (جای خالی در تعریف تابع) و `'علی'` آرگومان است (مقداری که هنگام صدا زدن می‌دهیم).

مثال ۳: بازگرداندن نتیجه با return

`return` مقدار را از تابع بیرون می‌دهد. با آن می‌توان نتیجه را در متغیر ذخیره کرد.

```
def area(w, h):
    return w * h

a = area(4, 5)
print('مساحت:', a)
print(area(2, 3) + area(1, 1))
```

خروجی:

```
20: مساحت
7
```

> **نکته:** تفاوت `print` و `return`: `print` فقط چیزی را نشان می‌دهد ولی `return` مقدار را برای استفاده در ادامه برنامه برمی‌گرداند. بعد از `return` تابع تمام می‌شود.

مثال ۴: مقدار پیش‌فرض

```
def power(base, exp=2):
    return base ** exp

print(power(5))
print(power(2, 10))
```

خروجی:

```
25
1024
```

مثال ۵: آرگومان با نام

```
def intro(name, city='تهران', age=15):
    print(f'{name} اهل {city} و {age} ساله است')


intro('علی')
intro('سارا', 'شیراز')
intro('رضا', age=20)
intro(age=12, name='مریم', city='اصفهان')
```

خروجی:

```
علی اهل تهران و 15 ساله است
سارا اهل شیراز و 15 ساله است
رضا اهل تهران و 20 ساله است
مریم اهل اصفهان و 12 ساله است
```

مثال ۶: تابع با شرط

```
def grade(score):
    if score >= 17:
        return 'عالی'
    elif score >= 10:
        return 'قبول'
    return 'مردود'


print(grade(18))
print(grade(12))
print(grade(7))
```

خروجی:

```
عالی
قبول
مردود
```

مثال ۷: چند مقدار برگرداندن

```
def min_max(nums):
    return min(nums), max(nums)


lo, hi = min_max([4, 9, 2, 7])
print('کمترین:', lo, '- بیشترین:', hi)
```

خروجی:

```
کمترین: 2 - بیشترین: 9
```

مثال ۸: متغیر محلی و سراسری

متغیری که داخل تابع ساخته می‌شود فقط همان‌جا وجود دارد (محلی). متغیر بیرون تابع در همه جا قابل دیدن است (سراسری).

```
x = 10


def show():
    y = 5
    print('داخل تابع:', x, y)


show()
print('بیرون:', x)
# print(y)   # خطا: y فقط داخل تابع وجود دارد
```

خروجی:

```
داخل تابع: 5 10
بیرون: 10
```

هشدار: اگر بخواهید داخل تابع مقدار یک متغیر سراسری را **تغییر** دهید، باید اول بنویسید `global x`. بدون آن، پایتون یک متغیر محلی جدید با همین نام می‌سازد.

مثال ۹: تابع‌هایی که با هم کار می‌کنند

```
def is_even(n):
    return n % 2 == 0


def evens_in(nums):
    result = []
    for n in nums:
        if is_even(n):
            result.append(n)
    return result


print(evens_in([1, 2, 3, 4, 5, 6]))
```

خروجی:

```
[2, 4, 6]
```

مثال ۱۰: یک تابع، دو برنامه

زیبایی تابع این است که یک بار نوشته می‌شود ولی هم در کنسول و هم در پنجره کار می‌کند.

```
def area(w, h):
    return w * h


w = float(input('عرض: '))
h = float(input('طول: '))
print('مساحت:', area(w, h))
```

خروجی:

```
4: عرض
5: طول
20.0: مساحت
```

```
import tkinter as tk


def area(w, h):
    return w * h


win = tk.Tk()
win.title('مساحت')
win.geometry('260x220')
tk.Label(win, text='عرض:').pack()
w = tk.Entry(win)
w.pack()
tk.Label(win, text='طول:').pack()
h = tk.Entry(win)
h.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def show():
    out.config(text=area(float(w.get()), float(h.get())))


tk.Button(win, text='مساحت', command=show).pack()
win.mainloop()
```

مثال ۱۱: تغییر رنگ پنجره با پارامتر

برای اینکه به تابع آرگومان بدهیم و کلیک دکمه آن را صدا بزند، از `lambda` استفاده می‌کنیم.

```
import tkinter as tk


win = tk.Tk()
win.title('رنگ‌ها')
win.geometry('300x200')


def paint(color):
    win.config(bg=color)


tk.Button(win, text='قرمز', command=lambda: paint('tomato')).pack(pady=5)
tk.Button(win, text='سبز', command=lambda: paint('lightgreen')).pack(pady=5)
tk.Button(win, text='آبی', command=lambda: paint('lightblue')).pack(pady=5)
win.mainloop()
```

مثال ۱۲: ماشین‌حساب با تابع کمکی

```
import tkinter as tk


win = tk.Tk()
win.title('ماشین‌حساب')
win.geometry('300x280')
tk.Label(win, text='عدد اول:').pack()
e1 = tk.Entry(win)
e1.pack()
tk.Label(win, text='عدد دوم:').pack()
e2 = tk.Entry(win)
e2.pack()
out = tk.Label(win, text='نتیجه', font=('Tahoma', 14))
out.pack(pady=10)


def calc(op):
    a = float(e1.get())
    b = float(e2.get())
    if op == '+':
        out.config(text=a + b)
    elif op == '-':
        out.config(text=a - b)
    elif op == '*':
        out.config(text=a * b)
    elif op == '/':
        out.config(text=a / b)


for symbol in ['+', '-', '*', '/']:
    tk.Button(win, text=symbol, width=4,
              command=lambda s=symbol: calc(s)).pack(side='left', padx=8)
win.mainloop()
```

> **نکته:** در `lambda s=symbol:` مقدار فعلی `symbol` را همان لحظه ذخیره می‌کنیم. بدون آن، همه دکمه‌ها به آخرین مقدار حلقه (یعنی `/`) اشاره می‌کردند.

مثال ۱۳: تابع بازگشتی (برای کنجکاوها)

تابعی که خودش را صدا می‌زند. فاکتوریل نمونه کلاسیک آن است.

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)

print(fact(5))
```

خروجی:

```
120
```

> **کوییز:** وقت سنجش یادگیری است.

کوییز

س۱. کدام کلمه برای تعریف تابع به کار می‌رود؟

الف) `function`

ب) `def`

ج) `fun`

د) `define`

س۲. خروجی کد زیر چیست؟ `def f(x): return x * 2` و `print(f(4))`

الف) ۴

ب) ۸

ج) `x * 2`

د) `None`

س۳. در `def hi(name='دوست'):` عبارت `'دوست'` چه نام دارد؟

الف) پارامتر اجباری

ب) مقدار پیش‌فرض

ج) خروجی

د) متغیر سراسری

س۴. اگر تابعی `return` نداشته باشد، چه مقداری برمی‌گرداند؟

الف) `0`

ب) `False`

ج) `None`

د) خطا می‌دهد

س۵. متغیری که داخل تابع ساخته می‌شود چه نام دارد؟

الف) سراسری

ب) محلی

ج) ثابت

د) رشته

تمرین کلاسی

۱. تابعی بنویسید که دو عدد بگیرد و بزرگ‌تر را برگرداند.

۲. تابعی بنویسید که نام بگیرد و پیام «سلام نام عزیز، روزت بخیر» را چاپ کند.

۳. تابع `area` را برای محاسبه محیط مستطیل هم تکمیل کنید و در پنجره دو دکمه بگذارید.

> **راهنمایی:** برای تمرین ۳ تابع `perimeter(w, h)` بنویسید و مثل مثال ۱۰ آن را به یک دکمه جدا وصل کنید.

تمرین در خانه

۱. تابع `is_prime(n)` بنویسید که `True` یا `False` برگرداند.

۲. تابع `average(nums)` که میانگین لیست را برگرداند. سپس با کمک آن میانگین نمره‌های کاربر را حساب کنید.

۳. تابع `celsius_to_f(c)` برای تبدیل دما و استفاده از آن در یک پنجره.

۴. تابع `count_vowels(text)` که تعداد حروف صدادار انگلیسی `aeiou` را بشمارد.

۵. نسخه گرافیکی تمرین ۱: عدد را بگیرد و با رنگ سبز «اول است» یا با رنگ قرمز «اول نیست» نشان دهد.

۶. **چالش:** تابع `fib(n)` که `n` عدد اول دنباله فیبوناچی را در یک لیست برگرداند و در `Listbox` نشان دهد.

> **راهنمایی:** برای `fib` با لیست `[0, 1]` شروع کنید و در هر دور مجموع دو عضو آخر را اضافه کنید.

درس ۹: دیکشنری، تاپل و انتخاب‌ها

هدف‌ها

- ساختن دیکشنری و خواندن، افزودن و حذف عضوها
- پیمایش دیکشنری با `keys`، `values` و `items`
- شناخت تاپل و مجموعه (`set`)
- استفاده از `Radiobutton` و `Checkbutton` (اسنیپت‌های `tirRB` و `tirCB`) و `Combobox`
- ساخت دفترچه تلفن با دیکشنری

مفهوم: دیکشنری

در لیست عضوها را با شماره پیدا می‌کنیم، ولی در **دیکشنری** هر مقدار را با یک **کلید** پیدا می‌کنیم، مثل یک دفترچه تلفن که با نام شخص شماره را پیدا می‌کنید. دیکشنری با آکولاد ساخته می‌شود و هر عضو به شکل `کلید: مقدار` است.

مثال ۱: ساختن و خواندن

```
student = {'نام': 'سارا', 'سن': 15, 'نمره': 19}
print(student)
print(student['نام'])
print(student.get('سن'))
print(student.get('شهر', 'نامشخص'))
```

خروجی:

```
{'نام': 'سارا', 'سن': 15, 'نمره': 19}
سارا
15
نامشخص
```

> **نکته:** اگر کلیدی وجود نداشته باشد، `student['شهر']` خطای `KeyError` می‌دهد ولی `student.get('شهر')` خطا نمی‌دهد و `None` (یا مقدار پیش‌فرضی که دادید) را برمی‌گرداند.

مثال ۲: افزودن، تغییر و حذف

```
student = {'نام': 'سارا', 'سن': 15}
student['شهر'] = 'تهران'
student['سن'] = 16
print(student)
del student['شهر']
print(student)
print(len(student))
```

خروجی:

```
{'نام': 'سارا', 'سن': 16, 'شهر': 'تهران'}
{'نام': 'سارا', 'سن': 16}
2
```

مثال ۳: پیمایش دیکشنری

```
student = {'نام': 'سارا', 'سن': 15, 'نمره': 19}
for key in student:
    print(key)
for value in student.values():
    print(value)
for key, value in student.items():
    print(key, ':', value)
```

خروجی:

```
نام
سن
نمره
سارا
15
19
نام : سارا
سن : 15
نمره : 19
```

مثال ۴: دیکشنری برای شمارش

دیکشنری جای خوبی برای شمارش تکرار است. می‌خواهیم ببینیم هر حرف چند بار آمده است.

```
word = 'banana'
counts = {}
for ch in word:
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

خروجی:

```
{'b': 1, 'a': 3, 'n': 2}
```

مثال ۵: دیکشنری تو در تو و لیست دیکشنری

```
students = [
    {'نام': 'علی', 'نمره': 17},
    {'نام': 'سارا', 'نمره': 20},
    {'نام': 'رضا', 'نمره': 14},
]
for s in students:
    print(s['نام'], s['نمره'])

best = max(students, key=lambda s: s['نمره'])
print('بهترین:', best['نام'])
```

خروجی:

```
علی 17
سارا 20
رضا 14
بهترین: سارا
```

مثال ۶: تاپل

تاپل مثل لیست است ولی بعد از ساخته شدن قابل تغییر نیست. با پرانتز ساخته می‌شود.

```
point = (3, 5)
print(point[0], point[1])
x, y = point
print(x + y)
# point[0] = 9   # خطا: تاپل تغییر نمی‌کند
```

خروجی:

```
3 5
8
```

> **یادآوری:** تاپل برای داده‌هایی که نباید عوض شوند (مثل مختصات یا رنگ) مناسب است. تابعی که چند مقدار برمی‌گرداند (درس ۸) در واقع یک تاپل برمی‌گرداند.

مثال ۷: مجموعه (set)

مجموعه عضو تکراری ندارد و ترتیب مهم نیست. برای حذف تکراری‌ها عالی است.

```
nums = [1, 2, 2, 3, 3, 3, 4]
unique = set(nums)
print(unique)
print(len(unique))
print(3 in unique)
```

خروجی:

```
{1, 2, 3, 4}
4
True
```

مثال ۸: دفترچه تلفن متنی

```
book = {}
while True:
    cmd = input('add / find / quit: ')
    if cmd == 'add':
        name = input('نام: ')
        book[name] = input('شماره: ')
    elif cmd == 'find':
        name = input('نام: ')
        print(book.get(name, 'پیدا نشد'))
    elif cmd == 'quit':
        break
```

خروجی:

```
add / find / quit: add
نام: علی
شماره: 0912
add / find / quit: find
نام: علی
0912
add / find / quit: quit
```

مثال ۹: انتخاب یکی از چند گزینه (Radiobutton)

همه رادیوها باید به **یک متغیر مشترک** وصل باشند. `StringVar` مقدار انتخاب‌شده را نگه می‌دارد.

```
import tkinter as tk


win = tk.Tk()
win.title('سطح')
win.geometry('260x240')
level = tk.StringVar(value='آسان')


tk.Label(win, text='سطح بازی را انتخاب کنید:').pack(pady=5)
for t in ['آسان', 'متوسط', 'سخت']:
    tk.Radiobutton(win, text=t, variable=level, value=t).pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def ok():
    out.config(text='انتخاب شما: ' + level.get())


tk.Button(win, text='تایید', command=ok).pack()
win.mainloop()
```

مثال ۱۰: گزینه بله/خیر (Checkbutton)

هر `Checkbutton` یک متغیر `BooleanVar` جدا دارد.

```
import tkinter as tk


win = tk.Tk()
win.title('سفارش')
win.geometry('260x260')
pizza = tk.BooleanVar()
drink = tk.BooleanVar()
salad = tk.BooleanVar()


tk.Checkbutton(win, text='پیتزا', variable=pizza).pack(anchor='w', padx=20)
tk.Checkbutton(win, text='نوشابه', variable=drink).pack(anchor='w', padx=20)
tk.Checkbutton(win, text='سالاد', variable=salad).pack(anchor='w', padx=20)
out = tk.Label(win, text='')
out.pack(pady=10)


def order():
    items = []
    if pizza.get():
        items.append('پیتزا')
    if drink.get():
        items.append('نوشابه')
    if salad.get():
        items.append('سالاد')
    out.config(text='سفارش: ' + ('، '.join(items) if items else 'خالی'))


tk.Button(win, text='ثبت سفارش', command=order).pack()
win.mainloop()
```

مثال ۱۱: لیست کشویی (Combobox)

```
import tkinter as tk
from tkinter import ttk


win = tk.Tk()
win.title('شهر')
win.geometry('260x160')
cities = ['تهران', 'شیراز', 'اصفهان', 'تبریز']
combo = ttk.Combobox(win, values=cities, state='readonly')
combo.current(0)
combo.pack(pady=15)
out = tk.Label(win, text='')
out.pack()


def show():
    out.config(text='شهر: ' + combo.get())


tk.Button(win, text='نمایش', command=show).pack(pady=10)
win.mainloop()
```

مثال ۱۲: دفترچه تلفن گرافیکی

```
import tkinter as tk
from tkinter import messagebox

book = {'علی': '0912', 'سارا': '0935'}

win = tk.Tk()
win.title('دفترچه تلفن')
win.geometry('280x320')
tk.Label(win, text='نام:').pack()
e_name = tk.Entry(win)
e_name.pack()
tk.Label(win, text='شماره:').pack()
e_num = tk.Entry(win)
e_num.pack()
lb = tk.Listbox(win)
lb.pack(pady=8)

def refresh():
    lb.delete(0, tk.END)
    for name, num in book.items():
        lb.insert(tk.END, f'{name}: {num}')

def add():
    book[e_name.get()] = e_num.get()
    refresh()

def find():
    num = book.get(e_name.get())
    if num:
        messagebox.showinfo('نتیجه', num)
    else:
        messagebox.showwarning('نتیجه', 'پیدا نشد')

tk.Button(win, text='افزودن', command=add).pack(side='left', padx=30)
tk.Button(win, text='جستجو', command=find).pack(side='left')
refresh()
win.mainloop()
```

> **کوییز:** وقت سنجش یادگیری است.

کوییز

س۱. برای ساختن دیکشنری از کدام علامت استفاده می‌کنیم؟

الف) `[ ]`

ب) `( )`

ج) `{ }` با `کلید :مقدار`

د) `< >`

س۲. اگر `d = {'a': 1}` باشد، `d.get('b', 0)` چیست؟

الف) خطا

ب) `None`

ج) `0`

د) `'b'`

س۳. تفاوت اصلی تاپل و لیست چیست؟

الف) تاپل تغییرناپذیر است

ب) تاپل عضو تکراری ندارد

ج) تاپل فقط عدد می‌پذیرد

د) هیچ فرقی ندارند

س۴. `len(set([1, 1, 2, 2, 3]))` چند است؟

الف) ۵

ب) ۳

ج) ۲

د) ۱

س۵. چرا همه `Radiobutton`های یک گروه باید `variable` یکسان داشته باشند؟

الف) تا رنگشان یکی شود

ب) تا فقط یکی از آنها انتخاب‌شده بماند

ج) تا سریع‌تر اجرا شوند

د) چون پایتون اجبار کرده

تمرین کلاسی

۱. دیکشنری از سه دانش‌آموز و نمره‌هایشان بسازید و همه را چاپ کنید.
۲. نمره یک دانش‌آموز را تغییر دهید و یک دانش‌آموز جدید اضافه کنید.
۳. در مثال ۹ یک گزینه چهارم به نام «خیلی سخت» اضافه کنید.

> **راهنمایی:** برای تمرین ۳ فقط کافی است عنوان جدید را به لیست داخل `for` اضافه کنید.

تمرین در خانه

۱. برنامه‌ای که متنی بگیرد و تعداد تکرار هر کلمه را با دیکشنری بشمارد.
۲. برنامه‌ای که دیکشنری معنی چند کلمه انگلیسی (مثلاً `کتاب :book`) داشته باشد و معنی کلمه واردشده را بدهد.
۳. برنامه‌ای که لیستی از اعداد را بگیرد و فقط عددهای تکراری آن را چاپ کند.
۴. در پنجره، دو `Radiobutton` برای «پسر / دختر» و یک `Checkbutton` برای «ثبت‌نام خبرنامه» بسازید و با دکمه، خلاصه را نشان دهید.
۵. دفترچه تلفن را طوری تکمیل کنید که دکمه «حذف» هم داشته باشد.
۶. **چالش:** مترجم ساده: یک دیکشنری ۵ کلمه‌ای انگلیسی به فارسی بسازید و در پنجره با `Combobox` کلمه را انتخاب کنید تا ترجمه نمایش داده شود.

> **راهنمایی:** برای حذف از دفترچه از `del book[name]` استفاده کنید و قبلش با `if name in book:` وجود کلید را بررسی کنید.

درس ۱۰: کار با فایل

هدف‌ها

- نوشتن، خواندن و اضافه کردن به فایل متنی (اسنیپت‌های `izehwritefile` و `izehreadfile`)
- استفاده از `with` و `encoding='utf-8'`
- خواندن خط‌به‌خط و پردازش محتوای فایل
- کار با `Text` و `filedialog` در پنجره (اسنیپت `tirT`)
- ساخت یک دفترچه یادداشت (نوت‌پد) ساده

مفهوم: فایل

متغیرها با بسته شدن برنامه از بین می‌روند. برای نگه‌داشتن داده باید آن را در **فایل** ذخیره کنیم. با تابع `open(نام‌فایل, حالت)` فایل را باز می‌کنیم. حالت‌های اصلی:

- `'w'` نوشتن (محتوای قبلی پاک می‌شود)
- `'r'` خواندن
- `'a'` اضافه کردن به انتهای فایل

دستور `with open(...) as f:` فایل را باز می‌کند و پایان کار خودش آن را می‌بندد، حتی اگر خطایی رخ دهد.

> **هشدار:** برای فایل‌های فارسی همیشه `encoding='utf-8'` بنویسید. در غیر این صورت ممکن است در ویندوز خطای `UnicodeEncodeError` بگیرید یا حروف فارسی خراب شوند.

مثال ۱: نوشتن در فایل (اسنیپت izehwritefile)

```
with open('note.txt', 'w', encoding='utf-8') as f:
    f.write('این یک یادداشت است')
print('ذخیره شد')
```

خروجی:

```
ذخیره شد
```

بعد از اجرا، فایل `note.txt` در همان پوشه برنامه ساخته می‌شود. آن را در VS Code باز کنید و ببینید.

مثال ۲: خواندن فایل (اسنیپت izehreadfile)

```
with open('note.txt', 'r', encoding='utf-8') as f:
    content = f.read()
print(content)
```

خروجی:

```
این یک یادداشت است
```

مثال ۳: چند خط نوشتن

`write` خودش خط جدید نمی‌گذارد. باید `\n` را خودتان بنویسید.

```
with open('names.txt', 'w', encoding='utf-8') as f:
    f.write('علی\n')
    f.write('سارا\n')
    f.write('رضا\n')
```

مثال ۴: خواندن خط‌به‌خط

```
with open('names.txt', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())
```

خروجی:

```
علی
سارا
رضا
```

> **نکته:** هر خطی که از فایل می‌خوانید در انتها `\n` دارد. با `strip()` آن را پاک کنید.

مثال ۵: خواندن به شکل لیست

```
with open('names.txt', 'r', encoding='utf-8') as f:
    names = f.read().splitlines()
print(names)
print(len(names), 'نام در فایل است')
```

خروجی:

```
['علی', 'سارا', 'رضا']
3 نام در فایل است
```

مثال ۶: اضافه کردن به انتهای فایل

```
with open('names.txt', 'a', encoding='utf-8') as f:
    f.write('مریم\n')

with open('names.txt', 'r', encoding='utf-8') as f:
    print(f.read())
```

خروجی:

```
علی
سارا
رضا
مریم
```

> **یادآوری:** حالت `'w'` همه محتوای قبلی فایل را پاک می‌کند. اگر می‌خواهید چیزی را حفظ کنید و به آن اضافه کنید از `'a'` استفاده کنید.

مثال ۷: ذخیره لیست در فایل

```
scores = [18, 15, 20, 12]
with open('scores.txt', 'w', encoding='utf-8') as f:
    for s in scores:
        f.write(str(s) + '\n')
```

مثال ۸: خواندن عددها و محاسبه

```
total = 0
count = 0
with open('scores.txt', 'r', encoding='utf-8') as f:
    for line in f:
        total += int(line)
        count += 1
print('میانگین:', total / count)
```

خروجی:

```
میانگین: 16.25
```

مثال ۹: بررسی وجود فایل

```
import os

if os.path.exists('names.txt'):
    print('فایل هست')
else:
    print('فایل پیدا نشد')
```

خروجی:

```
فایل هست
```

مثال ۱۰: دفتر ثبت (لاگ) ورودی‌ها

```
text = input('یادداشت امروز: ')
with open('diary.txt', 'a', encoding='utf-8') as f:
    f.write(text + '\n')
print('ثبت شد')
```

خروجی:

```
یادداشت امروز: پایتون یاد گرفتم
ثبت شد
```

مثال ۱۱: Text در پنجره: ذخیره و بازکردن

`Text` یک کادر متن چندخطی است. مکان‌ها به شکل `'خط.ستون'` داده می‌شوند و `'1.0'` یعنی ابتدای متن.

```
import tkinter as tk

win = tk.Tk()
win.title('یادداشت')
text = tk.Text(win, width=40, height=10)
text.pack(padx=10, pady=10)

def save():
    with open('note.txt', 'w', encoding='utf-8') as f:
        f.write(text.get('1.0', tk.END))

def load():
    with open('note.txt', 'r', encoding='utf-8') as f:
        text.delete('1.0', tk.END)
        text.insert('1.0', f.read())

tk.Button(win, text='ذخیره', command=save).pack(side='left', padx=10)
tk.Button(win, text='باز کردن', command=load).pack(side='left')
win.mainloop()
```

هشدار: اگر فایل `note.txt` هنوز وجود نداشته باشد و دکمه «باز کردن» را بزنید، خطای `FileNotFoundError` می‌گیرید. در درس بعد یاد می‌گیریم چطور این را مدیریت کنیم.

مثال ۱۲: نوت‌پد با انتخاب فایل

ماژول `filedialog` پنجره آشنای «ذخیره» و «باز کردن» ویندوز را باز می‌کند و مسیر فایل را می‌دهد.

```
import tkinter as tk
from tkinter import filedialog


win = tk.Tk()
win.title('نـوتپـد سـاده')
text = tk.Text(win, width=50, height=15)
text.pack(padx=10, pady=10)


def save_as():
    path = filedialog.asksaveasfilename(defaultextension='.txt')
    if path:
        with open(path, 'w', encoding='utf-8') as f:
            f.write(text.get('1.0', tk.END))


def open_file():
    path = filedialog.askopenfilename(filetypes=[('مـتـن', '*.txt')])
    if path:
        with open(path, 'r', encoding='utf-8') as f:
            text.delete('1.0', tk.END)
            text.insert('1.0', f.read())


tk.Button(win, text='بـاز کـردن', command=open_file).pack(side='left', padx=10)
tk.Button(win, text='ذخـیـره', command=save_as).pack(side='left')
win.mainloop()
```

مثال ۱۳: شمارنده کلمه برای فایل

```
with open('diary.txt', 'r', encoding='utf-8') as f:
    text = f.read()
print('تـعـداد حروف:', len(text))
print('تـعـداد کـلـمـه:', len(text.split()))
print('تـعـداد خط:', len(text.splitlines()))
```

خروجی:

```
25: تـعـداد حروف
5: تـعـداد کـلـمـه
1: تـعـداد خط
```

نکته: تعداد دقیق خروجی به محتوای فایل شما بستگی دارد.

کوییز: وقت سنجش یادگیری است.

کوییز

س۱. کدام حالت `open` محتوای قبلی فایل را پاک می‌کند؟

الف) `'r'`

ب) `'a'`

ج) `'w'`

د) `'x'`

س۲. مزیت `with open(...)` چیست؟

الف) فایل سریع‌تر خوانده می‌شود

ب) فایل خودکار بسته می‌شود

ج) فایل رمزگذاری می‌شود

د) فایل کوچک‌تر می‌شود

س۳. برای ذخیره فارسی در فایل کدام گزینه مهم است؟

الف) `mode='fa'`

ب) `encoding='utf-8'`

ج) `lang='persian'`

د) `font='Tahoma'`

س۴. `f.write('abc')` چند خط جدید به فایل اضافه می‌کند؟

الف) یک

ب) دو

ج) هیچ

د) بستگی به طول متن دارد

س۵. در `Text` مکان ابتدای متن کدام است؟

الف) `'0'`

ب) `'1'`

ج) `'1.0'`

د) `'0.0'`

تمرین کلاسی

۱. سه جمله را در فایلی به نام `my.txt` ذخیره کنید و دوباره بخوانید.

۲. برنامه‌ای بنویسید که نام را بگیرد و به انتهای فایل `names.txt` اضافه کند.

۳. نوت‌پد را طوری تغییر دهید که عنوان پنجره «نوت‌پد من» باشد و دکمه «پاک کردن» هم داشته باشد.

> **راهنمایی:** برای تمرین ۳ از `text.delete('1.0', tk.END)` استفاده کنید.

تمرین در خانه

۱. برنامه‌ای که ۵ عدد از کاربر بگیرد، در فایل ذخیره کند، سپس فایل را بخواند و جمع را چاپ کند.

۲. برنامه‌ای که تعداد خط‌های یک فایل را بشمارد.

۳. برنامه‌ای که فایل `names.txt` را بخواند و نام‌ها را مرتب شده در فایل جدیدی ذخیره کند.

۴. برنامه‌ای که کلمه‌ای را بگیرد و بگوید در یک فایل چند بار آمده است.

۵. نوت‌پد گرافیکی را طوری تکمیل کنید که پایین پنجره تعداد کلمه‌ها را نشان دهد.

۶. **چالش:** دفترچه خاطرات: هر بار دکمه «ثبت» زده می‌شود، تاریخ امروز و متن به فایل اضافه شود (راهنمایی: `from datetime import date`، سپس `date.today()`).

> **راهنمایی:** برای تمرین ۴ فایل را با `read()` بخوانید و از `count` رشته‌ها استفاده کنید.

درس ۱۱: مدیریت خطا، منو و لغزنده

هدف‌ها

- شناخت انواع خطاهای رایج و خواندن پیام خطا
- استفاده از `try`، `except`، `else` و `finally`
- بررسی ورودی‌ها (اعتبارسنجی) پیش از استفاده
- ساختن منو (`Menu`، اسنیپت `tirM`) و لغزنده (`Scale`، اسنیپت `tirS`)
- ساختن برنامه‌ای که با ورودی اشتباه بسته نمی‌شود

مفهوم: خطا

وقتی پایتون نتواند دستوری را اجرا کند، برنامه را متوقف می‌کند و **پیام خطا** نشان می‌دهد. پیام خطا دشمن شما نیست.

آخرین خط آن نوع و علت خطا را می‌گوید و خط‌های بالاتر نشان می‌دهند خطا در کدام خط رخ داده است.

خطاهای رایج:

- `SyntaxError`: نوشتار اشتباه (فراموش کردن دونقطه یا پرانتز)
- `NameError`: استفاده از متغیری که تعریف نشده
- `TypeError`: ترکیب نوع‌های ناسازگار (مثل جمع متن و عدد)
- `ValueError`: مقدار نامناسب (مثل `int('abc')`)
- `ZeroDivisionError`: تقسیم بر صفر
- `IndexError`: اندیس بیرون از لیست
- `KeyError`: کلید ناموجود در دیکشنری
- `FileNotFoundError`: فایل پیدا نشد

مثال ۱: دیدن چند خطا

```
# هر خط را جدا امتحان کنید
# int('abc')          # ValueError
# 10 / 0              # ZeroDivisionError
# [1, 2, 3][5]        # IndexError
# {'a': 1}['b']       # KeyError
# open('nofile.txt')  # FileNotFoundError
print('هیچ‌کدام اجرا نشد')
```

خروجی:

```
هیچ‌کدام اجرا نشد
```

> **نکته:** خطوط را یکی‌یکی از حالت کامنت خارج کنید و پیام خطا را بخوانید. خواندن خطا مهارت مهمی است.

مثال ۲: اولین try و except

```
try:
    n = int(input('یک عدد: '))
    print('دو برابر:', n * 2)
except ValueError:
    print('این عدد نیست!')
print('برنامه ادامه دارد')
```

خروجی:

```
یک عدد: abc
این عدد نیست!
برنامه ادامه دارد
```

مثال ۳: چند نوع خطا

```
try:
    a = int(input('عدد اول: '))
    b = int(input('عدد دوم: '))
    print(a / b)
except ValueError:
    print('لطفاً عدد وارد کنید')
except ZeroDivisionError:
    print('تقسیم بر صفر ممکن نیست')
```

خروجی:

```
10: عدد اول
0: عدد دوم
تقسیم بر صفر ممکن نیست
```

مثال ۴: else و finally

بلوک `else` وقتی اجرا می‌شود که خطایی رخ نداده باشد و `finally` همیشه اجرا می‌شود.

```
try:
    n = int(input('عدد: '))
except ValueError:
    print('نامعتبر')
else:
    print('عدد معتبر بود:', n)
finally:
    print('پایان بررسی')
```

خروجی:

```
5: عدد
5: عدد معتبر بود
پایان بررسی
```

مثال ۵: تکرار تا ورودی درست

```
while True:
    try:
        age = int(input('سن: '))
        break
    except ValueError:
        print('دوباره، فقط عدد!')
print('سن شما', age)
```

خروجی:

```
؟: سن
دوباره، فقط عدد!
14: سن
سن شما 14
```

مثال ۶: فایل ناموجود

```
try:
    with open('missing.txt', 'r', encoding='utf-8') as f:
        print(f.read())
except FileNotFoundError:
    print('فایل پیدا نشد')
```

خروجی:

```
فایل پیدا نشد
```

مثال ۷: گرفتن همه خطاها و نمایش پیام

```
try:
    nums = [1, 2, 3]
    print(nums[10])
except Exception as e:
    print('خطا رخ داد:', e)
```

خروجی:

```
خطا رخ داد: list index out of range
```

هشدار: `except Exception` همه خطاها را می‌گیرد، حتی خطاهایی که باید ببینید. تا جایی که می‌شود نوع مشخص خطا را بنویسید.

مثال ۸: خطای ایجاد شده توسط خودمان (raise)

```
def set_age(age):
    if age < 0:
        raise ValueError('سن نمی‌تواند منفی باشد')
    return age


try:
    set_age(-5)
except ValueError as e:
    print(e)
```

خروجی:

```
سن نمی‌تواند منفی باشد
```

مثال ۹: ماشین‌حساب گرافیکی ایمن

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.title('تقسیم')
win.geometry('260x200')
e1 = tk.Entry(win)
e1.pack(pady=8)
e2 = tk.Entry(win)
e2.pack()


def divide():
    try:
        r = float(e1.get()) / float(e2.get())
        messagebox.showinfo('نتیجه', r)
    except ValueError:
        messagebox.showerror('خطا', 'لطفاً عدد وارد کنید')
    except ZeroDivisionError:
        messagebox.showerror('خطا', 'تقسیم بر صفر ممکن نیست')


tk.Button(win, text='تقسیم', command=divide).pack(pady=12)
win.mainloop()
```

مثال ۱۰: بازکردن ایمن در نوت‌پد

مشکل «باز کردن» در درس قبل را حل می‌کنیم.

```
import tkinter as tk
from tkinter import messagebox

win = tk.Tk()
text = tk.Text(win, width=40, height=10)
text.pack(padx=10, pady=10)

def load():
    try:
        with open('note.txt', 'r', encoding='utf-8') as f:
            text.delete('1.0', tk.END)
            text.insert('1.0', f.read())
    except FileNotFoundError:
        messagebox.showwarning('توجه', 'هنوز یادداشتی ذخیره نشده است')

def save():
    with open('note.txt', 'w', encoding='utf-8') as f:
        f.write(text.get('1.0', tk.END))

tk.Button(win, text='ذخیره', command=save).pack(side='left', padx=10)
tk.Button(win, text='باز کردن', command=load).pack(side='left')
win.mainloop()
```

مثال ۱۱: منوی پنجره (اسنیپت tirM)

```
import tkinter as tk
from tkinter import messagebox

win = tk.Tk()
win.title('منو')
win.geometry('300x200')

def about():
    messagebox.showinfo('درباره', 'برنامه آموزشی Tirotir.ir')

menubar = tk.Menu(win)
filemenu = tk.Menu(menubar, tearoff=0)
filemenu.add_command(label='درباره', command=about)
filemenu.add_separator()
filemenu.add_command(label='خروج', command=win.destroy)
menubar.add_cascade(label='فایل', menu=filemenu)
win.config(menu=menubar)
win.mainloop()
```

مثال ۱۲: لغزنده (Scale) (اسنیپت tirS)

```
import tkinter as tk

win = tk.Tk()
win.title('لغزنده')
win.geometry('300x200')
scale = tk.Scale(win, from_=0, to=100, orient='horizontal', length=220)
scale.pack(pady=15)
out = tk.Label(win, text='مقدار: 0', font=('Tahoma', 14))
out.pack()

def show(value):
    out.config(text=f'مقدار: {value}')

scale.config(command=show)
win.mainloop()
```

مثال ۱۳: تنظیم رنگ با سه لغزنده

```
import tkinter as tk

win = tk.Tk()
win.title('رنگساز')
win.geometry('300x300')
r = tk.Scale(win, from_=0, to=255, orient='horizontal', label='قرمز')
g = tk.Scale(win, from_=0, to=255, orient='horizontal', label='سبز')
b = tk.Scale(win, from_=0, to=255, orient='horizontal', label='آبی')
for s in (r, g, b):
    s.pack(fill='x', padx=20)
box = tk.Label(win, text='    ', bg='#000000', height=3)
box.pack(fill='x', padx=20, pady=15)

def update(event=None):
    color = f'#{r.get():02x}{g.get():02x}{b.get():02x}'
    box.config(bg=color)

for s in (r, g, b):
    s.config(command=update)
win.mainloop()
```

نکته: `02x:` عدد را به شکل دو رقمی هگزادسیمال می‌نویسد. رنگ‌ها در Tkinter به شکل `#RRGGBB` هستند.

کوییز: وقت سنجش یادگیری است.

کوییز

س۱. `int('abc')` کدام خطا را می‌دهد؟

الف) `TypeError`

ب) `ValueError`

ج) `NameError`

د) `SyntaxError`

س۲. بلوک `finally` چه زمانی اجرا می‌شود؟

الف) فقط وقتی خطا رخ دهد

ب) فقط وقتی خطا رخ ندهد

ج) همیشه

د) هیچ‌وقت

س۳. کدام خطا هنگام `10 / 0` رخ می‌دهد؟

الف) `ValueError`

ب) `ZeroDivisionError`

ج) `IndexError`

د) `KeyError`

س۴. در یک ساختار `try / except`، کد خطرناک را کجا می‌نویسیم؟

الف) داخل `except`

ب) داخل `finally`

ج) داخل `try`

د) بیرون از همه

س۵. کدام دستور برای ساختن پنجره‌های منوی بالای برنامه به کار می‌رود؟

الف) `tk.Menu`

ب) `tk.Label`

ج) `tk.Frame`

د) `tk.Canvas`

تمرین کلاسی

۱. برنامه‌ای بنویسید که دو عدد بگیرد و بدون خطا تقسیم کند.

۲. برنامه‌ای که تا وارد کردن عدد درست، سن را بپرسد (مثال ۵).

۳. به نوت‌پد منوی «فایل» با دو گزینه «ذخیره» و «خروج» اضافه کنید.

> **راهنمایی:** برای تمرین ۳ ساختار مثال ۱۱ را ببینید و به جای `about` تابع `save` را وصل کنید.

تمرین در خانه

۱. برنامه‌ای که ۵ بار از کاربر عدد بگیرد و اگر وارد شده نامعتبر بود آن را نادیده بگیرد و جمع عددهای معتبر را چاپ کند.

۲. برنامه‌ای که اندیس لیستی را بگیرد و عضو مربوط را بدون خطا چاپ کند (`IndexError`).

۳. تابع `safe_int(text)` که اگر متن عدد بود آن را عدد کند و اگر نبود `None` برگرداند.

۴. نسخه ایمن ماشین‌حساب چهار عمل درس ۸ را بنویسید.

۵. برنامه‌ای با لغزنده که اندازه فونت یک برچسب را از ۸ تا ۴۰ تغییر دهد.

۶. **چالش:** نوت‌پد با منوی کامل: «جدید»، «باز کردن»، «ذخیره» و «خروج» و پیام هشدار اگر فایل وجود نداشت.

> **راهنمایی:** برای تغییر فونت با لغزنده از `lbl.config(font=('Tahoma', int(value)))` استفاده کنید.

درس ۱۲: پروژه‌های پایانی

هدف‌ها

- ترکیب همه آموخته‌ها در چند پروژه کامل
- طراحی برنامه با تقسیم آن به تابع‌های کوچک
- ذخیره دائمی داده‌ها در فایل
- ساخت فایل اجرایی (exe) با `pyinstaller`
- مرور کلی دوره و آزمون پایانی

مرور سریع دوره

> **یادآوری:** این فهرست فشرده همه مفهوم‌های دوره است. هر جا چیزی را فراموش کردید به درس مربوطه برگردید.

- **درس ۱:** `print`، کامنت `#`، پنجره `Tk()`، `Label`، `Button`، `mainloop`
- **درس ۲:** متغیر، انواع `str`، `int`، `float`، `bool`، `input`، `Entry` و `.get()`
- **درس ۳:** عملگرهای `+ - * / // % **`، اولویت، `round`، `math`
- **درس ۴:** رشته‌ها، برش `[a:b]`، متدهای `upper`، `strip`، `replace`، `split`، `f-string`، `messagebox`
- **درس ۵:** `if / elif / else`، مقایسه‌ها، `and / or / not`
- **درس ۶:** `for`، `range`، `while`، `break`، `continue`، `Listbox`
- **درس ۷:** لیست، `append`، `remove`، `sort`، `random`
- **درس ۸:** `def`، پارامتر، `return`، متغیر محلی و سراسری، `lambda`
- **درس ۹:** دیکشنری، تاپل، مجموعه، `Radiobutton`، `Checkbutton`، `Combobox`
- **درس ۱۰:** `open`، `with`، `read`، `write`، `Text`، `filedialog`
- **درس ۱۱:** `try / except`، `raise`، `Menu`، `Scale`

اصول طراحی یک برنامه

پیش از نوشتن کد، این سه سؤال را از خودتان بپرسید:

۱. برنامه چه ورودی‌هایی می‌گیرد؟
۲. چه پردازشی روی آنها انجام می‌دهد؟
۳. چه خروجی‌ای می‌دهد و کجا ذخیره می‌شود؟

سپس هر کار را یک تابع کنید. تابع‌های کوچک راحت‌تر نوشته و آزمایش می‌شوند.

> **نکته:** برنامه را قدم‌به‌قدم بسازید: اول یک پنجره خالی، بعد یک دکمه، بعد کار دکمه. بعد از هر قدم اجرا کنید تا اگر خطایی آمد بدانید از کدام قدم است.

پروژه ۱: دفترچه کارها (نسخه متنی)

برنامه‌ای با منو که کارها را در فایل نگه می‌دارد.

```
FILE = 'tasks.txt'

def load():
    try:
        with open(FILE, 'r', encoding='utf-8') as f:
            return f.read().splitlines()
    except FileNotFoundError:
        return []

def save(tasks):
    with open(FILE, 'w', encoding='utf-8') as f:
        for t in tasks:
            f.write(t + '\n')

tasks = load()
while True:
    cmd = input('add / list / done / quit: ')
    if cmd == 'add':
        tasks.append(input('کار: '))
        save(tasks)
    elif cmd == 'list':
        for i, t in enumerate(tasks, 1):
            print(i, t)
    elif cmd == 'done':
        try:
            n = int(input('شماره کار: '))
            print('انجام شد:', tasks.pop(n - 1))
            save(tasks)
        except (ValueError, IndexError):
            print('شماره نامعتبر')
    elif cmd == 'quit':
        break
```

خروجی:

```
add / list / done / quit: add
کار :خرید نان
add / list / done / quit: list
1 خرید نان
add / list / done / quit: done
1: شماره کار
انجام شد :خرید نان
add / list / done / quit: quit
```

> **نکته:** `except (ValueError, IndexError)` چند نوع خطا را با یک بلوک می‌گیرد.

پروژه ۱: دفترچه کارها (نسخه گرافیکی)

همین برنامه با پنجره: ورودی، لیست، دکمه‌های افزودن و حذف و ذخیره خودکار در فایل.

```
import tkinter as tk
from tkinter import messagebox


FILE = 'tasks.txt'
win = tk.Tk()
win.title('کارهای من')
top = tk.Frame(win)
top.pack(pady=8)
entry = tk.Entry(top, width=25)
entry.pack(side='left')
lb = tk.Listbox(win, width=40)
lb.pack(padx=10, pady=10)


def save():
    with open(FILE, 'w', encoding='utf-8') as f:
        for t in lb.get(0, tk.END):
            print(t, file=f)


def add():
    if entry.get().strip():
        lb.insert(tk.END, entry.get())
        entry.delete(0, tk.END)
        save()
    else:
        messagebox.showwarning('توجه', 'متن خالی است')


def remove():
    sel = lb.curselection()
    if sel:
        lb.delete(sel[0])
        save()
```

```
tk.Button(top, text='افزودن', command=add).pack(side='left')
tk.Button(top, text='حذف', command=remove).pack(side='left')

try:
    with open(FILE, 'r', encoding='utf-8') as f:
        for line in f:
            lb.insert(tk.END, line.strip())
except FileNotFoundError:
    pass

win.mainloop()
```

یادآوری: `Frame` یک جعبه نامرئی برای گروه‌بندی اجزاست. اینجا کادر ورودی و دو دکمه کنار هم در یک `Frame` قرار گرفته‌اند.

پروژه ۲: بازی حدس عدد

```
import random
import tkinter as tk

secret = random.randint(1, 100)
tries = 0

win = tk.Tk()
win.title('حدس عدد')
win.geometry('300x240')
tk.Label(win, text='عددی بین ۱ تا ۱۰۰ حدس بزنید').pack(pady=10)
entry = tk.Entry(win)
entry.pack()
msg = tk.Label(win, text='', font=('Tahoma', 12))
msg.pack(pady=10)

def guess():
    global tries
    try:
        g = int(entry.get())
    except ValueError:
        msg.config(text='فقط عدد!', fg='red')
        return
    tries += 1
    if g > secret:
        msg.config(text='کوچکتر', fg='blue')
    elif g < secret:
        msg.config(text='بزرگتر', fg='blue')
    else:
        msg.config(text=f'درست! آفرین {tries} حدس', fg='green')
```

```
tk.Button(win, text='حدس', command=guess).pack()
win.mainloop()
```

> **نکته:** چون `tries` را داخل تابع تغییر می‌دهیم، باید `global tries` بنویسیم (درس ۸).

پروژه ۳: آزمون چندگزینه‌ای

سؤال‌ها در یک لیست از دیکشنری نگه‌داری می‌شوند. اضافه کردن سؤال تازه فقط یک خط است.

```
import tkinter as tk
from tkinter import messagebox

questions = [
    {'q': 'خروجی print(2 ** 3) چیست؟', 'opts': ['6', '8', '9'], 'ans': 1},
    {'q': 'کدام نوع داده برای متن است؟', 'opts': ['int', 'str', 'bool'], 'ans':
1},
    {'q': 'نماد کامنت در پایتون کدام است؟', 'opts': ['//', '#', '--'], 'ans':
1},
]
state = {'i': 0, 'score': 0}

win = tk.Tk()
win.title('آزمون پایتون')
win.geometry('360x300')
q_label = tk.Label(win, font=('Tahoma', 13), wraplength=320)
q_label.pack(pady=15)
choice = tk.IntVar(value=-1)
radios = []
for k in range(3):
    r = tk.Radiobutton(win, variable=choice, value=k)
    r.pack(anchor='w', padx=40)
    radios.append(r)

def show():
    q = questions[state['i']]
    q_label.config(text=q['q'])
    choice.set(-1)
    for k, r in enumerate(radios):
        r.config(text=q['opts'][k])

def next_q():
    if choice.get() == -1:
        messagebox.showwarning('توجه', 'یک گزینه انتخاب کنید')
        return
    if choice.get() == questions[state['i']]['ans']:
        state['score'] += 1
    state['i'] += 1
```

```
    if state['i'] == len(questions):
        messagebox.showinfo('پایان',
                            f'امتیاز شما: {state["score"]} از {len(questions)}')
        win.destroy()
    else:
        show()


tk.Button(win, text='بعدی', command=next_q).pack(pady=20)
show()
win.mainloop()
```

نکته: برای ذخیره حالت برنامه (شماره سؤال و امتیاز) از یک دیکشنری `state` استفاده کردیم تا بدون `global` بتوانیم آنها را در تابع‌ها تغییر دهیم.

پروژه ۴: دفتر نمره‌ها

نمره‌ها را وارد می‌کنیم و میانگین، بیشترین و کمترین نمایش داده می‌شود.

```
import tkinter as tk
from tkinter import messagebox


grades = []


win = tk.Tk()
win.title('دفتر نمره')
win.geometry('300x380')
entry = tk.Entry(win)
entry.pack(pady=8)
lb = tk.Listbox(win, height=8)
lb.pack()
info = tk.Label(win, text='', font=('Tahoma', 11))
info.pack(pady=10)


def refresh():
    if grades:
        avg = sum(grades) / len(grades)
        info.config(text=f'میانگین: {avg:.2f}\nبیشترین: {max(grades)} - کمترین:
{min(grades)}')
    else:
        info.config(text='')


def add():
    try:
        g = float(entry.get())
        if not 0 <= g <= 20:
            raise ValueError
    except ValueError:
```

```
        messagebox.showerror('خطا', 'نمره باید عددی بین ۰ تا ۲۰ باشد')
        return
    grades.append(g)
    lb.insert(tk.END, g)
    entry.delete(0, tk.END)
    refresh()

tk.Button(win, text='افزودن نمره', command=add).pack()
win.mainloop()
```

ساخت فایل اجرایی (exe)

برای اینکه برنامه روی کامپیوتری که پایتون ندارد هم اجرا شود:

۱. در ترمینال VS Code بنویسید: `pip install pyinstaller`

۲. در پوشه برنامه بنویسید: `pyinstaller --onefile --noconsole app.py` (به‌جای `app.py` نام فایل خودتان)

۳. فایل `exe` ساخته‌شده در پوشه `dist` است.

نکته: گزینه `--noconsole` پنجره سیاه کنسول را برای برنامه‌های گرافیکی پنهان می‌کند. برای برنامه‌های متنی آن را نگذارید.

هشدار: بعضی آنتی‌ویروس‌ها فایل‌های `exe` ساخته‌شده با `pyinstaller` را اشتباهاً مشکوک می‌دانند. اگر برنامه را به کسی می‌دهید به او بگویید.

قدم‌های بعدی

- **پروژه شخصی:** یکی از ایده‌های بخش «تمرین در خانه» را انتخاب کنید و تا آخر بسازید.
- **کتابخانه‌ها:** `turtle` برای نقاشی، `pyttsx3` برای تبدیل متن به گفتار و `tkintermapview` برای نقشه را بررسی کنید.
- **اسنیپت‌ها:** اکستنشن‌های `izeh` و `Tirotir` مثال‌ها و قالب‌های بیشتری برای تمرین دارند.
- **مرور:** هر ماه یک بار کوییز درس‌ها را دوباره حل کنید.

کوییز: آزمون پایانی دوره: ۱۰ سؤال از همه درس‌ها.

آزمون پایانی

س۱. خروجی `print(10 // 3, 10 % 3)` چیست؟

الف) `3.33 1`

ب) `3 1`

ج) `3 3`

د) `1 3`

س۲. کدام گزینه یک دیکشنری معتبر است؟

الف) `[1: 'a']`

ب) `{1: 'a'}`

ج) `(1: 'a')`

د) `{1, 'a'}`

س۳. خروجی کد زیر چیست؟ `for i in range(3): print(i, end=' ')`

الف) `1 2 3`

ب) `0 1 2`

ج) `0 1 2 3`

د) `3 2 1`

س۴. خروجی `'Tirotir'[1:4]` چیست؟

الف) `iro`

ب) `Tir`

ج) `iroti`

د) `ro`

س۵. کدام دستور فایلی را برای نوشتن باز می‌کند و محتوای قبلی را پاک می‌کند؟

الف) `open('a.txt', 'r')`

ب) `open('a.txt', 'a')`

ج) `open('a.txt', 'w')`

د) `open('a.txt', 'x')`

س۶. اگر `nums = [3, 1, 2]` باشد، بعد از `nums.append(0)` و `nums.sort()` مقدار `nums` چیست؟

الف) `[0, 1, 2, 3]`

ب) `[3, 1, 2, 0]`

ج) `[3, 2, 1, 0]`

د) `[0, 3, 1, 2]`

س۷. کدام بلوک همیشه اجرا می‌شود؟

الف) `try`

ب) `except`

ج) `else`

د) `finally`

س۸. برای گرفتن متن کادر ورودی `Entry` به نام `e` از چه استفاده می‌کنیم؟

الف) `e.get()`

ب) `e.text`

ج) `e.input()`

د) `e.read()`

س۹. کدام عبارت درباره `return` درست است؟

الف) مقدار را چاپ می‌کند

ب) تابع را متوقف و مقدار را برمی‌گرداند

ج) حلقه را قطع می‌کند

د) فایل را می‌بندد

س۱۰. در `win.after(1000, tick)` عدد ۱۰۰۰ چیست؟

الف) تعداد تکرار

ب) تأخیر به میلی‌ثانیه

ج) عرض پنجره

د) شماره خط

تمرین کلاسی

۱. دفترچه کارها را طوری تغییر دهید که با زدن Enter در کادر ورودی، کار اضافه شود.

۲. بازی حدس عدد را با دکمه «بازی جدید» که عدد مخفی را دوباره می‌سازد، تکمیل کنید.

۳. در آزمون، یک سؤال چهارم اضافه کنید.

> **راهنمایی:** برای تمرین ۱ از `entry.bind('<Return>', lambda e: add())` استفاده کنید. برای تمرین ۲ تابعی بنویسید که `global secret, tries` را تعریف و دوباره مقدار دهد.

تمرین در خانه

یکی از ایده‌های زیر را انتخاب کنید و کامل بسازید. بعضی راه‌حل‌های نمونه در پاسخنامه آمده است.

۱. **ماشین‌حساب کامل** با دکمه‌های ۰ تا ۹، چهار عمل و پاک کردن.

۲. **ساعت دیجیتال** که هر ثانیه زمان را به‌روز کند (`time.strftime` و `win.after`).

۳. **دفترچه مخاطبین** با نام، شماره و ذخیره در فایل.

۴. **بازی سنگ، کاغذ، قیچی** با رایانه (`random.choice`).

۵. **مبدل واحد** برای طول، وزن و دما با `Combobox`.

۶. **برنامه نمره‌ها** که نمره‌ها را از فایل بخواند و میانگین هر دانش‌آموز را نشان دهد.

۷. **یادآور ساده** با شمارش معکوس و پیام در پایان.

۸. **چالش:** بازی «دوز» (Tic-Tac-Toe) با ۹ دکمه.

> **راهنمایی:** برای ساعت دیجیتال تابعی بنویسید که زمان را روی برچسب بگذارد و در انتها `win.after(1000, ...)` را صدا بزند. برای دوز، دکمه‌ها را با حلقه تو در تو و `grid` بچینید.

پاسخنامه و راهنمای حل تمرین‌ها

در این بخش پاسخ کوییزها و حل تمرین‌های هر درس آمده است. **اول خودتان تلاش کنید** و بعد پاسخ را ببینید. اگر جواب شما با کد این بخش فرق دارد ولی درست کار می‌کند، اشکالی ندارد: برای هر مسئله چند راه حل وجود دارد.

پاسخ‌های درس ۱: اولین برنامه و اولین پنجره

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: ب (عرض). س۵: ج.

حل تمرین‌های خانه:

تمرین ۱: مثلث

```
print('    *')
print('   ***')
print('  *****')
print(' *******')
print('*********')
```

یک راه کوتاه‌تر با حلقه در درس ششم یاد می‌گیریم.

تمرین ۲: کارت ویزیت

```
print('+' + '-' * 28 + '+')
print('|  علی رضایی')
print('|  تلفن: 0912-000-0000')
print('|  ایمیل: ali@example.com')
print('+' + '-' * 28 + '+')
```

تمرین ۳: سه برچسب رنگی

```
import tkinter as tk


win = tk.Tk()
win.geometry('300x200')
win.config(bg='#fff3e0')
tk.Label(win, text='قرمز', fg='red', bg='#fff3e0').pack(pady=8)
tk.Label(win, text='سبز', fg='green', bg='#fff3e0').pack(pady=8)
tk.Label(win, text='آبی', fg='blue', bg='#fff3e0').pack(pady=8)
win.mainloop()
```

تمرین ۴: پنجره ثابت با دو دکمه

```
import tkinter as tk


win = tk.Tk()
win.geometry('400x300')
win.resizable(False, False)
tk.Button(win, text='خروج', command=win.destroy).pack(pady=40)
tk.Button(win, text='دکمه دوم').pack()
win.mainloop()
```

دکمه دوم فعلاً `command` ندارد و کاری انجام نمی‌دهد.

پاسخ‌های درس ۲: متغیر و دریافت ورودی

پاسخ کوییز: س۱: ج. س۲: ج. س۳: ب. س۴: ج. س۵: ج.

حل تمرین‌های خانه:

تمرین ۱: محاسبه سن

```
birth = int(input('سال تولد: '))
current = 1405
print('سن شما:', current - birth)
```

تمرین ۲ و ۳: مساحت و محیط

```
w = float(input('عرض: '))
h = float(input('طول: '))
print('مساحت:', w * h)
print('محیط:', 2 * (w + h))
```

```
import tkinter as tk


win = tk.Tk()
win.geometry('280x240')
tk.Label(win, text='عرض:').pack()
e_w = tk.Entry(win)
e_w.pack()
tk.Label(win, text='طول:').pack()
e_h = tk.Entry(win)
e_h.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def calc():
    w = float(e_w.get())
    h = float(e_h.get())
    out.config(text=f'مساحت: {w * h} - محیط: {2 * (w + h)}')


tk.Button(win, text='محاسبه', command=calc).pack()
win.mainloop()
```

تمرین ۴: سلام و پاک کردن

```
import tkinter as tk


win = tk.Tk()
win.geometry('280x200')
entry = tk.Entry(win)
entry.pack(pady=10)
out = tk.Label(win, text='')
out.pack()


def hello():
    out.config(text='خوش آمدید ' + entry.get())


def clear():
    entry.delete(0, tk.END)
    out.config(text='')


tk.Button(win, text='سلام', command=hello).pack(pady=5)
tk.Button(win, text='پاک کردن', command=clear).pack()
win.mainloop()
```

تمرین ۵: تبدیل دما

```
c = float(input('دما به سانتی‌گراد: '))
f = c * 9 / 5 + 32
print(f'{c} سانتی‌گراد = {f} فارنهایت')
```

پاسخ‌های درس ۳: عملیات ریاضی

پاسخ کوییز: س۱: ب. س۲: ب. س۳: ب. س۴: د. س۵: ب.

حل تمرین‌های خانه:

تمرین ۱: دایره

```
import math

r = float(input('شعاع: '))
print('مساحت:', math.pi * r ** 2)
print('محیط:', 2 * math.pi * r)
```

تمرین ۲: انعام

```
bill = float(input('مبلغ صورتحساب: '))
tip = float(input('درصد انعام: '))
total = bill + bill * tip / 100
print('جمع کل:', total)
```

تمرین ۳: هفته و روز

```
days = int(input('تعداد روز: '))
print(days // 7, 'هفته و', days % 7, 'روز')
```

تمرین ۴: دایره گرافیکی

```
import math
import tkinter as tk

win = tk.Tk()
win.geometry('280x200')
tk.Label(win, text='شعاع:').pack()
e = tk.Entry(win)
e.pack()
out = tk.Label(win, text='')
out.pack(pady=10)

def calc():
    r = float(e.get())
    out.config(text=f'مساحت: {math.pi * r ** 2:.2f}')

tk.Button(win, text='محاسبه', command=calc).pack()
win.mainloop()
```

تمرین ۵: وتر مثلث

```
import math
import tkinter as tk


win = tk.Tk()
win.geometry('280x240')
tk.Label(win, text='ضلع اول:').pack()
e1 = tk.Entry(win)
e1.pack()
tk.Label(win, text='ضلع دوم:').pack()
e2 = tk.Entry(win)
e2.pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def calc():
    a = float(e1.get())
    b = float(e2.get())
    out.config(text=f'وتر: {math.sqrt(a ** 2 + b ** 2):.2f}')


tk.Button(win, text='محاسبه', command=calc).pack()
win.mainloop()
```

تمرین ۶: اسکناس‌ها

```
money = int(input('مبلغ: '))
n50 = money // 50000
money = money % 50000
n10 = money // 10000
money = money % 10000
n5 = money // 5000
money = money % 5000
print(n50, 'اسکناس ۵۰۰۰۰،', n10, 'اسکناس ۱۰۰۰۰،', n5, 'اسکناس ۵۰۰۰')
print('باقی‌مانده:', money)
```

پاسخ‌های درس ۴: رشته‌ها و پیام‌ها

پاسخ کوییز: س۱: ب. س۲: الف. س۳: ج. س۴: ب. س۵: ج.

حل تمرین‌های خانه:

تمرین ۱: خانوادگی، نام

```
first = input('نام: ')
last = input('نام خانوادگی: ')
print(f'{last}، {first}')
```

تمرین ۲: اولین و آخرین حرف

```
word = input('یک کلمه: ')
print('اولین:', word[0], '- آخرین:', word[-1])
```

تمرین ۳: جایگزینی فاصله

```
s = input('یک جمله: ')
print(s.replace(' ', '-'))
```

تمرین ۴: پیام خوشامد

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.geometry('280x220')
tk.Label(win, text='نام:').pack()
e1 = tk.Entry(win)
e1.pack()
tk.Label(win, text='نام خانوادگی:').pack()
e2 = tk.Entry(win)
e2.pack()


def hello():
    messagebox.showinfo('سلام', f'سلام {e1.get()} {e2.get()} عزیز')


tk.Button(win, text='سلام', command=hello).pack(pady=10)
win.mainloop()
```

تمرین ۵: شمارنده کلمه و حرف

```
import tkinter as tk


win = tk.Tk()
win.geometry('320x180')
entry = tk.Entry(win, width=35)
entry.pack(pady=15)
lbl = tk.Label(win, text='')
lbl.pack()


def count(event):
    t = entry.get()
    lbl.config(text=f'حروف: {len(t)} - کلمه ها: {len(t.split())}')


entry.bind('<KeyRelease>', count)
win.mainloop()
```

تمرین ۶: بررسی معکوس

```
word = input('یک کلمه: ')
print(word == word[::-1])
```

پاسخ‌های درس ۵: تصمیم‌گیری با if

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: ج. س۵: الف.

حل تمرین‌های خانه:

تمرین ۱: کدام بزرگ‌تر

```
a = float(input('عدد اول: '))
b = float(input('عدد دوم: '))
if a > b:
    print('عدد اول بزرگتر است')
elif b > a:
    print('عدد دوم بزرگتر است')
else:
    print('مساوی هستند')
```

تمرین ۲: بخش‌پذیری بر ۳

```
n = int(input('عدد: '))
if n % 3 == 0:
    print('بر ۳ بخشپذیر است')
else:
    print('بر ۳ بخشپذیر نیست')
```

تمرین ۳: نوع مثلث

```
a = float(input('ضلع اول: '))
b = float(input('ضلع دوم: '))
c = float(input('ضلع سوم: '))
if a == b and b == c:
    print('متساوی‌الاضلاع')
elif a == b or b == c or a == c:
    print('متساوی‌الساقین')
else:
    print('مختلف‌الاضلاع')
```

تمرین ۴: بخش‌پذیری گرافیکی

```
import tkinter as tk


win = tk.Tk()
win.geometry('260x180')
entry = tk.Entry(win)
entry.pack(pady=10)
lbl = tk.Label(win, text='')
lbl.pack()


def check():
    n = int(entry.get())
    if n % 3 == 0:
        lbl.config(text='بخش‌پذیر است', fg='green')
    else:
        lbl.config(text='بخش‌پذیر نیست', fg='red')


tk.Button(win, text='بررسی', command=check).pack()
win.mainloop()
```

تمرین ۵: هشدار برای کادر خالی

```
def login():
    if e_user.get() == '':
        messagebox.showwarning('توجه', 'نام کاربری را وارد کنید')
    elif e_user.get() == 'ali' and e_pass.get() == '1234':
        messagebox.showinfo('ورود', 'خوش آمدید')
    else:
        messagebox.showerror('خطا', 'نام کاربری یا رمز اشتباه است')
```

فقط تابع `login` در مثال ۱۱ را با این نسخه عوض کنید.

تمرین ۶: حدس عدد

```
secret = 7
guess = int(input('حدس شما: '))
if guess > secret:
    print('کوچکتر')
elif guess < secret:
    print('بزرگتر')
else:
    print('درست حدس زدید!')
```

پاسخ‌های درس ۶: حلقه‌ها

پاسخ کوییز: س۱: ب. س۲: ب. س۳: د. س۴: ب. س۵: ب.

حل تمرین‌های خانه:

تمرین ۱: جمع تا n

```
n = int(input('n: '))
total = 0
for i in range(1, n + 1):
    total += i
print(total)
```

تمرین ۲: فاکتوریل

```
n = int(input('عدد: '))
f = 1
for i in range(2, n + 1):
    f *= i
print(f'{n}! = {f}')
```

تمرین ۳: میانگین پنج نمره

```
total = 0
for i in range(5):
    total += float(input(f'نمره {i + 1}: '))
print('میانگین:', total / 5)
```

تمرین ۴: جمع تا عدد منفی

```
total = 0
n = int(input('عدد: '))
while n >= 0:
    total += n
    n = int(input('عدد: '))
print('جمع:', total)
```

تمرین ۵: فاکتوریل گرافیکی

```
import tkinter as tk


win = tk.Tk()
win.geometry('260x180')
entry = tk.Entry(win)
entry.pack(pady=10)
lbl = tk.Label(win, text='')
lbl.pack()


def calc():
    n = int(entry.get())
    f = 1
    for i in range(2, n + 1):
        f *= i
    lbl.config(text=f'{n}! = {f}')


tk.Button(win, text='محاسبه', command=calc).pack(pady=10)
win.mainloop()
```

تمرین ۶: اعداد اول

```
import tkinter as tk


win = tk.Tk()
win.geometry('200x320')
lb = tk.Listbox(win, width=15, height=15)
lb.pack(padx=20, pady=20)


for n in range(2, 50):
    is_prime = True
    for d in range(2, n):
        if n % d == 0:
            is_prime = False
            break
    if is_prime:
        lb.insert(tk.END, n)


win.mainloop()
```

پاسخ‌های درس ۷: لیست‌ها

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: الف (عضو سوم خودش یک لیست است ولی فقط یک عضو حساب می‌شود). س۵: ب.

حل تمرین‌های خانه:

تمرین ۱: زوج‌ها

```
evens = []
for i in range(1, 21):
    if i % 2 == 0:
        evens.append(i)
print(evens)
print('جمع:', sum(evens))
```

تمرین ۲: مرتب‌سازی تا ۰

```
nums = []
n = int(input('عدد (۰ برای پایان): '))
while n != 0:
    nums.append(n)
    n = int(input('عدد (۰ برای پایان): '))
nums.sort()
print(nums)
```

تمرین ۳: بالای میانگین

```
grades = [12, 18, 15, 20, 9, 17]
avg = sum(grades) / len(grades)
count = 0
for g in grades:
    if g > avg:
        count += 1
print('میانگین:', avg)
print('تعداد بالای میانگین:', count)
```

تمرین ۴: حذف تکراری‌ها

```
items = [3, 5, 3, 7, 5, 9]
unique = []
for x in items:
    if x not in unique:
        unique.append(x)
print(unique)
```

تمرین ۵: حذف همه

```
def clear_all():
    lb.delete(0, tk.END)


tk.Button(win, text='حذف همه', command=clear_all).pack(pady=3)
```

این تکه را به مثال ۱۱ اضافه کنید، قبل از `win.mainloop()`.

تمرین ۶: انتقال بین دو لیست

```
import tkinter as tk


win = tk.Tk()
win.geometry('420x300')
left = tk.Listbox(win, width=16, exportselection=False)
right = tk.Listbox(win, width=16, exportselection=False)
left.pack(side='left', padx=15, pady=15)
right.pack(side='right', padx=15, pady=15)
for name in ['علی', 'سارا', 'رضا', 'مریم']:
    left.insert(tk.END, name)


def move_right():
    sel = left.curselection()
    if sel:
        right.insert(tk.END, left.get(sel[0]))
        left.delete(sel[0])


def move_left():
    sel = right.curselection()
    if sel:
        left.insert(tk.END, right.get(sel[0]))
        right.delete(sel[0])


tk.Button(win, text='>>', command=move_right).pack(pady=40)
tk.Button(win, text='<<', command=move_left).pack()
win.mainloop()
```

پاسخ‌های درس ۸: تابع‌ها

پاسخ کوییز: س۱: ب. س۲: ب. س۳: ب. س۴: ج. س۵: ب.

حل تمرین‌های خانه:

تمرین ۱: تابع اول بودن

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
            return False
    return True


print(is_prime(7))
print(is_prime(10))
```

تمرین ۲: میانگین

```
def average(nums):
    return sum(nums) / len(nums)


grades = []
for i in range(3):
    grades.append(float(input('نمره: ')))
print('میانگین:', average(grades))
```

تمرین ۳: تبدیل دما در پنجره

```
import tkinter as tk


def celsius_to_f(c):
    return c * 9 / 5 + 32


win = tk.Tk()
win.geometry('260x180')
entry = tk.Entry(win)
entry.pack(pady=10)
out = tk.Label(win, text='')
out.pack()


def convert():
    out.config(text=f'{celsius_to_f(float(entry.get())):.1f} فارنهایت')


tk.Button(win, text='تبدیل', command=convert).pack(pady=10)
win.mainloop()
```

تمرین ۴: شمارش حروف صدادار

```
def count_vowels(text):
    count = 0
    for ch in text.lower():
        if ch in 'aeiou':
            count += 1
    return count


print(count_vowels('Hello World'))
```

تمرین ۵: اول است یا نه

```
import tkinter as tk


def is_prime(n):
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
            return False
    return True


win = tk.Tk()
win.geometry('260x180')
entry = tk.Entry(win)
entry.pack(pady=10)
out = tk.Label(win, text='', font=('Tahoma', 14))
out.pack()


def check():
    if is_prime(int(entry.get())):
        out.config(text='اول است', fg='green')
    else:
        out.config(text='اول نیست', fg='red')


tk.Button(win, text='بررسی', command=check).pack(pady=10)
win.mainloop()
```

تمرین ۶: فیبوناچی

```
import tkinter as tk


def fib(n):
    seq = [0, 1]
    while len(seq) < n:
        seq.append(seq[-1] + seq[-2])
    return seq[:n]


win = tk.Tk()
win.geometry('200x320')
lb = tk.Listbox(win, width=15, height=15)
lb.pack(padx=20, pady=20)
for x in fib(12):
    lb.insert(tk.END, x)
win.mainloop()
```

پاسخ‌های درس ۹: دیکشنری، تاپل و انتخاب‌ها

پاسخ کوییز: س۱: ج. س۲: ج. س۳: الف. س۴: ب. س۵: ب.

حل تمرین‌های خانه:

تمرین ۱: شمارش کلمه

```
text = input('یک متن: ')
counts = {}
for w in text.split():
    counts[w] = counts.get(w, 0) + 1
for w, c in counts.items():
    print(w, ':', c)
```

تمرین ۲: دیکشنری معنی

```
words = {'book': 'کتاب', 'pen': 'خودکار', 'school': 'مدرسه'}
w = input('کلمه انگلیسی: ')
print(words.get(w, 'در دیکشنری نیست'))
```

تمرین ۳: تکراری‌ها

```
nums = [1, 2, 2, 3, 3, 3, 4]
seen = set()
dups = set()
for n in nums:
    if n in seen:
        dups.add(n)
    seen.add(n)
print(dups)
```

تمرین ۴: فرم ثبت‌نام

```
import tkinter as tk


win = tk.Tk()
win.geometry('260x240')
gender = tk.StringVar(value='پسر')
news = tk.BooleanVar()
tk.Radiobutton(win, text='پسر', variable=gender, value='پسر').pack()
tk.Radiobutton(win, text='دختر', variable=gender, value='دختر').pack()
tk.Checkbutton(win, text='ثبتنام خبرنامه', variable=news).pack()
out = tk.Label(win, text='')
out.pack(pady=10)


def done():
    out.config(text=f'{gender.get()} - خبرنامه: {"بله" if news.get() else
"خیر"}')


tk.Button(win, text='ثبت', command=done).pack()
win.mainloop()
```

تمرین ۵: دکمه حذف

```
def remove():
    name = e_name.get()
    if name in book:
        del book[name]
        refresh()
    else:
        messagebox.showwarning('توجه', 'این نام نیست')


tk.Button(win, text='حذف', command=remove).pack(side='left', padx=5)
```

این تکه را به مثال ۱۲ اضافه کنید، قبل از `refresh()` انتهایی.

تمرین ۶: مترجم ساده

```
import tkinter as tk
from tkinter import ttk


words = {'apple': 'سیب', 'book': 'کتاب', 'cat': 'گربه',
         'water': 'آب', 'sun': 'خورشید'}


win = tk.Tk()
win.geometry('260x180')
combo = ttk.Combobox(win, values=list(words.keys()), state='readonly')
combo.current(0)
combo.pack(pady=15)
out = tk.Label(win, text='', font=('Tahoma', 14))
out.pack()


def translate():
    out.config(text=words[combo.get()])


tk.Button(win, text='ترجمه', command=translate).pack(pady=10)
win.mainloop()
```

پاسخ‌های درس ۱۰: کار با فایل

پاسخ کوییز: س۱: ج. س۲: ب. س۳: ب. س۴: ج. س۵: ج.

حل تمرین‌های خانه:

تمرین ۱: ذخیره و جمع

```
with open('nums.txt', 'w', encoding='utf-8') as f:
    for i in range(5):
        f.write(input('عدد: ') + '\n')

total = 0
with open('nums.txt', 'r', encoding='utf-8') as f:
    for line in f:
        total += float(line)
print('جمع:', total)
```

تمرین ۲: شمارش خط

```
with open('names.txt', 'r', encoding='utf-8') as f:
    lines = f.readlines()
print('تعداد خط:', len(lines))
```

تمرین ۳: مرتب‌سازی نام‌ها

```
with open('names.txt', 'r', encoding='utf-8') as f:
    names = f.read().splitlines()
names.sort()
with open('sorted.txt', 'w', encoding='utf-8') as f:
    for n in names:
        f.write(n + '\n')
print('ذخیره شد')
```

تمرین ۴: شمارش کلمه در فایل

```
word = input('کلمه: ')
with open('diary.txt', 'r', encoding='utf-8') as f:
    text = f.read()
print(word, 'تعداد:', text.count(word))
```

تمرین ۵: نمایش تعداد کلمه

```
import tkinter as tk

win = tk.Tk()
text = tk.Text(win, width=40, height=10)
text.pack(padx=10, pady=10)
info = tk.Label(win, text='کلمه ها: 0')
info.pack()

def count(event):
    n = len(text.get('1.0', tk.END).split())
    info.config(text=f'کلمه ها: {n}')

text.bind('<KeyRelease>', count)
win.mainloop()
```

تمرین ۶: دفترچه خاطرات

```
import tkinter as tk
from datetime import date

win = tk.Tk()
win.title('خاطرات')
text = tk.Text(win, width=40, height=8)
text.pack(padx=10, pady=10)

def record():
    with open('diary.txt', 'a', encoding='utf-8') as f:
        f.write(f'{date.today()}: {text.get("1.0", tk.END).strip()}\n')
    text.delete('1.0', tk.END)

tk.Button(win, text='ثبت', command=record).pack(pady=5)
win.mainloop()
```

پاسخ‌های درس ۱۱: مدیریت خطا، منو و لغزنده

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: ج. س۵: الف.

حل تمرین‌های خانه:

تمرین ۱: جمع عددهای معتبر

```
total = 0
for i in range(5):
    try:
        total += float(input('عدد: '))
    except ValueError:
        print('نادیده گرفته شد')
print('جمع:', total)
```

تمرین ۲: اندیس ایمن

```
names = ['علی', 'سارا', 'رضا']
try:
    i = int(input('اندیس: '))
    print(names[i])
except ValueError:
    print('عدد وارد کنید')
except IndexError:
    print('اندیس خارج از لیست است')
```

تمرین ۳: safe_int

```
def safe_int(text):
    try:
        return int(text)
    except ValueError:
        return None


print(safe_int('25'))
print(safe_int('abc'))
```

تمرین ۴: ماشین‌حساب ایمن

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.geometry('300x240')
e1 = tk.Entry(win)
e1.pack(pady=5)
e2 = tk.Entry(win)
e2.pack()
out = tk.Label(win, text='نتیجه')
out.pack(pady=10)


def calc(op):
    try:
        a = float(e1.get())
```

```
        b = float(e2.get())
        if op == '+':
            out.config(text=a + b)
        elif op == '-':
            out.config(text=a - b)
        elif op == '*':
            out.config(text=a * b)
        else:
            out.config(text=a / b)
    except ValueError:
        messagebox.showerror('خطا', 'عدد وارد کنید')
    except ZeroDivisionError:
        messagebox.showerror('خطا', 'تقسیم بر صفر')


for s in ['+', '-', '*', '/']:
    tk.Button(win, text=s, width=4,
              command=lambda x=s: calc(x)).pack(side='left', padx=8)
win.mainloop()
```

تمرین ۵: تغییر اندازه فونت

```
import tkinter as tk


win = tk.Tk()
win.geometry('360x240')
lbl = tk.Label(win, text='سلام', font=('Tahoma', 8))
lbl.pack(pady=20)


def resize(value):
    lbl.config(font=('Tahoma', int(value)))


tk.Scale(win, from_=8, to=40, orient='horizontal', length=260,
         command=resize).pack()
win.mainloop()
```

تمرین ۶: نوت‌پد با منوی کامل

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.title('نوت‌پد')
text = tk.Text(win, width=50, height=15)
text.pack(padx=10, pady=10)


def new():
    text.delete('1.0', tk.END)


def save():
    with open('note.txt', 'w', encoding='utf-8') as f:
```

```
        f.write(text.get('1.0', tk.END))

def load():
    try:
        with open('note.txt', 'r', encoding='utf-8') as f:
            text.delete('1.0', tk.END)
            text.insert('1.0', f.read())
    except FileNotFoundError:
        messagebox.showwarning('توجه', 'فایلی برای باز کردن نیست')

menubar = tk.Menu(win)
filemenu = tk.Menu(menubar, tearoff=0)
filemenu.add_command(label='جدید', command=new)
filemenu.add_command(label='باز کردن', command=load)
filemenu.add_command(label='ذخیره', command=save)
filemenu.add_separator()
filemenu.add_command(label='خروج', command=win.destroy)
menubar.add_cascade(label='فایل', menu=filemenu)
win.config(menu=menubar)
win.mainloop()
```

پاسخ‌های درس ۱۲: پروژه‌های پایانی

پاسخ کوییز (آزمون پایانی): س۱: ب. س۲: ب. س۳: ب. س۴: الف. س۵: ج. س۶: الف. س۷: د. س۸: الف. س۹: ب.

س۱۰: ب.

توضیح چند پاسخ: س۱: `10 // 3` برابر ۳ و `10 % 3` برابر ۱ است. س۴: برش از اندیس ۱ تا ۳ است، یعنی `iro`.

س۶: بعد از افزودن لیست `[3, 1, 2, 0]` است و مرتب‌سازی آن را `[0, 1, 2, 3]` می‌کند.

حل تمرین‌های خانه:

تمرین ۱ (کلاسی): ثبت با Enter

```
entry.bind('<Return>', lambda e: add())
```

این خط را قبل از `win.mainloop()` در دفترچه کارها بگذارید.

تمرین ۲ (کلاسی): بازی جدید

```
def new_game():
    global secret, tries
    secret = random.randint(1, 100)
    tries = 0
    msg.config(text='', fg='black')
    entry.delete(0, tk.END)


tk.Button(win, text='بـازی جدیـد', command=new_game).pack(pady=5)
```

ایده ۲: ساعت دیجیتال

```
import time
import tkinter as tk


win = tk.Tk()
win.title('سـاعت')
win.geometry('260x120')
lbl = tk.Label(win, font=('Tahoma', 32))
lbl.pack(expand=True)


def tick():
    lbl.config(text=time.strftime('%H:%M:%S'))
    win.after(1000, tick)


tick()
win.mainloop()
```

ایده ۴: سنگ، کاغذ، قیچی

```
import random
import tkinter as tk


win = tk.Tk()
win.title('سنگ کاغذ قیچی')
win.geometry('320x220')
out = tk.Label(win, text='انتخاب کنید', font=('Tahoma', 12))
out.pack(pady=20)


def play(mine):
    pc = random.choice(['سنگ', 'کاغذ', 'قیچی'])
    wins = [('سنگ', 'قیچی'), ('کاغذ', 'سنگ'), ('قیچی', 'کاغذ')]
    if mine == pc:
        result = 'مساوی'
    elif (mine, pc) in wins:
        result = 'شما بردید'
    else:
        result = 'رایانه برد'
    out.config(text=f'شما: {mine} - رایانه: {pc}\n{result}')


for name in ['سنگ', 'کاغذ', 'قیچی']:
    tk.Button(win, text=name, width=8,
              command=lambda n=name: play(n)).pack(side='left', padx=10)
win.mainloop()
```

ایده ۸: دوز (راهنمای ساخت)

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.title('دوز')
turn = {'p': 'X'}
buttons = []


def check():
    b = [x['text'] for x in buttons]
    lines = [(0, 1, 2), (3, 4, 5), (6, 7, 8), (0, 3, 6),
             (1, 4, 7), (2, 5, 8), (0, 4, 8), (2, 4, 6)]
    for a, c, d in lines:
        if b[a] != '' and b[a] == b[c] == b[d]:
            return b[a]
    return None


def click(i):
    if buttons[i]['text'] != '':
        return
    buttons[i].config(text=turn['p'])
```

```
    winner = check()
    if winner:
        messagebox.showinfo('پـایـان', f'بـرنـده: {winner}')
        win.destroy()
        return
    turn['p'] = 'O' if turn['p'] == 'X' else 'X'


for i in range(9):
    b = tk.Button(win, text='', width=6, height=3, font=('Tahoma', 16),
                  command=lambda i=i: click(i))
    b.grid(row=i // 3, column=i % 3)
    buttons.append(b)
win.mainloop()
```

این نسخه حالت مساوی را بررسی نمی‌کند. اضافه کردن آن را به‌عنوان تمرین انجام دهید.

پیوست الف: ریپوهای مرتبط و فایل‌های مهم

این پیوست نقشه راه شما به مجموعه فایل‌های آموزشگاه است. هر جا در کتاب گفته شد «مثال بیشتر»، به اینجا مراجعه کنید.

> **یادآوری:** توضیح فایل‌ها در این پیوست بر پایه نام پوشه‌ها و فایل‌ها و توضیحات `README` هر ریپو نوشته شده است. پیش از استفاده در کلاس، فایل را یک بار اجرا کنید.

۱. ریپوی کدهای همین کتاب: python-jozveh

همه کدهای این کتاب (مثال‌ها و حل تمرین‌ها) آماده اجرا در یک پوشه‌بندی منظم همراه کتاب داده شده است. ساختار پیشنهادی ریپو:

```
python-jozveh/
  README.md
  .gitignore
  requirements.txt
  lesson01/ ... lesson11/       # مثال‌های هر درس
      ex01_console.py
      ex06_gui.py
  answers/
      lesson01/ ... lesson11/   # حل تمرین‌های خانه (hwNN_*.py)
          snippets/             # تکه‌کدهایی که به مثال‌ها اضافه می‌شوند
  projects/                     # پروژه‌ها و ایده‌های پایانی درس ۱۲
  tools/smoke_test_gui.py       # آزمایش خودکار پنجره‌ها
```

نام فایل‌ها به شکل `exNN_console.py` یا `exNN_gui.py` است و `NN` شماره مثال در کتاب است. بالای هر فایل یک کامنت با شماره درس و مثال آمده است. در مجموع ۲۰۵ فایل کد دارد: ۱۳۲ مثال، ۶۰ حل تمرین، ۸ پروژه و ایده و ۵ تکه‌کد.

ساخت ریپو و بارگذاری:

```
cd python-jozveh
git init
git add .
git commit -m "Initial commit: Python book code"
git branch -M main
git remote add origin https://github.com/worker2025/python-jozveh.git
git push -u origin main
```

> **نکته:** پیش از `git push` ریپوی خالی `python-jozveh` را در GitHub بسازید (بدون README تا تداخل پیش نیاید).

۲. ریپوی ۱–Python (ترم اول پایتون)

آدرس: `https://github.com/tirotir-ir/Python-1`

ریپوی کلاسی ترم اول آموزشگاه است که بر اساس **هفته‌ها** مرتب شده است. پوشه‌های `week1` تا `week9` و پوشه `python_classes_examples` دارد. نمایش آنلاین آن در `https://tirotir-ir.github.io/Python-1/` در دسترس است.

دریافت ریپو:

```
git clone https://github.com/tirotir-ir/Python-1.git
```

فایل‌ها و پوشه‌های سطح اول:

- `README.md`: معرفی دوره و ساختار هفته‌ها
- `index.html` و `index.txt` و `README for index.md`: صفحه نمایشی ریپو
- `demo.mp4`: ویدیوی نمایشی
- `TK app`: یک نمونه برنامه گرافیکی Tkinter با چند پنجره (`Toplevel`) مثل پنجره ورود
- `python_classes_examples/`: آموزش کلاس‌ها (بخش «فراتر از کتاب» را ببینید)
- `LICENSE`: مجوز MIT

۳. نقشه درس‌های کتاب به فایل‌های ریپوی ۱–Python

برای هر درس، پوشه‌ها و فایل‌هایی که بیشترین ارتباط را دارند معرفی شده است. مسیرها نسبت به ریشه ریپو هستند.

درس ۱ و ۲: شروع، متغیر و ورودی

- `week1/VariablesandSimpleDataTypes/` شامل `hello_world.py`، `comment.py`، `name.py`، `full_name.py`، `apostrophe.py`
- `week1/hello.py`، `week1/sum1.py`، `week1/1.py`
- `week1/tk/` شامل `tk0.py`، `tk0001.py` برای اولین پنجره‌ها
- `week1/TextColor/`: رنگی کردن متن کنسول با `colorama` و `termcolor`
- `week1/sound/`: پخش صدا و ترکیب آن با پنجره (`soundTK1.py` تا `soundTK4.py`)

- `week1/video/`: پخش فیلم کوتاه (`video1.py`، `moon.mp4`)
- `week2/tk/tkhello.py`

درس ۳: عملیات ریاضی

- `week3/kg2g.py`، `week3/projects/calc2.py` و `convert1.py`
- `week6/projects/`: `math1.py`، `math2.py`، `math3.py`، `simplecalc.py`، `simplecalc2.py`
- `week8/tk/`: `simpleCalc.py`، `sum.py`، `temperature.py`
- `week2/algorithm/`: تصویر فلوچارت‌ها (`f2c.PNG`، `plus2.PNG`، `rectangle.PNG`)

درس ۴: رشته‌ها

- `week3/txt1.py`، `txt2.py`، `txt3.py`
- `week3/TextAnimFor/`: انیمیشن متن (`animateText1.py` تا `animateText5.py`)
- `week8/chapter_08/formatted_name.py`

درس ۵: شرط‌ها

- `week2/if/`: `if1.py` تا `if4.py`، `light_control_1.py`، `light_control_2.py`
- `week1/if_walking/`: تشخیص «پیاده‌روی، دویدن یا دوچرخه‌سواری» از روی سرعت (`speed0.py` تا `speed2.py`) همراه تصاویر؛ این مثال به کتابخانه `pillow` نیاز دارد
- `week4/chapter_04/`: `voting.py`، `amusement_park.py`، `toppings.py`
- `week3/projects/login2.py`: فرم ورود
- `week4/if Statements.txt`: یادداشت‌های درس

درس ۶: حلقه‌ها

- `week2/for/`: `for0.py` تا `for8.py` و `new-for0001.py` تا `new-for0007.py`
- `week3/while1/` و `week3/demo/newWhile/`
- `week3/chapter_03/`: `squares.py`، `even_numbers.py`، `first_numbers.py`
- `week6/chapter_06/`: `counting.py`، `parrot.py`، `rollercoaster.py`، `even_or_odd.py`
- `week3/turtle/`: نقاشی با لاک‌پشت

درس ۷: لیست‌ها

- `week2/chapter_02_Introducing Lists/`: `bicycles.py`، `cars.py`، `motorcycles.py`
- `week2/listsNote.txt` و `week3/Working with Lists.txt`
- `week3/chapter_03/`: `magicians.py`، `players.py`، `foods.py`
- `week4/projects/flashcard2.py` و `flashcards.txt`

درس ۸: تابع‌ها

- `week3/function/`: `hi.py`، `ali.py`، `zarb.py`، `testfunc.py`
- `week3/demo/newDef/`
- `week8/chapter_08/`: `greeter.py`، `person.py`، `pizza.py`، `printing_models.py`، `user_profile.py`
- `week1/func-takbir.py`

درس ۹: دیکشنری و انتخاب‌ها

- `week5/Dictionaries.txt`
- `week5/projects/phone1.py` (دفترچه تلفن) و `quiz1.py` (آزمون)
- `week5/tk/`: مثال‌های گرافیکی

درس ۱۰: فایل‌ها

- `week7/files/`: `read1.py`، `write1.py`، `append1.py`، `del1.py`
- `week7/files/reading_from_a_file/`، `writing_to_a_file/`، `storing_data/` (ذخیره با JSON)
- `week6/data.csv` و `week6/data1.py`: کار با CSV
- `week7/projects/toDoList2.py` و `week8/tk/toDoList.py`

درس ۱۱: خطا و منو

- `week7/files/exceptions/`: `division_calculator.py`، `alice.py`، `word_count.py`
- `week8/tk/`: نمونه‌های رابط گرافیکی دیگر

درس ۱۲: پروژه‌ها

- `week8/tk/`: `guess1.py` (حدس عدد)، `clock1.py` (ساعت)، `traffic.py` (چراغ راهنما با `Canvas`)، `draw1.py`، `color1.py`
- `week8/projects/`: `dice.py` (تاس با کاراکترهای ASCII)، `hamster1.py` تا `hamster3.py` (بازی کلیک با GIF و `pillow`، همراه فایل exe نمونه)، `plan1.py` (درخت تصمیم با `scikit-learn`)
- `week7/projects/morse/` و `week8/tk/morse1.py`: مترجم کد مورس
- `week4/projects/`: برنامه یادآور دارو با SQLite و Tkinter (`drug3.py`، `drugSqlite1.py`)
- `week9/`: فایل‌های `02.py` تا `06.py`: تبدیل متن به گفتار در پنجره با `gTTS` (و `pygame` برای پخش)

۴. فراتر از کتاب: کلاس‌ها (OOP)

پوشه `python_classes_examples/` یک راهنمای قدم‌به‌قدم برای برنامه‌نویسی شی‌گراست و پس از این کتاب بهترین گام بعدی است. فایل‌ها به ترتیب شماره‌گذاری شده‌اند:

- `01_without_classes_cars.py`: مدل‌سازی ماشین‌ها بدون کلاس
- `02_with_classes_car.py`: همان مسئله با کلاس `Car`
- `03_bank_account.py`: کلاس حساب بانکی
- `04_school_system/`: پروژه چندفایلی مدرسه (`student.py`، `teacher.py`، `course.py`، `exam.py`، `grade.py`، `main.py`، `uml.txt`)
- `run_all.sh` و `run_all.bat`: اجرای سریع نمونه‌ها

```
python 02_with_classes_car.py
python 04_school_system/main.py
```

۵. ریپوی PythonKids_Lessons_ENG

آدرس: `https://github.com/tirotir-ir/PythonKids_Lessons_ENG`

برگه تقلب بزرگ و نوت‌بوک‌های داستان‌محور برای کودکان. فایل‌های مهم:

- `Python_TirotirPy_Combined_EN.html`: برگه تقلب جامع (منبع اصلی پیوست ب)
- `PythonKids_EN_Part1_01-12.ipynb`، `Part2_13-24.ipynb`، `Part3_25-36.ipynb`: ۳۶ درس نوت‌بوک
- `index.html`: صفحه نمایشی (نسخه آنلاین: `https://tirotir-ir.github.io/PythonKids_Lessons_ENG/index.html`)
- `tirotir-logo.svg`: آرم

اجرای نوت‌بوک‌ها:

```
pip install jupyter
jupyter lab
```

۶. اکستنشن‌های VS Code

- **izeh**: اسنیپت‌های پایتون. مخزن: `https://github.com/tirotir-ir/izeh`
- **Tirotir**: اسنیپت‌های Tkinter و ابزارهای آموزشگاه، در بازار اکستنشن‌ها (Marketplace) با ناشر `DaryoushAlipourtirotir`
- **Auto Python Installer**: نصب خودکار پایتون، از همان ناشر

> **نکته:** برای پیدا کردن اکستنشن‌ها در VS Code با Ctrl+Shift+X نام آنها را جستجو کنید.

۷. مسیر پیشنهادی یادگیری با این منابع

۱. درس‌های این کتاب را به ترتیب بخوانید و کدهای `python-jozveh` را اجرا کنید.
۲. هر هفته، پوشه هفته مربوط در `Python-1` را باز کنید و فایل‌ها را اجرا و دستکاری کنید.
۳. هر وقت چیزی را فراموش کردید، به برگه تقلب (پیوست ب) نگاه کنید.
۴. پس از پایان، سراغ `python_classes_examples` و نوت‌بوک‌های `PythonKids_Lessons_ENG` بروید.

پیوست ب: برگه تقلب جامع پایتون

این برگه تقلب بر پایه ریپوی `PythonKids_Lessons_ENG` (فایل `Python_TirotirPy_Combined_EN.html`) و مطالب کتاب، به فارسی و فشرده بازنویسی شده است. برای هر موضوع یک قطعه کد کوتاه و کاربردی آمده است. نسخه آنلاین با دکمه کپی و جستجو در همان ریپو موجود است.

> **نکته:** برای پیدا کردن سریع، از Ctrl+F در فایل استفاده کنید. عنوان‌ها ساده و به ترتیب موضوع هستند.

۱. پایه‌ها

چاپ و ورودی

```
print('الف', 'ب', sep='-', end='!\n')   # الف-ب!
name = input('نام؟ ')                    # همیشه str
age = int(input('سن؟ '))                 # تبدیل به عدد
print(f'{name} {age}')                   # f-string
```

نوع‌ها و تبدیل

```
type(5), type(5.0), type('a'), type(True), type([]), type({}), type(())
isinstance(5, int)          # True (برای بررسی نوع بهتر از type)
int('12'), float('1.5'), str(7), bool(0), list('abc'), tuple([1, 2])
x: int = 5                  # اجباری نیست (یادداشت نوع)
a, b = 1, 2                 # انتساب چندتایی
a, b = b, a                 # جابه‌جایی
```

عملگرها

```
7 + 2, 7 - 2, 7 * 2, 7 / 2, 7 // 2, 7 % 2, 2 ** 5
x += 1; x -= 1; x *= 2; x /= 2          # عملگرهای ترکیبی
a == b, a != b, a < b, a >= b           # مقایسه
a and b, a or b, not a                  # منطقی
'a' in 'cat', 3 in [1, 2, 3]            # عضویت
x is None, x is not None                # مقایسه با None
```

قالب‌بندی عدد و متن

```
n = 1234.5678
f'{n:.2f}'      # 1234.57  دو رقم اعشار
f'{n:,.1f}'     # 1,234.6  جداکننده هزارگان
f'{42:05d}'     # 00042    صفر در ابتدا
f'{255:x}'      # ff       مبنای ۱۶
f'{255:b}'      # 11111111 مبنای ۲
f'{"سلام":>10}' # راست‌چین در عرض ۱۰
f'{0.256:.1%}'  # 25.6%
f'{x = }'       # چاپ نام و مقدار (برای اشکال‌زدایی)
```

۲. رشته‌ها

```
s = 'Python Course'
s[0], s[-1], s[0:6], s[:6], s[7:], s[::-1], s[::2]
len(s); s.upper(); s.lower(); s.title(); s.swapcase(); s.capitalize()
s.strip(); s.lstrip(); s.rstrip(); s.strip('!')
s.replace('Python', 'Java'); s.find('C'); s.index('C'); s.count('o')
s.startswith('Py'); s.endswith('se'); s.isdigit(); s.isalpha(); s.isalnum()
s.split(); s.split(','); ' - '.join(['a', 'b', 'c']); s.splitlines()
s.center(30, '*'); s.zfill(20); 'ab' * 3; s + '!'
```

یادآوری: رشته‌ها تغییرناپذیرند. متدهایی مثل `upper` رشته جدید می‌سازند و رشته اصلی را عوض نمی‌کنند.

۳. لیست، تاپل، مجموعه، دیکشنری

لیست

```
a = [3, 1, 2]
a.append(4); a.insert(0, 9); a.extend([7, 8])
a.remove(9); a.pop(); a.pop(0); del a[0]
a.sort(); a.sort(reverse=True); a.reverse(); sorted(a); reversed(a)
len(a), sum(a), min(a), max(a), a.index(2), a.count(2), a.copy()
a[1:3], a[::-1], a[-1], a + [5], a * 2, 3 in a
```

تاپل و مجموعه

```
t = (1, 2, 3)              # تغییرناپذیر
x, y, z = t                # باز کردن (unpacking)
s = {1, 2, 3}
s.add(4); s.remove(1); s.discard(99)    # discard خطا نمی‌دهد
{1, 2} | {2, 3}            # اجتماع  {1, 2, 3}
{1, 2} & {2, 3}            # اشتراک  {2}
{1, 2} - {2, 3}            # تفاضل   {1}
list(set([1, 1, 2, 2]))    # حذف تکراری‌ها
```

دیکشنری

```
d = {'a': 1, 'b': 2}
d['c'] = 3; d['a'] = 10; del d['b']
d.get('x'); d.get('x', 0); d.setdefault('k', [])
d.keys(); d.values(); d.items(); 'a' in d; len(d)
d.pop('a'); d.update({'z': 26}); d.copy()
for k, v in d.items():
    print(k, v)
sorted(d.items(), key=lambda kv: kv[1])     # مرتب‌سازی بر اساس مقدار
{k: v for k, v in d.items() if v > 1}       # دیکشنری با فیلتر
```

پیمایش و ابزارها

```
for i, x in enumerate(['a', 'b'], start=1): print(i, x)
for a, b in zip([1, 2], ['x', 'y']): print(a, b)
sorted(words, key=len)                          # مرتب با کلید دلخواه
sorted(people, key=lambda p: p['age'])          # لیست دیکشنری
any([0, 1]), all([1, 1])                        # آیا حداقل یکی / همه درست‌اند
[x * 2 for x in range(5)]                       # فهرست‌سازی (comprehension)
[x for x in range(10) if x % 2 == 0]
{x for x in 'hello'}
{n: n * n for n in range(4)}
```

۴. کنترل جریان

```
if x > 0:
    ...
elif x == 0:
    ...
else:
    ...
label = 'زوج' if n % 2 == 0 else 'فرد'      # شرط یکخطی


for i in range(10): ...                     # 0..9
for i in range(2, 11, 2): ...               # گام ۲
while cond:
    if bad: break
    if skip: continue
else:
    print('تمام شد بدون break')             # برای حلقه else


match command:                              # از پایتون ۳٫۱۰
    case 'go':
        print('برو')
    case 'stop' | 'halt':
        print('بایست')
    case _:
        print('نامعتبر')
```

۵. تابع‌ها

```
def f(a, b=2, *args, **kwargs):
    return a + b


f(1); f(1, 5); f(1, b=5); f(1, 2, 3, 4, x=9)
nums = [1, 2, 3]
f(*nums[:2])                    # باز کردن لیست به آرگومان‌ها
square = lambda x: x * x
list(map(square, [1, 2, 3]))                # [1, 4, 9]
list(filter(lambda x: x > 1, [1, 2, 3]))    # [2, 3]


def gen(n):                     # مولد
    for i in range(n):
        yield i * i


from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

یادآوری: برای تغییر متغیر سراسری داخل تابع از `global` استفاده کنید و برای متغیر تابع بیرونی در تابع تودرتو از `nonlocal`.

۶. خطاها

```
try:
    x = int(input())
except ValueError:
    print('عدد نیست')
except (ZeroDivisionError, IndexError) as e:
    print('خطا:', e)
else:
    print('بدون خطا')
finally:
    print('همیشه')


raise ValueError('پیام')                  # ایجاد خطا
from contextlib import suppress
with suppress(FileNotFoundError):         # نادیده گرفتن یک خطا
    open('x.txt')
```

۷. کلاس‌ها

```
class Dog:
    species = 'سگ'                          # ویژگی کلاس
    def __init__(self, name):
        self.name = name                    # ویژگی شیء
    def speak(self):
        return f'{self.name}: هاپ!'
    def __str__(self):
        return f'Dog({self.name})'
    def __eq__(self, other):
        return self.name == other.name


class Puppy(Dog):                           # ارثبری
    def speak(self):
        return super().speak() + ' (کوچولو)'


from dataclasses import dataclass
@dataclass
class Point:
    x: int
    y: int = 0

print(Point(3, 4))                          # Point(x=3, y=4)
```

۸. فایل، مسیر و داده

```
with open('a.txt', 'w', encoding='utf-8') as f:
    f.write('سلام\n')
with open('a.txt', 'r', encoding='utf-8') as f:
    text = f.read(); lines = f.readlines()


from pathlib import Path
p = Path('data') / 'a.txt'
p.exists(); p.parent.mkdir(exist_ok=True)
p.write_text('متن', encoding='utf-8')
p.read_text(encoding='utf-8'); p.suffix; p.stem; list(Path('.').glob('*.py'))


import csv
with open('d.csv', 'w', newline='', encoding='utf-8') as f:
    csv.writer(f).writerow(['نام', 'نمره'])
with open('d.csv', newline='', encoding='utf-8') as f:
    for row in csv.reader(f): print(row)


import json
data = {'نام': 'علی', 'نمره': [18, 20]}
Path('d.json').write_text(json.dumps(data, ensure_ascii=False), encoding='utf-8')
data = json.loads(Path('d.json').read_text(encoding='utf-8'))


from datetime import datetime, timedelta, date
now = datetime.now()
now.strftime('%Y-%m-%d %H:%M')
now + timedelta(days=7)
date.today()
datetime.strptime('2025-08-30', '%Y-%m-%d')
```

۹. کتابخانه‌های استاندارد پرکاربرد

```
import math, random, time, os, sys
math.sqrt(16); math.pi; math.ceil(2.1); math.floor(2.9); math.gcd(12, 18)
random.randint(1, 6); random.choice(lst); random.shuffle(lst); random.random()
time.time(); time.sleep(1)
os.listdir('.'); os.getcwd(); os.path.exists('a.txt'); os.remove('a.txt')
sys.argv; sys.exit()


from collections import Counter, defaultdict, deque
Counter('banana').most_common(2)           # [('a', 3), ('n', 2)]
dd = defaultdict(list); dd['k'].append(1)
q = deque([1, 2]); q.appendleft(0); q.pop()


import itertools as it
list(it.permutations([1, 2, 3], 2))
list(it.combinations([1, 2, 3], 2))
list(it.product('ab', repeat=2))
list(it.islice(it.count(10), 3))           # [10, 11, 12]
for k, g in it.groupby(sorted('aabbc')): print(k, list(g))
```

عبارت‌های باقاعده (re)

```
import re
re.findall(r'\d+', 'a12b345')             # ['12', '345']
re.search(r'(\w+)@(\w+)\.com', 'x ali@mail.com')
re.sub(r'\s+', ' ', 'a   b    c')          # 'a b c'
re.match(r'^09\d{9}$', '09123456789')      # شماره موبایل
```

آرگومان خط فرمان و نخ

```
import argparse
ap = argparse.ArgumentParser()
ap.add_argument('--name', default='دنیا')
args = ap.parse_args()
print(args.name)


import threading
t = threading.Thread(target=lambda: print('در نخ دیگر'))
t.start(); t.join()
```

۱۰. محیط مجازی و pip

```
python -m venv .venv
.venv\Scripts\activate          # ویندوز
source .venv/bin/activate       # مک و لینوکس
pip install requests
pip install -r requirements.txt
pip freeze > requirements.txt
pip list
pip uninstall requests
deactivate
```

نکته: برای هر پروژه یک محیط مجازی بسازید تا نسخه کتابخانه‌ها با پروژه‌های دیگر قاطی نشود.

۱۱. Tkinter: خلاصه ابزارها

ویجت‌ها

- `Label(win, text='...')` متن ثابت
- `Button(win, text='...', command=f)` دکمه
- `Entry(win)` کادر یک‌خطی و `.get()`، `.delete(0, END)`، `.insert(0, 'x')`
- `Text(win)` کادر چندخطی و `.get('1.0', END)`
- `Listbox(win)` لیست و `.insert(END, x)`، `.curselection()`، `.delete()`
- `Checkbutton`، `Radiobutton` با `BooleanVar`، `StringVar`، `IntVar`
- `Scale(win, from_=0, to=100)` لغزنده
- `Spinbox(win, from_=0, to=10)` عدد با فلش
- `ttk.Combobox(win, values=[...])` لیست کشویی
- `Frame(win)` گروه‌بندی و `Canvas(win)` نقاشی
- `Menu(win)` منو و `messagebox` پیام‌ها و `filedialog` انتخاب فایل

چیدمان

```
w.pack(side='left', padx=5, pady=5, fill='x', expand=True)
w.grid(row=0, column=1, sticky='w')
w.place(x=20, y=30)
```

هشدار: در یک پنجره هرگز `pack` و `grid` را با هم برای فرزندان یک ظرف یکسان به کار نبرید. یکی را انتخاب کنید.

رویدادها، زمان‌بندی و نقاشی

```
w.bind('<Button-1>', f)              # کلیک چپ
w.bind('<Return>', f)                # Enter
w.bind('<KeyRelease>', f)            # رها کردن کلید
win.after(1000, tick)                # بعد از ۱ ثانیه
canvas = tk.Canvas(win, width=300, height=200, bg='white')
canvas.create_rectangle(10, 10, 100, 80, fill='red')
canvas.create_oval(120, 10, 200, 80, fill='green')
canvas.create_line(0, 0, 300, 200)
canvas.create_text(150, 150, text='سلام')
win.attributes('-topmost', True)     # همیشه بالا
win.resizable(False, False)
win.geometry('400x300+100+50')       # اندازه و مکان
```

۱۲. Turtle: نقاشی با لاک‌پشت

```
import turtle
t = turtle.Turtle()
t.speed(0); t.color('blue'); t.pensize(3)
for _ in range(4):
    t.forward(100); t.right(90)      # مربع
t.penup(); t.goto(-100, 50); t.pendown()
t.circle(50)
t.begin_fill(); t.circle(30); t.end_fill()
turtle.done()
```

۱۳. رنگی کردن متن و گفتار

```
from colorama import init, Fore, Back, Style      # pip install colorama
init(autoreset=True)
print(Fore.RED + 'خطا' + Style.RESET_ALL)


from termcolor import colored                     # pip install termcolor
print(colored('سلام', 'green', attrs=['bold']))


import pyttsx3                                    # pip install pyttsx3
engine = pyttsx3.init()
engine.say('Hello from Python')
engine.runAndWait()
```

۱۴. ساخت فایل اجرایی با PyInstaller

```
pip install --upgrade pyinstaller
pyinstaller --onefile app.py
pyinstaller --onefile --noconsole --name MyApp --icon app.ico app.py
pyinstaller --onefile --add-data "assets;assets" app.py      # ویندوز
pyinstaller --onefile --add-data "assets:assets" app.py      # مک و لینوکس
```

خروجی در پوشه `dist` است. پرچم‌های مهم: `--noconsole` (برنامه گرافیکی)، `--icon` (آیکون)، `--name` (نام خروجی)، `--hidden-import` (ماژول پنهان) و `--add-data` (فایل‌های کمکی).

۱۵. دکتر خطا: معنی پیام‌های رایج

- `SyntaxError`: نوشتار اشتباه. پرانتز، نقل‌قول، دونقطه را بررسی کنید.
- `IndentationError`: فرورفتگی نامناسب. فاصله‌ها و Tab را یکدست کنید.
- `NameError`: نام تعریف نشده یا غلط تایپی.
- `TypeError`: نوع داده نامناسب برای عملیات، مثل جمع متن با عدد.
- `ValueError`: مقدار نامناسب، مثل `int('abc')`.
- `IndexError`: اندیس خارج از لیست.
- `KeyError`: کلید ناموجود در دیکشنری.
- `AttributeError`: شیء آن ویژگی یا متد را ندارد.
- `ZeroDivisionError`: تقسیم بر صفر.
- `FileNotFoundError`: فایل یا مسیر پیدا نشد.
- `ModuleNotFoundError`: کتابخانه نصب نشده. `pip install` بزنید.
- `UnicodeDecodeError`: رمزگذاری اشتباه. `encoding='utf-8'` بگذارید.
- `RecursionError`: تابع بازگشتی بی‌پایان.

۱۶. میانبرهای VS Code

- `Ctrl+/` کامنت کردن خط
- `Ctrl+S` ذخیره و `Ctrl+Shift+S` ذخیره با نام دیگر
- `Ctrl+Shift+P` پالت دستورها
- `Ctrl+` (بک‌تیک) باز و بسته کردن ترمینال
- `Alt+↑` و `Alt+↓` جابه‌جایی خط
- `Shift+Alt+↓` کپی خط
- `Ctrl+D` انتخاب مورد بعدی هم‌نام
- `Ctrl+Space` پیشنهاد کدنویسی
- `F5` اجرا با اشکال‌زدا و `Ctrl+F5` اجرا بدون آن
- `Shift+Alt+F` مرتب‌سازی (فرمت) کد

۱۷. ۵۳ کتابخانه معروف پایتون

برای نصب هر کتابخانه: `pip install نـــام`.

وب و شبکه

- `requests`: ارسال درخواست HTTP
- `httpx`: درخواست HTTP مدرن (همزمان و ناهمزمان)
- `aiohttp`: HTTP ناهمزمان
- `websockets`: ارتباط وب‌سوکت
- `beautifulsoup4`: تجزیه HTML
- `lxml`: تجزیه سریع XML و HTML
- `scrapy`: خزنده وب
- `selenium`: خودکارسازی مرورگر

داده و علم

- `numpy`: آرایه و محاسبات عددی
- `pandas`: جدول داده (DataFrame)
- `matplotlib`: رسم نمودار
- `seaborn`: نمودار آماری زیبا
- `scipy`: محاسبات علمی
- `scikit-learn`: یادگیری ماشین
- `sympy`: ریاضیات نمادین
- `statsmodels`: مدل‌های آماری

تصویر، صدا و بازی

- `pillow`: کار با تصویر
- `opencv-python`: بینایی ماشین
- `pydub`: ویرایش صدا
- `pygame`: بازی‌سازی و پخش صدا
- `pyttsx3`: تبدیل متن به گفتار آفلاین
- `gTTS`: تبدیل متن به گفتار با گوگل
- `SpeechRecognition`: تبدیل گفتار به متن

سخت‌افزار و خودکارسازی

- `pyserial`: ارتباط سریال (آردوینو و ...)
- `pyautogui`: کنترل ماوس و صفحه‌کلید
- `schedule`: زمان‌بندی ساده کارها
- `APScheduler`: زمان‌بندی پیشرفته
- `paramiko`: اتصال SSH
- `fabric`: اجرای دستور از راه دور

وب‌اپلیکیشن و پایگاه داده

- `fastapi`: ساخت API سریع
- `flask`: چارچوب سبک وب
- `django`: چارچوب کامل وب
- `sqlalchemy`: ORM پایگاه داده
- `alembic`: مهاجرت پایگاه داده
- `psycopg2-binary`: اتصال به PostgreSQL
- `PyMySQL`: اتصال به MySQL
- `redis`: ارتباط با Redis
- `celery`: صف کارهای پس‌زمینه
- `kafka-python`: ارتباط با Kafka

ابزارهای توسعه و ظاهر

- `pydantic`: اعتبارسنجی داده
- `marshmallow`: تبدیل و اعتبارسنجی داده
- `attrs`: ساخت کلاس‌های داده
- `click`: ساخت دستورهای خط فرمان
- `typer`: خط فرمان مدرن
- `rich`: خروجی زیبا در ترمینال
- `loguru`: لاگ‌گیری ساده
- `tqdm`: نوار پیشرفت
- `colorama` و `termcolor`: رنگی کردن متن کنسول
- `pytest`: آزمون نرم‌افزار
- `hypothesis`: آزمون مبتنی بر ویژگی

ابر

- `boto3`: سرویس‌های Amazon AWS
- `google-cloud-storage`: فضای ذخیره‌سازی گوگل
- `azure-storage-blob`: فضای ذخیره‌سازی Azure

۱۸. ۲۰ ترفند داغ

```
# ۱) اشکال‌زدایی با f-string
x = 5; print(f'{x = }')                        # x = 5
# ۲) شروع شماره از ۱
for i, v in enumerate('ab', 1): print(i, v)
# ۳) ترانهاده ماتریس با zip
list(zip(*[[1, 2], [3, 4]]))                   # [(1, 3), (2, 4)]
# ۴) باز کردن ستاره‌ای
first, *middle, last = [1, 2, 3, 4]            # middle = [2, 3]
# ۵) شمارش با get
c = {}
for ch in 'hello': c[ch] = c.get(ch, 0) + 1
# ۶) گروه‌بندی با setdefault
g = {}
for w in ['ab', 'cd', 'ae']: g.setdefault(w[0], []).append(w)
# ۷) any و all
any(x > 3 for x in [1, 5]); all(x > 0 for x in [1, 5])
# ۸) مرتب‌سازی با کلید
sorted(['bb', 'a', 'ccc'], key=len, reverse=True)
# ۹) خواندن و نوشتن سریع با pathlib
from pathlib import Path
Path('a.txt').write_text('سلام', encoding='utf-8')
# ۱۰) نادیده گرفتن خطا
from contextlib import suppress
with suppress(KeyError): del {}['x']
# ۱۱) گروه‌بندی با groupby (پس از مرتب‌سازی)
import itertools
[(k, len(list(g))) for k, g in itertools.groupby(sorted('aabbbc'))]
# ۱۲) اندازه‌گیری سرعت
import timeit; timeit.timeit('sum(range(1000))', number=1000)
# ۱۳) عملگر والروس (:=)
data = [1, 2, 3]
if (n := len(data)) > 2: print('بزرگ است', n)
# ۱۴) فهرست‌سازی تودرتو
[(i, j) for i in range(2) for j in range(2)]
# ۱۵) خطای مشخص بگیرید، نه عمومی
try: int('x')
except ValueError: print('عدد نیست')
# ۱۶) dataclass به جای کلاس ساده
from dataclasses import dataclass
@dataclass
class P: x: int; y: int
# ۱۷) یادداشت نوع‌ها
```

```
def add(a: int, b: int) -> int: return a + b
# ۱۸) خط فرمان با argparse
import argparse; argparse.ArgumentParser().add_argument('--n', type=int)
# ۱۹) چاپ زیباتر
from pprint import pprint; pprint({'a': [1, 2, 3], 'b': {'c': 1}})
# ۲۰) نوشتن خطا در stderr
import sys; print('خطا!', file=sys.stderr)
```

> **نکته:** همه ترفندهای بالا را یکی‌یکی در ترمینال امتحان کنید. هرکدام ممکن است ساعت‌ها در کدنویسی شما صرفه‌جویی کند.

۱۹. ایده‌های بعدی برای یادگیری

- نوت‌بوک‌های کودکان در `PythonKids_Lessons_ENG` (۳۶ درس)
- مثال‌های کلاس‌ها در `python_classes_examples` از ریپوی `Python-1`
- ساخت برنامه‌های واقعی: ربات تلگرام، برنامه وب با `flask`، تحلیل داده با `pandas`
- یادگیری ماشین: ابتدا `numpy`، `pandas` و سپس `scikit-learn`

پیوست پ: ماژول‌ها و دستور import

در درس‌های کتاب چند بار نوشتیم `import math` یا `import random` بدون اینکه توضیح کاملی بدهیم. این پیوست نشان می‌دهد ماژول چیست، چطور از ماژول‌های آماده پایتون استفاده کنیم و چطور ماژول خودمان را بسازیم.

۱. ماژول چیست؟

ماژول یک فایل پایتون (`.py`) است که تابع، متغیر و کلاس‌هایی را برای استفاده در برنامه‌های دیگر نگه می‌دارد. پایتون صدها ماژول آماده دارد که به آنها **کتابخانه استاندارد** می‌گوییم و بدون نصب چیزی در دسترس‌اند. ماژول‌های دیگر را با `pip` نصب می‌کنیم و ماژول‌های خودمان را هم می‌توانیم بنویسیم.

دلیل استفاده از ماژول: کد تکراری ننویسیم، برنامه را به بخش‌های کوچک تقسیم کنیم و از کار دیگران استفاده کنیم.

۲. چهار شکل import

مثال ۱: شکل‌های مختلف import

```
import math
print(math.sqrt(25))

import math as m
print(m.pi)

from math import sqrt, pi
print(sqrt(16), pi)

from math import *
print(floor(2.7))
```

خروجی:

```
5.0
3.141592653589793
4.0 3.141592653589793
2
```

- `import math`: همه ماژول را می‌آورد و با `math.` به اجزایش دسترسی داریم.
- `import math as m`: نام کوتاه‌تر انتخاب می‌کنیم (در `numpy as np` و `pandas as pd` هم همین است).
- `from math import sqrt`: فقط همان تابع را مستقیم می‌آورد و دیگر نیازی به `math.` نیست.
- `from math import *`: همه نام‌ها را می‌ریزد داخل برنامه.

هشدار: از `import *` پرهیز کنید. نام‌های ماژول می‌توانند با متغیرها و تابع‌های خودتان تداخل کنند و پیدا کردن اینکه یک نام از کجا آمده سخت می‌شود.

نکته: `import` را معمولاً در ابتدای فایل و به ترتیب «استاندارد، کتابخانه‌های نصب‌شده، ماژول‌های خودمان» می‌نویسند.

۳. دیدن محتوای ماژول: dir و help

مثال ۲: کاوش یک ماژول

```
import math


print('sqrt' in dir(math))
print(math.__name__)
help(math.sqrt)
```

خروجی:

```
True
math
Help on built-in function sqrt in module math:


sqrt(x, /)
    Return the square root of x.
```

با `dir(ماژول)` فهرست نام‌های آن را می‌بینید و با `help(نام)` راهنمای هر کدام را. در `Python Shell` یا ترمینال هم همین‌ها کار می‌کنند.

۴. ماژول‌های استاندارد پرکاربرد

- `math`: ریشه، توان، مثلثات، `pi`، `ceil`، `floor`
- `random`: عدد و انتخاب تصادفی
- `statistics`: میانگین، میانه، نما
- `datetime`: تاریخ و زمان
- `time`: زمان و تأخیر (`sleep`)
- `os` و `pathlib`: کار با فایل‌ها و پوشه‌ها
- `json` و `csv`: ذخیره و خواندن داده ساختاریافته
- `re`: عبارت‌های باقاعده

- `collections`: `Counter`، `defaultdict`، `deque`
- `string`: ثابت‌هایی مثل حروف و ارقام
- `tkinter`: رابط گرافیکی
- `turtle`: نقاشی با لاک‌پشت
- `sqlite3`: پایگاه داده کوچک

مثال ۳: آمار با statistics

```
import statistics


grades = [18, 15, 20, 12, 17]
print('میانگین:', statistics.mean(grades))
print('میانه:', statistics.median(grades))
print('نما:', statistics.mode([1, 2, 2, 3]))
```

خروجی:

```
میانگین: 16.4
میانه: 17
نما: 2
```

مثال ۴: چند روز تا امتحان؟

```
from datetime import date


today = date(2026, 10, 30)
exam = date(2027, 1, 15)
print((exam - today).days, 'روز مانده')
print(exam.year, exam.month, exam.day)
```

خروجی:

```
77 روز مانده
2027 1 15
```

> **نکته:** برای تاریخ امروز از `date.today()` استفاده کنید. اینجا برای ثابت ماندن خروجی تاریخ را دستی نوشتیم.

مثال ۵: شمارش با Counter

```
from collections import Counter


c = Counter('banana')
print(c)
print(c.most_common(2))
```

خروجی:

```
Counter({'a': 3, 'n': 2, 'b': 1})
[('a', 3), ('n', 2)]
```

مثال ۶: رمز تصادفی با string و random

```
import random
import string

chars = string.ascii_letters + string.digits
password = ''.join(random.choice(chars) for _ in range(8))
print(password)
print(len(password))
```

خروجی:

```
aK3xQ9pZ
8
```

نکته: رمز بالا هر بار فرق می‌کند. برای رمزهای واقعی و امنیتی به‌جای `random` از ماژول `secrets` استفاده کنید.

۵. ساختن ماژول خودمان

هر فایل پایتون خودش یک ماژول است. دو فایل در یک پوشه بسازید.

مثال ۷: ماژول mymath و برنامه اصلی

```
# file: mymath.py
PI = 3.14159

def circle_area(r):
    return PI * r ** 2

def is_even(n):
    return n % 2 == 0
```

```
# file: main.py
import mymath

print(mymath.circle_area(2))
print(mymath.is_even(7))
print(mymath.PI)
```

خروجی:

```
12.56636
False
3.14159
```

هشدار: دو فایل باید در یک پوشه باشند و برنامه را از همان پوشه اجرا کنید. نام ماژول خودتان را مثل ماژول‌های استاندارد (`math`، `random`، `tkinter`) نگذارید، وگرنه پایتون فایل شما را به‌جای ماژول اصلی بارگذاری می‌کند.

۶. شرط `if __name__ == '__main__'`

وقتی ماژولی را `import` می‌کنیم، پایتون همه کدهای بیرون از تابع آن را اجرا می‌کند. اگر بخواهیم بخشی از کد (مثل آزمایش) فقط وقتی اجرا شود که فایل را مستقیم اجرا می‌کنیم، آن را زیر این شرط می‌نویسیم. هر ماژول متغیری به نام `__name__` دارد: وقتی مستقیم اجرا شود مقدارش `'__main__'` است و وقتی `import` شود نام ماژول است.

مثال ۸: ماژول با بخش آزمایش

```
# file: tools.py
def double(x):
    return x * 2


if __name__ == '__main__':
    print('آزمایش مستقیم:', double(21))
```

خروجی:

```
42: آزمایش مستقیم
```

اگر در فایل دیگری `import tools` کنید، این `print` اجرا نمی‌شود ولی `tools.double` در دسترس است.

۷. بسته (package)

پوشه‌ای که چند ماژول دارد و فایلی به نام `__init__.py` در آن است یک **بسته** است:

```
myapp/
  main.py
  tools/
    __init__.py
    texts.py
    numbers.py

from tools.numbers import double
from tools import texts
```

۸. ماژول‌های نصب‌شدنی و خطای ModuleNotFoundError

ماژول‌هایی که با پایتون نیامده‌اند را با `pip` نصب می‌کنیم:

```
python -m pip install colorama
python -m pip list
```

اگر ماژولی نصب نباشد، با خطای زیر روبه‌رو می‌شوید:

```
ModuleNotFoundError: No module named 'colorama'
```

راه‌حل: (۱) نام را درست نوشته باشید، (۲) با `pip` نصب کنید، (۳) مطمئن شوید همان مفسر پایتونی که برنامه را اجرا می‌کند همان است که ماژول را نصب کرده‌اید (در VS Code گزینه Python: Select Interpreter).

> **یادآوری:** برای هر پروژه محیط مجازی بسازید (`python -m venv .venv`) تا نسخه کتابخانه‌ها پروژه‌ها را خراب نکنند.

کوییز

س۱. کدام دستور فقط تابع `sqrt` را از `math` می‌آورد؟

الف) `import math.sqrt`

ب) `from math import sqrt`

ج) `import sqrt from math`

د) `from sqrt import math`

س۲. در `import random as rnd` نام `rnd` چیست؟

الف) نام ماژول اصلی

ب) نام مستعار (alias)

ج) نام فایل

د) نام تابع

س۳. مقدار `__name__` وقتی فایل مستقیم اجرا شود چیست؟

الف) `'main'`

ب) `'__main__'`

ج) نام فایل

د) `None`

س۴. چرا فایلی به نام `random.py` نباید بسازیم؟

الف) چون پایتون آن را اجرا نمی‌کند

ب) چون با ماژول استاندارد `random` تداخل پیدا می‌کند

ج) چون پسوند آن اشتباه است

د) چون کوچک است

تمرین

۱. ماژولی به نام `converters.py` بسازید که دو تابع `c_to_f(c)` (سانتی‌گراد به فارنهایت) و `km_to_mile(km)` (هر کیلومتر ۰٫۶۲۱۳۷۱ مایل) داشته باشد. در `main.py` آن را `import` کنید و هر دو را آزمایش کنید.

۲. با ماژول `statistics` میانگین، میانه و نمای لیست `[12, 15, 15, 18, 20, 20, 20]` را چاپ کنید.

۳. با `random` و `string` برنامه‌ای بنویسید که یک رمز ۱۰ حرفی فقط از حروف کوچک بسازد.

۴. با `datetime` بنویسید تعداد روزهای بین دو تاریخ دلخواه چند است.

> **راهنمایی:** برای تمرین ۳ از `string.ascii_lowercase` و `random.choice` استفاده کنید.

پاسخ کوییز و تمرین

پاسخ کوییز: س۱: ب. س۲: ب. س۳: ب. س۴: ب.

حل تمرین ۱

```
# file: converters.py
def c_to_f(c):
    return c * 9 / 5 + 32


def km_to_mile(km):
    return km * 0.621371
```

```
# file: main.py
import converters


print(converters.c_to_f(100))
print(round(converters.km_to_mile(10), 2))
```

خروجی:

```
212.0
6.21
```

حل تمرین ۲

```
import statistics


nums = [12, 15, 15, 18, 20, 20, 20]
print(statistics.mean(nums))
print(statistics.median(nums))
print(statistics.mode(nums))
```

خروجی:

```
17.142857142857142
18
20
```

حل تمرین ۳

```
import random
import string


password = ''.join(random.choice(string.ascii_lowercase) for _ in range(10))
print(password)
```

حل تمرین ۴

```
from datetime import date


d1 = date(2026, 3, 21)
d2 = date(2026, 12, 21)
print((d2 - d1).days)
```

خروجی:

```
275
```

پیوست ت: اشکال‌زدایی (Debugging)

هیچ برنامه‌نویسی نیست که برنامه‌اش بار اول بی‌خطا کار کند. تفاوت مبتدی و حرفه‌ای این نیست که اشتباه نمی‌کند، این است که **می‌داند چطور اشتباه را سریع پیدا کند**. این پیوست ابزارها و روش‌های پیدا کردن و رفع اشکال را مرحله‌به‌مرحله نشان می‌دهد.

۱. سه نوع خطا

- **خطای نحوی (Syntax Error)**: نوشتار غلط است (دونقطه یا پرانتز جا افتاده). برنامه اصلاً شروع نمی‌شود و پیام خطا شماره خط را می‌دهد.
- **خطای زمان اجرا (Runtime Error)**: برنامه شروع می‌شود ولی وسط کار می‌ایستد، مثل `ZeroDivisionError` یا `ValueError`.
- **خطای منطقی (Logic Error)**: برنامه بدون هیچ پیامی اجرا می‌شود، اما **جواب غلط** می‌دهد. پیدا کردن این نوع از همه سخت‌تر است.

> **نکته:** بیشتر وقت اشکال‌زدایی صرف خطاهای منطقی می‌شود، چون پایتون هیچ سرنخی نمی‌دهد. ابزارهای این پیوست بیشتر برای همین‌هاست.

۲. خواندن پیام خطا (Traceback)

مثال ۱: یک برنامه با خطا

```
def average(nums):
    return sum(nums) / len(nums)


grades = []
print(average(grades))
```

```
Traceback (most recent call last):
  File "demo.py", line 5, in <module>
    print(average(grades))
  File "demo.py", line 2, in average
    return sum(nums) / len(nums)
ZeroDivisionError: division by zero
```

روش خواندن: از **پایین** شروع کنید.

۱. آخرین خط نوع و علت را می‌گوید: `ZeroDivisionError: division by zero`.

۲. خط بالاتر محل دقیق خطا را نشان می‌دهد: خط ۲ داخل تابع `average`.

۳. خط‌های بالاتر زنجیره صدا زدن تابع‌ها را نشان می‌دهند: خط ۵ برنامه، `average` را صدا زده است.

نتیجه: لیست خالی بوده و `len` صفر شده است. باید پیش از تقسیم خالی بودن لیست را بررسی کنیم.

> **یادآوری:** در نسخه‌های جدید پایتون زیر خط خطادار علامت `^^^^` هم نمایش داده می‌شود تا قسمت مشکل‌دار را نشان دهد.

۳. روش اول: ردیابی با print

ساده‌ترین روش این است که در چند نقطه برنامه مقدار متغیرها را چاپ کنیم و ببینیم کجا از انتظار ما فاصله می‌گیرند.

مثال ۲: پیدا کردن خطای منطقی با print

```
a, b, c = 18, 15, 12
average = a + b + c / 3
print(average)
```

خروجی:

```
37.0
```

انتظار داشتیم ۱۵ شود. مقدار هر قسمت را چاپ می‌کنیم:

```
a, b, c = 18, 15, 12
print(f'{c / 3 = }')
print(f'{a + b + c / 3 = }')
print(f'{(a + b + c) / 3 = }')
```

خروجی:

```
c / 3 = 4.0
a + b + c / 3 = 37.0
(a + b + c) / 3 = 15.0
```

مشکل از اولویت عملگرهاست: تقسیم قبل از جمع انجام می‌شود. با پرانتز درست می‌شود.

> **نکته:** `f'{x = }'` هم نام و هم مقدار را چاپ می‌کند و برای اشکال‌زدایی عالی است. پس از رفع اشکال `print`های اضافی را پاک کنید.

۴. روش دوم: اشکال‌زدای VS Code

وقتی برنامه بزرگ می‌شود، پر کردن آن با `print` سخت است. **اشکال‌زدا** (**Debugger**) اجازه می‌دهد برنامه را **خط‌به‌خط** اجرا کنید و مقدار همه متغیرها را در هر لحظه ببینید.

مثال ۳: برنامه‌ای برای قدم‌زدن

```
def total_price(prices, tax):
    total = 0
    for p in prices:
        total += p
    return total * (1 + tax)

items = [100, 250, 75]
result = total_price(items, 0.25)
print(result)
```

خروجی:

```
531.25
```

مراحل:

۱. روی حاشیه خاکستری **سمت چپ شماره خط** `total = 0` کلیک کنید تا یک نقطه قرمز (**Breakpoint**) ظاهر شود.
۲. کلید `F5` را بزنید و گزینه Python File را انتخاب کنید. برنامه اجرا می‌شود و روی همان خط می‌ایستد.
۳. در پنل **Variables** (سمت چپ) مقدار متغیرها را ببینید.
۴. با `F10` یک خط جلو بروید. هر بار ببینید `total` و `p` چطور عوض می‌شوند.
۵. هر وقت خواستید تا breakpoint بعدی یا پایان برنامه اجرا شود `F5` را بزنید و برای توقف `Shift+F5`.

کلیدهای اشکال‌زدا:

- `F5`: شروع یا ادامه اجرا
- `F10`: یک خط جلو، بدون رفتن به داخل تابع (Step Over)
- `F11`: رفتن به داخل تابعی که روی آن هستید (Step Into)
- `Shift+F11`: خارج شدن از تابع فعلی (Step Out)
- `Shift+F5`: توقف اشکال‌زدا

پنل‌های مهم:

- **Variables**: مقدار همه متغیرهای فعلی
- **Watch**: عبارتی (مثل `total * 2`) که می‌خواهید دائم زیر نظر باشد
- **Call Stack**: زنجیره تابع‌هایی که صدا زده شده‌اند
- **Debug Console**: اجرای عبارت‌های پایتون هنگام توقف

نکته: با راست‌کلیک روی breakpoint و انتخاب Edit Breakpoint می‌توانید **شرط** بگذارید، مثلاً `p > 200`.

برنامه فقط وقتی می‌ایستد که شرط درست باشد؛ در حلقه‌های طولانی بسیار مفید است.

هشدار: اشکال‌زدا برنامه‌های گرافیکی را هم اجرا می‌کند ولی وقتی پنجره باز است و به `mainloop` رسیده‌اید، ایستادن فقط روی خط‌هایی در توابع دکمه‌ها (callback) معنی دارد. breakpoint را داخل تابع دکمه بگذارید و روی دکمه کلیک کنید.

۵. روش سوم: assert و logging

`assert` شرطی را بررسی می‌کند که باید همیشه درست باشد. اگر غلط بود، برنامه با پیام شما می‌ایستد.

مثال ۴: assert

```
def average(nums):
    assert len(nums) > 0, 'لیست خالی است'
    return sum(nums) / len(nums)

print(average([1, 2, 3]))
try:
    print(average([]))
except AssertionError as e:
    print('خطا:', e)
```

خروجی:

```
2.0
خطا: لیست خالی است
```

`logging` مثل `print` است ولی سطح‌بندی دارد و با یک تغییر می‌توان آن را خاموش کرد.

مثال ۵: logging

```
import logging

logging.basicConfig(level=logging.DEBUG, format='%(levelname)s: %(message)s')
x = 5
logging.debug(f'x = {x}')
logging.info('شروع محاسبه')
logging.warning('مقدار زیاد است')
```

خروجی:

```
DEBUG: x = 5
INFO: شروع محاسبه
WARNING: مقدار زیاد است
```

> **نکته:** برای خاموش کردن پیام‌های debug، فقط `level=logging.INFO` یا `logging.WARNING` بگذارید؛ نیازی نیست خط‌ها را پاک کنید.

۶. استراتژی ده‌مرحله‌ای رفع اشکال

۱. **پیام خطا را کامل بخوانید**، از خط آخر شروع کنید.

۲. **خطا را دوباره تولید کنید**: با کدام ورودی اتفاق می‌افتد؟ همیشه یا گاهی؟

۳. **مسئله را کوچک کنید**: بخش‌های غیرمرتبط را موقتاً کامنت کنید تا کوچک‌ترین برنامه‌ای که هنوز خطا می‌دهد باقی بماند.

۴. **حدس بزنید** مشکل کجاست و برای آن حدس دلیل بیاورید.

۵. **حدس را بیازمایید** با `print`، `assert` یا اشکال‌زدا.

۶. **یک تغییر در هر نوبت** بدهید. اگر چند چیز را با هم عوض کنید نمی‌فهمید کدام اثر داشته است.

۷. **بعد از هر تغییر** برنامه را دوباره اجرا کنید.

۸. اگر بعد از ۱۵ تا ۲۰ دقیقه گیر کردید **استراحت** کنید یا مسئله را برای دیگری توضیح دهید.

۹. وقتی اشکال را پیدا کردید، **علتش را بفهمید** و فقط جواب را کپی نکنید.

۱۰. **آزمون کنید** که اشکال برنگردد و تغییر جدید چیز دیگری را خراب نکرده باشد.

> **نکته:** «روش اردک لاستیکی»: مشکل را با صدای بلند برای یک اردک اسباب‌بازی (یا هر کسی) توضیح دهید. خیلی وقت‌ها در همان توضیح دادن، خودتان اشتباه را می‌بینید.

۷. چطور سؤال خوب بپرسیم؟

وقتی از معلم، دوست یا یک انجمن کمک می‌خواهید این‌ها را همراه بپرسید:

- **چه می‌خواستید؟** (نتیجه مورد انتظار)
- **چه شد؟** (نتیجه واقعی یا متن کامل پیام خطا)
- **کوچک‌ترین کدی** که مشکل را نشان می‌دهد (نه کل برنامه)
- **چه چیزهایی را امتحان کردید؟**
- **نسخه پایتون و سیستم‌عامل**

۸. خطاهای رایج مبتدی‌ها: چک‌لیست

- دونقطه `:` بعد از `if`، `for`، `while`، `def`، `else` فراموش شده است.

- ایندنت‌ها (فرورفتگی) یکدست نیست (ترکیب Tab و فاصله).
- نقل‌قول یا پرانتز بسته نشده است.
- `=` به‌جای `==` در شرط.
- فراموش کردن `()int` بعد از `()input`.
- صدا زدن تابع با پرانتز در `()command=f` به‌جای `command=f`.
- فراموش کردن `return` یا گذاشتن `print` به‌جای آن.
- اندیس بیرون از لیست (از ۰ شروع می‌شود).
- تغییر لیست هنگام پیمایش آن.
- اسم فایل برنامه همان اسم یک ماژول (`random.py`، `tkinter.py`).

۹. برنامه‌های معیوب برای تمرین

هر برنامه یک اشکال دارد. اشکال را پیدا کنید و اصلاح کنید.

برنامه معیوب ۱: خطای نحوی

```
name = input('نام: ')
if name == 'علی'
    print('سلام علی')
```

برنامه معیوب ۲: خطای منطقی

```
price = 200000
discount = 20
final = price - price * discount / 10
print('قیمت نهایی:', final)
```

برنامه معیوب ۳: خطای زمان اجرا

```
names = ['علی', 'سارا', 'رضا']
for i in range(len(names) + 1):
    print(i, names[i])
```

برنامه معیوب ۴: برنامه گرافیکی که فوراً کار می‌کند

```
import tkinter as tk

win = tk.Tk()
lbl = tk.Label(win, text='کلیک نشده')
lbl.pack()

def clicked():
    lbl.config(text='کلیک شد')

tk.Button(win, text='کلیک', command=clicked()).pack()
win.mainloop()
```

برنامه معیوب ۵: حلقه بی‌پایان

```
count = 0
while count < 5:
    print('دور', count)
```

> **راهنمایی:** در برنامه ۱ سه مشکل هست. در برنامه ۲ به خروجی دقت کنید و درصد را حساب کنید. در برنامه ۴ فقط یک جفت پرانتز زیادی است. برای برنامه ۵ می‌توانید با Ctrl+C توقف کنید.

کوییز

س۱. پیام خطا را از کجا شروع می‌خوانیم؟

الف) از خط اول

ب) از خط آخر

ج) از وسط

د) فرق نمی‌کند

س۲. برنامه‌ای بدون پیام خطا جواب غلط می‌دهد. چه نوع خطایی است؟

الف) نحوی

ب) زمان اجرا

ج) منطقی

د) سخت‌افزاری

س۳. کدام کلید در اشکال‌زدا یک خط جلو می‌رود بدون ورود به داخل تابع؟

الف) `F5`

ب) `F10`

ج) `F11`

د) `Shift+F5`

س۴. breakpoint چیست؟

الف) خطای نحوی

ب) نقطه‌ای که برنامه در آن متوقف می‌شود

ج) نوعی تابع

د) پیام خطای پایتون

پاسخ‌ها

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: ب.

حل برنامه معیوب ۱

سه اشکال: دونقطه بعد از شرط، نقل‌قول بسته نشده، و ایندنت. نسخه درست:

```
name = input('نام: ')
if name == 'علی':
    print('سلام علی')
```

حل برنامه معیوب ۲

باید تقسیم بر ۱۰۰ باشد، نه ۱۰ (درصد):

```
price = 200000
discount = 20
final = price - price * discount / 100
print('قیمت نهایی:', final)
```

خروجی:

```
قیمت نهایی: 160000.0
```

حل برنامه معیوب ۳

`range(len(names) + 1)` یک عدد اضافه می‌دهد و `names[3]` وجود ندارد (`IndexError`). باید `range(len(names))` باشد:

```
names = ['علی', 'سارا', 'رضا']
for i in range(len(names)):
    print(i, names[i])
```

خروجی:

```
0 علی
1 سارا
2 رضا
```

حل برنامه معیوب ۴

در `command=clicked()` تابع همان لحظه اجرا می‌شود. پرانتز را بردارید:

```
import tkinter as tk

win = tk.Tk()
lbl = tk.Label(win, text='کلیک نشده')
lbl.pack()

def clicked():
    lbl.config(text='کلیک شد')

tk.Button(win, text='کلیک', command=clicked).pack()
win.mainloop()
```

حل برنامه معیوب ۵

مقدار `count` در حلقه تغییر نمی‌کند:

```
count = 0
while count < 5:
    print('دور', count)
    count += 1
```

خروجی:

```
0 دور
1 دور
2 دور
3 دور
4 دور
```

پیوست ث: چیدمان در Tkinter و نقاشی با Canvas

در درس‌ها اغلب از `pack()` استفاده کردیم که ساده است ولی برای فرم‌های مرتب کافی نیست. این پیوست سه مدیر چیدمان (`pack`، `grid`، `place`) و ابزار `Canvas` برای نقاشی و انیمیشن را آموزش می‌دهد.

۱. سه مدیر چیدمان

- **`pack`**: اجزا را پشت سر هم در یک جهت (بالا، پایین، چپ، راست) می‌چیند. برای چیدمان‌های ساده و نوار ابزار مناسب است.
- **`grid`**: اجزا را در سطر و ستون جدول می‌چیند. برای فرم‌ها و ماشین‌حساب بهترین است.
- **`place`**: اجزا را با مختصات دقیق می‌گذارد. برای موارد خاص (مثلاً قرار دادن روی تصویر).

> **هشدار:** داخل یک ظرف (پنجره یا `Frame`) فقط از **یکی** از `pack` و `grid` استفاده کنید. ترکیب آنها در یک ظرف باعث می‌شود برنامه قفل کند یا خطا بدهد. اما ظرف‌های مختلف می‌توانند مدیر متفاوت داشته باشند.

۲. pack: گزینه‌ها

گزینه‌های مهم: `side` (جهت: `'top'`، `'bottom'`، `'left'`، `'right'`)، `fill` (پر کردن: `'x'`، `'y'`، `'both'`)، `expand` (گرفتن فضای باقی‌مانده)، `padx` و `pady` (فاصله) و `anchor` (جهت‌گیری در فضای خود).

مثال ۱: چیدمان کلاسیک با pack

```
import tkinter as tk

win = tk.Tk()
win.title('pack')
win.geometry('360x240')
tk.Label(win, text='نوار بالا', bg='lightblue').pack(fill='x')
tk.Label(win, text='نوار پایین', bg='lightgreen').pack(side='bottom', fill='x')
tk.Label(win, text='چپ', bg='khaki', width=10).pack(side='left', fill='y')
tk.Label(win, text='مرکز', bg='white').pack(expand=True, fill='both')
win.mainloop()
```

> **نکته:** **ترتیب نوشتن مهم است.** `pack` هر جزء را از فضای باقی‌مانده برمی‌دارد. اجزای کناری را قبل از جزء مرکزی (`expand=True`) بنویسید.

۳. grid: سطر و ستون

گزینه‌های مهم: `row` و `column` (جایگاه)، `rowspan` و `columnspan` (چند خانه را بگیرد)، `sticky` (چسبیدن به جهت‌ها: `'n'`، `'s'`، `'e'`، `'w'` یا ترکیب مثل `'ew'`) و `padx` و `pady`.

مثال ۲: فرم ثبت‌نام با grid

برای فرم فارسی، برچسب‌ها را در ستون ۱ (سمت راست) و کادرها را در ستون ۰ می‌گذاریم.

```
import tkinter as tk

win = tk.Tk()
win.title('ثبت‌نام')
labels = ['نام:', 'نام خانوادگی:', 'ایمیل:']
entries = []
for r, text in enumerate(labels):
    tk.Label(win, text=text).grid(row=r, column=1, sticky='e', padx=8, pady=5)
    e = tk.Entry(win, width=24)
    e.grid(row=r, column=0, padx=8, pady=5)
    entries.append(e)
tk.Button(win, text='ثبت', width=12).grid(row=3, column=0, columnspan=2,
pady=10)
win.mainloop()
```

> **یادآوری:** `sticky='e'` یعنی جزء به لبه شرقی (راست) خانه بچسبد. `columnspan=2` یعنی دکمه دو ستون را بگیرد.

مثال ۳: پنجره‌ای که با تغییر اندازه، می‌کشد

با `columnconfigure` و `rowconfigure` و `weight` تعیین می‌کنیم کدام سطر یا ستون هنگام بزرگ شدن پنجره بزرگ شود.

```
import tkinter as tk

win = tk.Tk()
win.title('کشسان')
win.geometry('320x200')
win.columnconfigure(0, weight=1)
win.rowconfigure(1, weight=1)
tk.Entry(win).grid(row=0, column=0, sticky='ew', padx=8, pady=8)
tk.Text(win, height=5).grid(row=1, column=0, sticky='nsew', padx=8)
tk.Button(win, text='ارسال').grid(row=2, column=0, pady=8)
win.mainloop()
```

۴. place: مختصات

`x` و `y` مختصات بر حسب پیکسل (از گوشه بالا و چپ) و `relx` و `rely` مختصات نسبی بین ۰ و ۱ هستند. `anchor` نقطه‌ای از جزء را مشخص می‌کند که روی مختصات می‌نشیند.

مثال ۴: قرار دادن دقیق

```
import tkinter as tk


win = tk.Tk()
win.geometry('300x200')
tk.Label(win, text='وسط', bg='gold', width=10).place(relx=0.5, rely=0.5,
anchor='center')
tk.Button(win, text='گوشه').place(x=10, y=10)
tk.Button(win, text='پایین راست').place(relx=1.0, rely=1.0, anchor='se')
win.mainloop()
```

> **نکته:** با `place` چیدمان با تغییر اندازه پنجره یا فونت به هم می‌ریزد. برای فرم‌های معمولی `grid` بهتر است.

۵. Frame و LabelFrame: ظرف‌ها

با `Frame` یک پنجره را به چند ناحیه تقسیم می‌کنیم و در هر ناحیه مدیر چیدمان جدا به کار می‌بریم. `LabelFrame` یک قاب با عنوان است.

مثال ۵: ترکیب ظرف‌ها

```
import tkinter as tk


win = tk.Tk()
win.title('ترکیب')
top = tk.Frame(win)
top.pack(pady=8)
tk.Label(top, text='نام:').pack(side='right')
entry = tk.Entry(top)
entry.pack(side='right', padx=5)


group = tk.LabelFrame(win, text='گزینه ها', padx=10, pady=5)
group.pack(padx=10, pady=5, fill='x')
choice = tk.StringVar(value='a')
tk.Radiobutton(group, text='گزینه اول', variable=choice, value='a').grid(row=0,
column=0)
tk.Radiobutton(group, text='گزینه دوم', variable=choice, value='b').grid(row=0,
column=1)


tk.Button(win, text='تایید').pack(pady=8)
win.mainloop()
```

۶. ماشین‌حساب با grid

یک کاربرد واقعی `grid`: دکمه‌های ماشین‌حساب در جدول ۴ ستونه و برنامه کامل.

مثال ۶: ماشین‌حساب

```
import tkinter as tk


win = tk.Tk()
win.title('ماشین‌حساب')
win.resizable(False, False)
display = tk.Entry(win, justify='right', font=('Tahoma', 18), width=14)
display.grid(row=0, column=0, columnspan=4, padx=6, pady=6)
display.insert(0, '0')
state = {'a': None, 'op': None, 'new': True}


def press(ch):
    if state['new']:
        display.delete(0, tk.END)
        state['new'] = False
    display.insert(tk.END, ch)


def set_op(op):
    try:
        state['a'] = float(display.get())
    except ValueError:
```

```
        return
    state['op'] = op
    state['new'] = True


def equals():
    if state['op'] is None:
        return
    try:
        b = float(display.get())
    except ValueError:
        return
    a, op = state['a'], state['op']
    try:
        if op == '+':
            r = a + b
        elif op == '-':
            r = a - b
        elif op == '*':
            r = a * b
        else:
            r = a / b
    except ZeroDivisionError:
        r = 'خطا'
    display.delete(0, tk.END)
    display.insert(0, r)
    state['op'] = None
    state['new'] = True


def clear():
    display.delete(0, tk.END)
    display.insert(0, '0')
    state.update(a=None, op=None, new=True)


keys = [['7', '8', '9', '/'], ['4', '5', '6', '*'],
        ['1', '2', '3', '-'], ['0', '.', '=', '+']]
for r, row in enumerate(keys, start=1):
    for c, ch in enumerate(row):
        if ch in '+-*/':
            cmd = lambda o=ch: set_op(o)
        elif ch == '=':
            cmd = equals
        else:
            cmd = lambda x=ch: press(x)
        tk.Button(win, text=ch, width=4, height=2, command=cmd).grid(
            row=r, column=c, padx=2, pady=2)
tk.Button(win, text='C', command=clear).grid(
    row=5, column=0, columnspan=4, sticky='ew', padx=2, pady=4)
win.mainloop()
```

> **نکته:** ساخت دکمه‌ها با دو حلقه تودرتو و فهرست `keys` کد را کوتاه می‌کند. اضافه کردن دکمه جدید فقط افزودن یک عضو به فهرست است.

۷. Canvas: بوم نقاشی

`Canvas` یک صفحه برای ترسیم شکل است. مختصات از **گوشه بالا و چپ** `(0, 0)` شروع می‌شود؛ `x` به راست و `y` به **پایین** زیاد می‌شود.

شکل‌ها: `create_line(x1, y1, x2, y2)`، `create_rectangle(x1, y1, x2, y2)`، `create_oval(x1, y1, x2, y2)` (بیضی داخل مستطیل مماس)، `create_polygon(...)`، `create_text(x, y, text=...)`.

مثال ۷: کشیدن یک خانه

```
import tkinter as tk

win = tk.Tk()
win.title('خانه')
canvas = tk.Canvas(win, width=300, height=260, bg='skyblue')
canvas.pack()
canvas.create_rectangle(0, 200, 300, 260, fill='forestgreen', outline='')
canvas.create_rectangle(80, 110, 220, 200, fill='khaki')
canvas.create_polygon(70, 110, 150, 50, 230, 110, fill='firebrick')
canvas.create_rectangle(135, 150, 165, 200, fill='saddlebrown')
canvas.create_oval(245, 20, 285, 60, fill='gold', outline='')
canvas.create_text(150, 235, text='خانه من', font=('Tahoma', 12))
win.mainloop()
```

مثال ۸: رسم با کلیک ماوس

هر رویداد یک شیء است که مختصات ماوس را در `event.x` و `event.y` دارد.

```
import tkinter as tk

win = tk.Tk()
win.title('کلیک کن')
canvas = tk.Canvas(win, width=400, height=300, bg='white')
canvas.pack()

def draw(event):
    r = 15
    canvas.create_oval(event.x - r, event.y - r, event.x + r, event.y + r,
                       fill='skyblue')

canvas.bind('<Button-1>', draw)
win.mainloop()
```

۸. انیمیشن با after و move

برای حرکت، شکل را با `canvas.move(شکل, dx, dy)` جابه‌جا می‌کنیم و تابع را با `win.after` تکرار می‌کنیم.

هرگز از `while` و `time.sleep` استفاده نکنید، چون پنجره قفل می‌شود.

مثال ۹: توپ پرتابی

```
import tkinter as tk

win = tk.Tk()
win.title('توپ')
canvas = tk.Canvas(win, width=400, height=300, bg='white')
canvas.pack()
ball = canvas.create_oval(20, 20, 60, 60, fill='tomato')
dx, dy = 4, 3

def move():
    global dx, dy
    canvas.move(ball, dx, dy)
    x1, y1, x2, y2 = canvas.coords(ball)
    if x1 <= 0 or x2 >= 400:
        dx = -dx
    if y1 <= 0 or y2 >= 300:
        dy = -dy
    win.after(20, move)

move()
win.mainloop()
```

مثال ۱۰: حرکت با کلیدهای جهت‌نما

```
import tkinter as tk

win = tk.Tk()
win.title('حرکت')
canvas = tk.Canvas(win, width=400, height=300, bg='white')
canvas.pack()
box = canvas.create_rectangle(180, 130, 220, 170, fill='purple')

def key(event):
    step = 10
    if event.keysym == 'Left':
        canvas.move(box, -step, 0)
    elif event.keysym == 'Right':
        canvas.move(box, step, 0)
    elif event.keysym == 'Up':
        canvas.move(box, 0, -step)
    elif event.keysym == 'Down':
        canvas.move(box, 0, step)

win.bind('<Key>', key)
win.mainloop()
```

نکته: `bind('<Key>', ...)` روی پنجره می‌نشیند و `event.keysym` نام کلید را می‌دهد (`'Left'`، `'a'`، `'space'`، `'Return'`).

کوییز

س۱. کدام مدیر چیدمان برای ساختن فرم با سطر و ستون مناسب است؟

الف) `pack`

ب) `grid`

ج) `place`

د) `Frame`

س۲. در `Canvas` مبدأ مختصات `(0, 0)` کجاست؟

الف) مرکز بوم

ب) گوشه پایین و چپ

ج) گوشه بالا و چپ

د) گوشه بالا و راست

س۳. `sticky='ew'` در `grid` چه می‌کند؟

الف) جزء را وسط می‌گذارد

ب) جزء را به دو طرف راست و چپ خانه می‌چسباند تا افقی کشیده شود

ج) جزء را پنهان می‌کند

د) جزء را به بالا می‌چسباند

س۴. چرا برای انیمیشن از `while` و `time.sleep` استفاده نمی‌کنیم؟

الف) چون کند است

ب) چون پنجره قفل می‌شود

ج) چون پایتون اجازه نمی‌دهد

د) چون `after` خطا می‌دهد

تمرین

۱. فرم ورود (نام کاربری و رمز) را با `grid` بسازید و دکمه «ورود» دو ستون را بگیرد.
۲. با `place` یک برچسب را دقیقاً وسط پنجره بگذارید.
۳. با `Canvas` یک آدم‌برفی بکشید (سه دایره روی هم، دو چشم و یک کلاه).
۴. در مثال توپ، سرعت توپ را با یک `Scale` قابل تغییر کنید.
۵. **چالش:** بازی «توپ را بگیر»: یک مربع که با جهت‌نما حرکت می‌کند و یک دایره که با کلیک روی آن شروع جدید می‌شود.

> **راهنمایی:** برای تمرین ۴ مقدار `dx` و `dy` را از `scale.get()` بخوانید. برای آدم‌برفی دایره‌ها را از پایین (بزرگ) به بالا (کوچک) بکشید.

پاسخ‌ها

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب. س۴: ب.

حل تمرین ۱: فرم ورود

```
import tkinter as tk


win = tk.Tk()
win.title('ورود')
tk.Label(win, text='نام کاربری:').grid(row=0, column=1, sticky='e', padx=8,
pady=6)
user = tk.Entry(win)
user.grid(row=0, column=0, padx=8, pady=6)
tk.Label(win, text='رمز عبور:').grid(row=1, column=1, sticky='e', padx=8,
pady=6)
pwd = tk.Entry(win, show='*')
pwd.grid(row=1, column=0, padx=8, pady=6)
tk.Button(win, text='ورود', width=14).grid(row=2, column=0, columnspan=2,
pady=10)
win.mainloop()
```

حل تمرین ۲: وسط پنجره

```
import tkinter as tk


win = tk.Tk()
win.geometry('300x200')
tk.Label(win, text='وسط', font=('Tahoma', 16)).place(relx=0.5, rely=0.5,
anchor='center')
win.mainloop()
```

حل تمرین ۳: آدم‌برفی

```
import tkinter as tk


win = tk.Tk()
win.title('آدم‌برفی')
c = tk.Canvas(win, width=240, height=340, bg='lightblue')
c.pack()
c.create_oval(50, 220, 190, 330, fill='white')
c.create_oval(70, 130, 170, 230, fill='white')
c.create_oval(90, 60, 150, 130, fill='white')
c.create_oval(105, 80, 115, 90, fill='black')
c.create_oval(125, 80, 135, 90, fill='black')
c.create_polygon(120, 92, 120, 100, 150, 96, fill='orange')
c.create_rectangle(90, 40, 150, 62, fill='black')
c.create_rectangle(105, 15, 135, 42, fill='black')
win.mainloop()
```

حل تمرین ۴: سرعت قابل تغییر

```
import tkinter as tk


win = tk.Tk()
canvas = tk.Canvas(win, width=400, height=300, bg='white')
canvas.pack()
speed = tk.Scale(win, from_=1, to=15, orient='horizontal', label='سرعت')
speed.set(4)
speed.pack(fill='x')
ball = canvas.create_oval(20, 20, 60, 60, fill='tomato')
dirs = {'x': 1, 'y': 1}


def move():
    s = speed.get()
    canvas.move(ball, dirs['x'] * s, dirs['y'] * s)
    x1, y1, x2, y2 = canvas.coords(ball)
    if x1 <= 0 or x2 >= 400:
        dirs['x'] = -dirs['x']
    if y1 <= 0 or y2 >= 300:
        dirs['y'] = -dirs['y']
    win.after(20, move)


move()
win.mainloop()
```

پیوست ج: نصب و عیب‌یابی

مشکل‌های نصب و تنظیم معمولاً اولین دیوار سر راه مبتدی‌هاست. این پیوست رایج‌ترین مشکل‌ها را با راه‌حل کوتاه فهرست می‌کند.

> **نکته:** اگر هنگام اجرای دستورها خطایی دیدید، **متن کامل آن** را در یک موتور جستجو بنویسید. تقریباً همیشه کس دیگری قبلاً با همان مشکل روبه‌رو شده است.

۱. نصب پایتون

ویندوز

۱. از وب‌سایت رسمی `python.org` بخش Downloads نسخه پایدار جدید را بگیرید.

۲. فایل نصب را اجرا کنید و **پیش از زدن Install** گزینه **Add python.exe to PATH** را تیک بزنید.

۳. در پایان در ترمینال (Command Prompt یا PowerShell) بنویسید: `python --version`

مک

نصب‌کننده رسمی را از `python.org` بگیرید (همراه `tkinter` می‌آید) یا با Homebrew نصب کنید: `brew install python` و برای tkinter: `brew install python-tk`.

لینوکس (اوبونتو و دبیان)

```
sudo apt update
sudo apt install python3 python3-pip python3-venv python3-tk
python3 --version
```

> **یادآوری:** این کتاب به پایتون **۳٫۱۰ یا بالاتر** نیاز دارد (دستور `match` در برگه تقلب از ۳٫۱۰ شروع شده است). با `python --version` نسخه را ببینید.

۲. ساخت و اجرای اولین فایل

```
import sys
print(sys.version)
print('پایتون آماده است')
```

اگر خروجی نسخه و جمله بالا را دیدید، همه چیز درست نصب شده است.

۳. مشکل: 'python' is not recognized

علامت: در ترمینال `python` را می‌زنید و پیامی مثل «'python' is not recognized as an internal or external command» می‌بینید.

علت: پایتون به `PATH` اضافه نشده است.

راه‌حل:

- فایل نصب پایتون را دوباره اجرا کنید، Modify را بزنید و **Add Python to environment variables** را فعال کنید.
- یا به‌جای `python` از `py` استفاده کنید (در ویندوز همراه پایتون نصب می‌شود): `py --version`
- بعد از تغییر، ترمینال را **ببندید و دوباره باز کنید**.

۴. مشکل: VS Code پایتون را پیدا نمی‌کند

۱. کلیدهای `Ctrl+Shift+P` را بزنید.
۲. عبارت `Python: Select Interpreter` را انتخاب کنید.
۳. نسخه نصب‌شده پایتون را از فهرست انتخاب کنید.
۴. اکستنشن **Python** نصب باشد (Ctrl+Shift+X).

نسخه انتخاب‌شده در نوار پایین سمت راست VS Code دیده می‌شود.

۵. مشکل: pip شناخته نمی‌شود یا کتابخانه نصب نمی‌شود

راه مطمئن: همیشه `pip` را از راه پایتون صدا بزنید:

```
python -m pip install --upgrade pip
python -m pip install colorama
```

- خطای `Permission denied` یا دسترسی: ترمینال را با دسترسی Administrator باز کنید یا گزینه `--user` را اضافه کنید: `python -m pip install --user colorama`
- نصب کند یا قطع می‌شود: اتصال اینترنت و فیلترشکن را بررسی کنید. دسترسی به سرور بسته‌ها (PyPI) ممکن است محدود باشد؛ در این صورت از منبع محلی (mirror) استفاده کنید.

۶. مشکل: No module named 'tkinter'

- **لینوکس:** `sudo apt install python3-tk`
- **مک:** نسخه نصب‌شده از `python.org` را استفاده کنید یا `brew install python-tk`

- **ویندوز:** هنگام نصب، گزینه `tcl/tk and IDLE` را تیک بزنید. نصب را دوباره اجرا و Modify کنید.

برای آزمایش: `python -m tkinter` یک پنجره کوچک نشان می‌دهد.

۷. مشکل: نمایش فارسی

در ترمینال (کنسول)

بیشتر ترمینال‌ها (از جمله پنجره سیاه ویندوز و ترمینال VS Code) **بدون پشتیبانی کامل راست‌به‌چپ و اتصال حروف** هستند. فارسی ممکن است جدا جدا یا برعکس دیده شود. این ضعف ترمینال است و برنامه شما درست کار می‌کند.

> **نکته:** اگر خروجی فارسی خوانا لازم دارید از **پنجره گرافیکی** (Tkinter) استفاده کنید یا خروجی را در فایل بنویسید.

اگر در ویندوز خطای `UnicodeEncodeError` گرفتید:

```
chcp 65001
set PYTHONUTF8=1
python myprogram.py
```

یا در PowerShell: `$env:PYTHONUTF8 = 1`

برای نمایش بهتر فارسی در ترمینال می‌توانید دو کتابخانه کمکی نصب کنید (`pip install arabic-reshaper python-bidi`):

```
import arabic_reshaper
from bidi.algorithm import get_display

text = 'سلام دنیا'
print(get_display(arabic_reshaper.reshape(text)))
```

در فایل‌ها

همیشه با `encoding='utf-8'` بنویسید و بخوانید (درس ۱۰). ویرایشگر VS Code فایل را به‌طور پیش‌فرض با UTF-۸ ذخیره می‌کند.

در پنجره Tkinter

در ویندوز Tkinter معمولاً فارسی را درست نشان می‌دهد. روی مک و لینوکس ممکن است حروف جدا شوند. نکته‌ها:

- فونتی انتخاب کنید که فارسی دارد: `font=('Tahoma', 12)` یا هر فونت فارسی نصب‌شده.

- متن‌هایی که ترکیب فارسی و انگلیسی دارند ممکن است ترتیب جابه‌جا شوند؛ ترکیب‌ها را کوتاه بگیرید.
- برای تراز راست در `Label` از `anchor='e'` و `justify='right'` استفاده کنید.

۸. مشکل: فایل پیدا نمی‌شود (FileNotFoundError)

علت متداول: برنامه را از پوشه‌ای دیگر اجرا می‌کنید و مسیر نسبی فایل در آن پوشه وجود ندارد.

راه‌حل: مسیر را نسبت به خود فایل برنامه بسازید.

```
from pathlib import Path

here = Path(__file__).parent
print(here / 'note.txt')
```

در VS Code می‌توانید با راست‌کلیک روی پوشه و Open in Integrated Terminal برنامه را از همان پوشه اجرا کنید.

۹. مشکل: پنجره گرافیکی باز نمی‌شود یا می‌بندد

- **فوراً بسته می‌شود:** `win.mainloop()` را ننوشته‌اید.
- **پنجره باز می‌شود ولی پشت VS Code است:** به نوار وظیفه نگاه کنید یا پنجره را جابه‌جا کنید.
- **پنجره بسته نمی‌شود و ترمینال هنوز در حال اجراست:** در ترمینال `Ctrl+C` بزنید. در VS Code دکمه مربع قرمز (Stop) را بزنید.
- **پنجره قفل می‌کند:** حلقه طولانی یا `time.sleep` را داخل تابع دکمه گذاشته‌اید. به‌جای آن از `win.after` استفاده کنید (پیوست ث).

۱۰. مشکل: فعال‌سازی محیط مجازی در PowerShell

اگر هنگام `.venv\Scripts\activate` پیام «running scripts is disabled on this system» دیدید:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

بعد از زدن `Y` دوباره فعال‌سازی کنید. این تغییر فقط برای کاربر فعلی است.

۱۱. مشکل: ساخت exe با PyInstaller

- **آنتی‌ویروس فایل را مشکوک می‌داند:** رایج است. برنامه را از لیست استثنا خارج کنید یا فایل را روی دستگاه آزمایشی بسازید.
- **برنامه باز می‌شود و فوراً می‌بندد:** یک بار بدون `--noconsole` بسازید تا پیام خطا را ببینید.

- **فایل‌های کمکی (عکس، داده) پیدا نمی‌شوند:** با `--add-data` همراه برنامه بسته‌بندی کنید (برگه تقلب، بخش ۱۴).

۱۲. جدول خلاصه: خطا، علت، راه‌حل

- **`ModuleNotFoundError`**: ماژول نصب نیست یا مفسر اشتباه است. `python -m pip install` و Select Interpreter.
- **`SyntaxError`**: نوشتار اشتباه. پرانتز، نقل‌قول و دونقطه را بررسی کنید (پیوست ت).
- **`IndentationError`**: ایندنت نامنظم. Tab و فاصله را ترکیب نکنید.
- **`UnicodeEncodeError` در ترمینال:** `chcp 65001` و `PYTHONUTF8=1`.
- **`UnicodeDecodeError` هنگام خواندن فایل:** `encoding='utf-8'` بنویسید.
- **`FileNotFoundError`**: مسیر یا پوشه اجرا را بررسی کنید.
- **`No module named 'tkinter'`**: بسته tkinter را نصب کنید.
- **`TclError: no display name`** (**لینوکس بدون صفحه**): محیط گرافیکی ندارید؛ برنامه را روی دستگاه دارای صفحه اجرا کنید.

کوییز

س۱. برای رفع خطای `'python' is not recognized` کدام گزینه درست است؟

الف) نصب دوباره VS Code

ب) افزودن پایتون به PATH یا استفاده از `py`

ج) حذف کتابخانه‌ها

د) تغییر فونت ترمینال

س۲. امن‌ترین شکل نصب کتابخانه کدام است؟

الف) `pip install x`

ب) `python -m pip install x`

ج) `install x`

د) `python install x`

س۳. برای اینکه فایل را مستقل از پوشه اجرا همیشه پیدا کنیم از چه استفاده می‌کنیم؟

الف) `open('C:/...')` با مسیر ثابت

ب) `Path(__file__).parent`

ج) `os.exit()`

د) `input()`

پاسخ‌ها

پاسخ کوییز: س۱: ب. س۲: ب. س۳: ب.

پیوست چ: شیوه‌نامه کدنویسی (PEP ۸)

کد را یک بار می‌نویسند و بارها می‌خوانند، پس خوانایی از زیبایی مهم‌تر است. جامعه پایتون سندی به نام **PEP ۸** دارد که قواعد نوشتن کد خوانا را تعیین می‌کند. اگر همه همین قواعد را رعایت کنند، خواندن کد دیگران مثل خواندن کد خودتان آسان می‌شود. این پیوست مهم‌ترین قواعد را در یک صفحه دارد.

۱. نام‌گذاری

- **متغیر و تابع:** حروف کوچک با زیرخط: `user_name`، `total_price()`، `is_even()`
- **ثابت‌ها:** حروف بزرگ با زیرخط: `MAX_SIZE`، `PI`
- **کلاس:** هر کلمه با حرف بزرگ و بدون زیرخط: `StudentRecord`
- **ماژول (نام فایل):** حروف کوچک و کوتاه: `converters.py`
- نام‌ها **معنادار** باشند: `age` بهتر از `a`، `count_vowels` بهتر از `f1`.
- از حروف تنها `l` (ال کوچک)، `O` و `I` به‌عنوان نام استفاده نکنید، چون شبیه ۱ و ۰ هستند.
- نام‌هایی مثل `list`، `str`، `input`، `max` را برای متغیر انتخاب نکنید.

۲. فاصله‌گذاری و ایندنت

- ایندنت **۴ فاصله** برای هر سطح. Tab و فاصله را ترکیب نکنید.
- بعد از کاما و دونقطه (در دیکشنری) یک فاصله و دور عملگرها یک فاصله: `x = a + b`
- داخل پرانتز فاصله اضافه نگذارید: `f(x)` نه `f( x )`.
- فاصله دور `=` در آرگومان پیش‌فرض نگذارید: `def f(a, b=2)`.
- طول هر خط **حداکثر ۷۹ نویسه**. خط بلند را در پرانتز بشکنید.
- بین تابع‌ها و کلاس‌های سطح بالا **دو خط خالی** و بین متدهای کلاس **یک خط خالی** بگذارید.

۳. import ها

- هر `import` در یک خط، در ابتدای فایل.
- ترتیب گروه‌ها: ماژول‌های استاندارد، سپس کتابخانه‌های نصب‌شده، سپس ماژول‌های خودتان، با یک خط خالی بین گروه‌ها.
- از `from x import *` پرهیز کنید.

۴. توضیح و docstring

- توضیح (#) بگویید **چرا**، نه اینکه کد **چه** می‌کند: `x += 1  # شمارش` بی‌فایده است ولی `# نیم‌ساعت از زمان ثبت‌نام گذشته` مفید است.
- برای هر تابع مهم یک **docstring** (رشته سه‌نقل‌قولی در خط اول بدنه) بنویسید که کارش را توضیح بدهد.
- توضیح‌های کهنه از نبودن توضیح بدترند. وقتی کد عوض شد، توضیح را هم به‌روز کنید.

۵. مقایسه: قبل و بعد

مثال ۱: کد نامرتب

```
def f(l,n):
  t=0
  for i in l:
      if i>n: t=t+i
  return t
print(f([1,5,8,2],3))
```

خروجی:

```
13
```

مثال ۲: همان کد با رعایت PEP ۸

```
def sum_above(numbers, limit):
    """مجموع عددهای بزرگتر از limit را برمی‌گرداند."""
    total = 0
    for n in numbers:
        if n > limit:
            total += n
    return total


print(sum_above([1, 5, 8, 2], 3))
```

خروجی:

```
13
```

هر دو کد همان نتیجه را می‌دهند، ولی دومی را می‌توان بدون فکر خواند.

۶. چند قاعده مهم دیگر

- مقایسه با `None` را با `is` بنویسید: `if x is None:`
- برای بررسی `True`/`False` ننویسید `if flag == True:`؛ بنویسید `if flag:`
- برای بررسی لیست خالی: `if not items:`

- `except:` خالی ننویسید؛ نوع خطا را مشخص کنید.
- از `lambda` برای تابع‌های ساده یک‌خطی استفاده کنید؛ برای کارهای پیچیده `def` بنویسید.
- یک خط یک کار: چند دستور را با `;` در یک خط نگذارید.

مثال ۳: چند عبارت درست

```
items = []
flag = True
value = None


if not items:
    print('لیست خالی است')
if flag:
    print('روشن')
if value is None:
    print('مقداردهی نشده')
```

خروجی:

```
لیست خالی است
روشن
مقداردهی نشده
```

۷. ابزارهای خودکار

- **فرمت خودکار در VS Code**: `Shift+Alt+F` کد را مرتب می‌کند (نیاز به یک فرمت‌کننده مثل Black دارد).
- **`black`**: فرمت‌کننده خودکار: `python -m pip install black` و سپس `black myfile.py`
- **`flake۸`**: بررسی‌کننده سبک: `python -m pip install flake8` و سپس `flake8 myfile.py`

> **نکته:** قاعده‌ها را مثل دستور زبان ببینید، نه قانون خشک. وقتی خواناتر می‌شود می‌توانید بشکنید؛ اما اگر در تیم کار می‌کنید، سبک مشترک را رعایت کنید.

کوییز

س۱. نام کدام یک مطابق PEP ۸ برای تابع است؟

الف) `CalculateTotal`

ب) `calculate_total`

ج) `calculateTotal`

د) `CALCULATE_TOTAL`

س۲. ایندنت استاندارد پایتون چند فاصله است؟

الف) ۲

ب) ۳

ج) ۴

د) ۸

س۳. کدام شکل بررسی `None` درست است؟

الف) `if x == None:`

ب) `if x is None:`

ج) `if x = None:`

د) `if None x:`

پاسخ‌ها

پاسخ کوییز: س۱: ب. س۲: ج. س۳: ب.

پیوست ح: راهنمای مدرس و آزمون‌ها

این پیوست برای مدرسان و معلمانی است که کتاب را در کلاس به کار می‌برند. شامل برنامه ۱۲ جلسه‌ای، نکات تدریس، پیشنهاد ارزیابی و دو آزمون کامل (میان‌ترم و پایان‌ترم) با پاسخنامه است. خوانندگان مستقل هم می‌توانند آزمون‌ها را برای سنجش خودشان به کار ببرند.

۱. ساختار پیشنهادی هر جلسه (۹۰ دقیقه)

- **مرور (۱۰ دقیقه):** کوییز کوتاه یا پرسش از تکلیف جلسه قبل.
- **مفهوم و مثال زنده (۲۵ دقیقه):** مدرس مثال‌ها را در لحظه تایپ می‌کند، دانش‌آموزان هم‌زمان تایپ می‌کنند. حتماً **خطا را عمداً** ایجاد کنید و به آنها نشان دهید.
- **کارگاه (۳۵ دقیقه):** تمرین‌های کلاسی به‌صورت انفرادی یا دونفره. مدرس بین گروه‌ها می‌چرخد.
- **کوییز و جمع‌بندی (۱۰ دقیقه):** کوییز درس و جمع‌بندی مفهوم اصلی.
- **تکلیف (۱۰ دقیقه توضیح):** تمرین‌های خانه با اشاره به راهنمایی‌ها.

> **نکته:** روش «برنامه‌نویسی دونفره» بسیار مؤثر است: یک نفر می‌نویسد (راننده) و دیگری می‌خواند و راهنمایی می‌کند (ناوبر). هر ۱۰ دقیقه جای‌شان عوض شود.

۲. برنامه ۱۲ جلسه‌ای

- **جلسه ۱ (درس ۱):** نصب و آشنایی با VS Code، مثال‌های ۱ تا ۶ و ۸. تمرین کلاسی: ۱ تا ۳. تکلیف: تمرین ۱ و ۳.
- **جلسه ۲ (درس ۲):** مثال‌های ۱ تا ۸ و ۱۰. اهمیت `int()` را با خطای عمدی نشان دهید. تکلیف: تمرین ۱ تا ۳.
- **جلسه ۳ (درس ۳):** مثال‌های ۱ تا ۸. تکلیف: تمرین ۱ و ۳.
- **جلسه ۴ (درس ۴):** مثال‌های ۱ تا ۷ و ۹ و ۱۱. تکلیف: تمرین ۱ تا ۴.
- **جلسه ۵ (درس ۵):** مثال‌های ۲ تا ۶ و ۹ و ۱۱. تکلیف: تمرین ۱ تا ۳ و ۵.
- **جلسه ۶ (درس ۶):** مثال‌های ۱ تا ۹ و ۱۱. تکلیف: تمرین ۱ تا ۴.
- **جلسه ۷:** مرور دروس ۱ تا ۶ و **آزمون میان‌ترم**.
- **جلسه ۸ (درس ۷):** مثال‌های ۱ تا ۷ و ۱۱. تکلیف: تمرین ۱ تا ۳.
- **جلسه ۹ (درس ۸):** مثال‌های ۱ تا ۷ و ۱۰ و ۱۱. تکلیف: تمرین ۱ تا ۳.
- **جلسه ۱۰ (دروس ۹ و ۱۰):** دیکشنری و فایل (جلسه فشرده؛ در صورت نیاز دو جلسه).
- **جلسه ۱۱ (درس ۱۱ و پیوست ت):** خطا، منو و اشکال‌زدایی. برنامه‌های معیوب را به‌صورت مسابقه گروهی حل کنید.

- **جلسه ۱۲ (درس ۱۲):** ارائه پروژه نهایی و **آزمون پایان‌ترم**.

> **نکته:** اگر کلاس شما ۱۰ جلسه دارد، درس‌های ۷ و ۸ را در یک جلسه فشرده کنید و مثال‌های گرافیکی درس ۶ را به تکلیف بسپارید.

۳. نکات تدریس و اشتباه‌های رایج هر درس

- **درس ۱:** دانش‌آموزان فراموش می‌کنند `mainloop` را بنویسند. روی تفاوت حرف بزرگ و کوچک (`print` و `Print`) تأکید کنید.
- **درس ۲:** علامت `=` را معادل «برابر است» ریاضی می‌فهمند. بگویید «بگذار» یا «ذخیره کن». ورودی را بدون `int()` جمع می‌کنند.
- **درس ۳:** تفاوت `/` و `//` و کاربرد `%` را با مثال شکلات و نفرات توضیح دهید.
- **درس ۴:** رشته‌ها را مثل لیست تغییرپذیر می‌پندارند (`s[0] = 'x'` خطا می‌دهد). فرق اندیس ۰ و ۱ را مرور کنید.
- **درس ۵:** `=` را به‌جای `==` می‌نویسند و ایندنت را رعایت نمی‌کنند. ترتیب `elif` را با مثال نمره نشان دهید.
- **درس ۶:** `range(5)` را شامل ۵ می‌دانند. حلقه بی‌پایان را عمداً اجرا کنید و `Ctrl+C` را نشان دهید.
- **درس ۷:** اندیس از ۰ و تفاوت `sort()` و `sorted()`. در `Listbox` با `curselection` گیج می‌شوند.
- **درس ۸:** تفاوت `print` و `return` مهم‌ترین نکته است. تابع را بدون پرانتز در `command` بنویسند.
- **درس ۹:** `get` را با `[]` اشتباه می‌گیرند. چند `Radiobutton` باید یک متغیر مشترک داشته باشند.
- **درس ۱۰:** حالت `'w'` فایل را پاک می‌کند. `encoding='utf-8'` را فراموش می‌کنند.
- **درس ۱۱:** از `except:` خالی استفاده می‌کنند. بگویید فقط خطای مشخص را بگیرند.
- **درس ۱۲:** پروژه را بزرگ تعریف می‌کنند. بگویید اول نسخه کوچک و کار کند، بعد توسعه.

۴. پیشنهاد ارزیابی

- **کوییزهای درس‌ها و تکالیف:** ۲۰٪
- **آزمون میان‌ترم:** ۲۵٪
- **پروژه نهایی:** ۲۵٪
- **آزمون پایان‌ترم:** ۳۰٪

> **نکته:** این درصدها پیشنهادی است. اگر آموزشگاه شما نظام ارزیابی دیگری دارد، به‌راحتی عوض کنید.

۵. آزمون میان‌ترم (دروس ۱ تا ۶)

زمان پیشنهادی: ۷۵ دقیقه. **بارم:** ۲۰ نمره (۱۰ سؤال چهارگزینه‌ای هر کدام ۱ نمره، ۴ سؤال خروجی هر کدام ۱٫۵ نمره، ۲ سؤال برنامه‌نویسی هر کدام ۲ نمره).

بخش الف: چهارگزینه‌ای

س۱. خروجی `print('a', 'b', sep='-')` چیست؟

الف) `a b`

ب) `ab`

ج) `a-b`

د) `-a-b`

س۲. خروجی تابع `input()` از چه نوعی است؟

الف) `int`

ب) `str`

ج) `float`

د) بستگی به ورودی دارد

س۳. حاصل `15 // 4` چیست؟

الف) `3.75`

ب) `4`

ج) `3`

د) `0`

س۴. حاصل `2 + 3 * 2 ** 2` چیست؟

الف) `20`

ب) `16`

ج) `10`

د) `14`

س۵. حاصل `'Python'[-2]` چیست؟

الف) `'o'`

ب) `'n'`

ج) `'h'`

د) `'y'`

س۶. حاصل `'a,b,c'.split(',')` چیست؟

الف) `'abc'`

ب) `['a', 'b', 'c']`

ج) `('a', 'b', 'c')`

د) `3`

س۷. کدام شرط درست نوشته شده است؟

الف) `if x = 5:`

ب) `if x == 5`

ج) `if x == 5:`

د) `if x === 5:`

س۸. حاصل `list(range(1, 10, 3))` چیست؟

الف) `[1, 4, 7, 10]`

ب) `[1, 3, 6, 9]`

ج) `[1, 4, 7]`

د) `[3, 6, 9]`

س۹. دستور `break` در حلقه چه می‌کند؟

الف) دور فعلی را رد می‌کند

ب) برنامه را می‌بندد

ج) حلقه را کاملاً قطع می‌کند

د) خطا می‌دهد

س۱۰. کدام دستور پنجره Tkinter را باز نگه می‌دارد؟

الف) `win.destroy()`

ب) `win.title()`

ج) `win.pack()`

د) `win.mainloop()`

بخش ب: خروجی برنامه

س۱۱. خروجی کد زیر را بنویسید.

```
x = 7
y = 2
print(x // y, x % y, x ** y)
```

س۱۲. خروجی کد زیر را بنویسید.

```
s = 'Tirotir'
print(s[1:4], s[::-1], len(s))
```

س۱۳. خروجی کد زیر را بنویسید.

```
total = 0
for i in range(1, 6):
    if i % 2 == 0:
        continue
    total += i
print(total)
```

س۱۴. خروجی کد زیر را بنویسید.

```
n = 3
while n > 0:
    print(n, end=' ')
    n -= 1
print('پایان')
```

بخش پ: برنامه‌نویسی

س۱۵. برنامه‌ای (متنی) بنویسید که عدد `n` را بگیرد و **جمع عددهای زوج از ۱ تا n** را چاپ کند.

س۱۶. برنامه‌ای گرافیکی بنویسید که یک کادر ورودی و یک دکمه داشته باشد. با زدن دکمه، اگر عدد واردشده زوج بود برچسب با رنگ سبز «زوج» و اگر فرد بود با رنگ قرمز «فرد» را نشان دهد.

۶. آزمون پایان‌ترم (کل کتاب)

زمان پیشنهادی: ۹۰ دقیقه. **بارم:** ۲۰ نمره (۱۰ سؤال چهارگزینه‌ای هر کدام ۱ نمره، ۴ سؤال خروجی هر کدام ۱٫۵ نمره، ۲ سؤال برنامه‌نویسی هر کدام ۲ نمره).

بخش الف: چهارگزینه‌ای

س۱. بعد از اجرای `nums = [3, 1, 2]` و `nums.append(0)` و `nums.sort()` مقدار `nums` چیست؟

الف) `[3, 1, 2, 0]`

ب) `[0, 1, 2, 3]`

ج) `[3, 2, 1, 0]`

د) `None`

س۲. حاصل `{'a': 1}.get('b', 5)` چیست؟

الف) خطا

ب) `None`

ج) `5`

د) `'b'`

س۳. اگر `def f(x): print(x * 2)` و سپس `r = f(4)` اجرا شود، مقدار `r` چیست؟

الف) `8`

ب) `None`

ج) `4`

د) خطا

س۴. اگر فایلی از قبل موجود باشد و آن را با `open('a.txt', 'w')` باز کنیم چه می‌شود؟

الف) به انتهای فایل اضافه می‌شود

ب) خطا می‌دهد

ج) محتوای قبلی پاک می‌شود

د) فقط خوانده می‌شود

س۵. کدام بلوک در `try / except / finally` همیشه اجرا می‌شود؟

الف) `finally`

ب) `else`

ج) `except`

د) `try`

س۶. کدام ویجت برای انتخاب یکی از چند گزینه ثابت مناسب است؟

الف) `Checkbutton`

ب) `Entry`

ج) `Radiobutton`

د) `Label`

س۷. `len(set([1, 1, 2, 3, 3]))` چند است؟

الف) ۵

ب) ۳

ج) ۲

د) ۱

س۸. حاصل `(lambda x: x + 1)(4)` چیست؟

الف) `4`

ب) `5`

ج) `lambda`

د) خطا

س۹. تفاوت اصلی تاپل و لیست چیست؟

الف) تاپل عضو تکراری ندارد

ب) تاپل فقط عدد می‌پذیرد

ج) تاپل بعد از ساخت قابل تغییر نیست

د) لیست مرتب نیست

س۱۰. `win.after(500, f)` چه می‌کند؟

الف) `f` را ۵۰۰ بار اجرا می‌کند

ب) `f` را بلافاصله اجرا می‌کند

ج) پنجره را ۵۰۰ پیکسل جابه‌جا می‌کند

د) `f` را بعد از ۵۰۰ میلی‌ثانیه یک بار اجرا می‌کند

بخش ب: خروجی برنامه

س۱۱. خروجی را بنویسید.

```
def f(a, b=2):
    return a * b


print(f(3), f(3, 4), f(b=5, a=1))
```

س۱۲. خروجی را بنویسید.

```
names = ['علی', 'سارا', 'رضا']
names.append('مریم')
names.remove('سارا')
print(names, len(names))
```

س۱۳. خروجی را بنویسید.

```
d = {'a': 1, 'b': 2}
d['c'] = d['a'] + d['b']
for k, v in d.items():
    print(k, v)
```

س۱۴. خروجی را بنویسید.

```
try:
    x = int('12a')
    print('بدون خطا')
except ValueError:
    print('عدد نیست')
finally:
    print('پایان')
```

بخش پ: برنامه‌نویسی

س۱۵. برنامه‌ای (متنی) بنویسید که تا وارد شدن عدد `۰` عدد بگیرد و در پایان **تعداد، جمع، میانگین و بزرگ‌ترین** عدد را چاپ کند. محاسبه‌ها را در یک تابع جدا بنویسید.

س۱۶. برنامه‌ای گرافیکی بنویسید که از کاربر نام بگیرد، یک `Checkbutton` به نام «عضویت در خبرنامه» داشته باشد و با زدن دکمه «ثبت»، نام و وضعیت عضویت را در فایل `registrations.txt` اضافه کند (به‌صورت یک خط) و پیام «ثبت شد» را در یک `messagebox` نشان دهد.

۷. معیار ارزیابی پروژه نهایی (۲۰ نمره)

- **عملکرد (۶ نمره):** برنامه همان کاری را که تعریف شده درست انجام می‌دهد.
- **کیفیت کد (۴ نمره):** نام‌گذاری معنادار، توضیح‌های مفید، رعایت PEP ۸ (پیوست چ)، تقسیم به تابع.
- **مدیریت خطا (۳ نمره):** ورودی نامعتبر باعث بسته شدن برنامه نمی‌شود و پیام مناسب می‌دهد.
- **رابط کاربری (۳ نمره):** چیدمان مرتب (پیوست ث)، متن‌های واضح و کار با کلیک و کیبورد.
- **ذخیره داده (۲ نمره):** در صورت نیاز، داده‌ها در فایل ذخیره و بازیابی می‌شوند.
- **ارائه (۲ نمره):** توضیح روشن برنامه و پاسخ به سؤال‌ها.

> **نکته:** از دانش‌آموزان بخواهید **ایده پروژه را پیش از شروع** با شما هماهنگ کنند و دامنه را کوچک نگه دارند. پروژه کوچکی که کامل کار می‌کند از پروژه بزرگ نیمه‌کاره بهتر است.

۸. پاسخنامه آزمون‌ها

آزمون میان‌ترم

چهارگزینه‌ای: س۱: ج. س۲: ب. س۳: ج. س۴: د. س۵: الف. س۶: ب. س۷: ج. س۸: ج. س۹: ج. س۱۰: د.

خروجی برنامه:

- س۱۱: `3 1 49`
- س۱۲: `iro ritoriT 7`
- س۱۳: `9` (جمع ۱ و ۳ و ۵)
- س۱۴: `پـایـان 1 2 3`

س۱۵: نمونه پاسخ

```
n = int(input('n: '))
total = 0
for i in range(1, n + 1):
    if i % 2 == 0:
        total += i
print('جمع زوج‌ها:', total)
```

س۱۶: نمونه پاسخ

```
import tkinter as tk


win = tk.Tk()
win.title('زوج یا فرد')
entry = tk.Entry(win)
entry.pack(pady=10)
lbl = tk.Label(win, text='', font=('Tahoma', 14))
lbl.pack()

def check():
    n = int(entry.get())
    if n % 2 == 0:
        lbl.config(text='زوج', fg='green')
    else:
        lbl.config(text='فرد', fg='red')


tk.Button(win, text='بررسی', command=check).pack(pady=10)
win.mainloop()
```

راهنمای نمره‌دهی برنامه‌نویسی: ۱ نمره برای منطق درست، ۰٫۵ برای ورودی و تبدیل نوع، ۰٫۵ برای خروجی مناسب.

آزمون پایان‌ترم

چهارگزینه‌ای: س۱: ب. س۲: ج. س۳: ب. س۴: ج. س۵: الف. س۶: ج. س۷: ب. س۸: ب. س۹: ج. س۱۰: د.

خروجی برنامه:

- س۱۱: `5 12 6`
- س۱۲: `['علی', 'رضا', 'مریم', '3']`
- س۱۳: سه خط: `1 a` و `2 b` و `3 c`
- س۱۴: دو خط: `عدد نیست` و `پایان`

س۱۵: نمونه پاسخ

```
def stats(nums):
    return len(nums), sum(nums), sum(nums) / len(nums), max(nums)


nums = []
n = int(input('عدد (۰ برای پایان): '))
while n != 0:
    nums.append(n)
    n = int(input('عدد (۰ برای پایان): '))


if nums:
    count, total, avg, biggest = stats(nums)
    print('تعداد:', count)
    print('جمع:', total)
    print('میانگین:', avg)
    print('بزرگترین:', biggest)
else:
    print('عددی وارد نشد')
```

س۱۶: نمونه پاسخ

```
import tkinter as tk
from tkinter import messagebox


win = tk.Tk()
win.title('ثبت‌نام')
win.geometry('280x200')
tk.Label(win, text='نام:').pack(pady=5)
entry = tk.Entry(win)
entry.pack()
news = tk.BooleanVar()
tk.Checkbutton(win, text='عضویت در خبرنامه', variable=news).pack(pady=5)


def register():
    name = entry.get().strip()
    if not name:
        messagebox.showwarning('توجه', 'نام را وارد کنید')
        return
    with open('registrations.txt', 'a', encoding='utf-8') as f:
        f.write(f'{name} - خبرنامه: {"بله" if news.get() else "خیر"}\n')
    messagebox.showinfo('ثبت', 'ثبت شد')


tk.Button(win, text='ثبت', command=register).pack(pady=10)
win.mainloop()
```

راهنمای نمره‌دهی برنامه‌نویسی: ۱ نمره برای منطق درست، ۰/۵ برای استفاده از تابع یا ساختار فایل، ۰/۵ برای مدیریت حالت‌های خاص (لیست خالی یا نام خالی).

پیوست خ: واژه‌نامه فارسی ـ انگلیسی

این واژه‌نامه اصطلاح‌های اصلی کتاب را به ترتیب **الفبای انگلیسی** فهرست می‌کند. در دنیای برنامه‌نویسی اغلب از معادل انگلیسی استفاده می‌شود، پس هر دو را یاد بگیرید. معادل‌های فارسی پیشنهادی هستند و ممکن است در منابع دیگر کمی فرق کنند.

> **نکته:** وقتی اصطلاحی را در گوگل جستجو می‌کنید، **معادل انگلیسی** نتایج بسیار بهتری می‌دهد.

A تا C

- `argument` — آرگومان (مقداری که هنگام صدا زدن تابع می‌دهیم)
- `assignment` — انتساب، مقداردهی
- `attribute` — ویژگی
- `backslash` — بک‌اسلش (\)
- `boolean` — بولی، منطقی (`True` یا `False`)
- `break` — شکستن (قطع کامل حلقه)
- `breakpoint` — نقطه توقف
- `bug` — اشکال، باگ
- `built-in` — توکار، درون‌ساخت
- `button` — دکمه
- `callback` — فراخوانی بازگشتی (تابعی که بعداً صدا زده می‌شود)
- `canvas` — بوم
- `case sensitive` — حساس به بزرگی و کوچکی حروف
- `character` — نویسه، کاراکتر
- `class` — کلاس
- `code` — کد
- `comment` — توضیح، کامنت
- `comparison` — مقایسه
- `concatenation` — الصاق، چسباندن
- `condition` — شرط
- `console` — کنسول

- `constant` — ثابت
- `continue` — ادامه (رد کردن دور فعلی)

D تا F

- `data type` — نوع داده
- `debug` — اشکال‌زدایی
- `debugger` — اشکال‌زدا
- `default value` — مقدار پیش‌فرض
- `dictionary` — دیکشنری (فرهنگ‌لغت)
- `docstring` — رشته مستندات
- `element` — عنصر، عضو
- `else` — در غیر این صورت
- `entry` — کادر ورودی
- `error` — خطا
- `event` — رویداد
- `exception` — استثنا
- `execute` — اجرا کردن
- `expression` — عبارت
- `extension` — افزونه (اکستنشن)
- `file` — فایل، پرونده
- `float` — عدد اعشاری
- `folder` — پوشه
- `for loop` — حلقه `for`
- `format` — قالب‌بندی
- `frame` — قاب
- `function` — تابع

G تا L

- `global` — سراسری
- `GUI (Graphical User Interface)` — رابط گرافیکی کاربر
- `handler` — کنترل‌گر

- `IDE (Integrated Development Environment)` — محیط توسعه یکپارچه
- `if statement` — دستور شرطی
- `import` — وارد کردن (ماژول)
- `indentation` — فرورفتگی، ایندنت
- `index` — اندیس، شماره جایگاه
- `infinite loop` — حلقه بی‌پایان
- `input` — ورودی
- `instance` — نمونه
- `integer` — عدد صحیح
- `interpreter` — مفسر
- `iteration` — تکرار، پیمایش
- `key` — کلید
- `keyword` — کلمه کلیدی
- `keyword argument` — آرگومان نام‌دار
- `label` — برچسب
- `lambda` — لامبدا (تابع بی‌نام کوتاه)
- `library` — کتابخانه
- `list` — لیست، فهرست
- `listbox` — جعبه لیست
- `local variable` — متغیر محلی
- `logic error` — خطای منطقی
- `loop` — حلقه

M تا P

- `main loop` — حلقه اصلی
- `menu` — منو
- `message box` — جعبه پیام
- `method` — متد
- `module` — ماژول
- `mutable` — تغییرپذیر
- `immutable` — تغییرناپذیر

- `None` — هیچ (مقدار تهی)
- `object` — شیء
- `operand` — عملوند
- `operator` — عملگر
- `output` — خروجی
- `package` — بسته
- `parameter` — پارامتر (جایگاه ورودی در تعریف تابع)
- `path` — مسیر
- `pixel` — پیکسل
- `print` — چاپ
- `program` — برنامه
- `programming language` — زبان برنامه‌نویسی
- `project` — پروژه
- `prompt` — اعلان (پیام درخواست ورودی)

S تا R

- `radio button` — دکمه رادیویی
- `random` — تصادفی
- `range` — بازه
- `recursion` — بازگشت (تابعی که خودش را صدا می‌زند)
- `reserved word` — کلمه رزروشده
- `return` — بازگرداندن
- `runtime error` — خطای زمان اجرا
- `scale` — لغزنده
- `scope` — دامنه (محدوده اعتبار متغیر)
- `script` — اسکریپت
- `sequence` — دنباله
- `set` — مجموعه
- `slice` — برش
- `snippet` — تکه‌کد، اسنیپت
- `sort` — مرتب‌سازی

- `statement` — دستور
- `string` — رشته
- `syntax` — نحو، دستور زبان
- `syntax error` — خطای نحوی

T تا W

- `terminal` — ترمینال
- `test` — آزمون، آزمایش
- `text widget` — کادر متن
- `traceback` — ردگیری خطا
- `tuple` — چندتایی (تاپل)
- `type` — نوع
- `unpacking` — باز کردن (جدا کردن عضوها از یک مجموعه)
- `user` — کاربر
- `value` — مقدار
- `variable` — متغیر
- `virtual environment` — محیط مجازی
- `while loop` — حلقه `while`
- `widget` — ویجت، جزء رابط
- `window` — پنجره
- `with statement` — دستور `with` (مدیریت منابع مثل فایل)

اصطلاح‌های هم‌ریشه و تفاوت‌های ظریف

- **پارامتر و آرگومان:** پارامتر نام جای خالی در تعریف تابع است و آرگومان مقداری که هنگام صدا زدن می‌دهیم. در گفت‌وگوی روزمره اغلب جابه‌جا به کار می‌روند.
- **خطا (error) و استثنا (exception):** خطا مفهوم کلی است؛ استثنا خطایی است که در زمان اجرا رخ می‌دهد و می‌توان آن را گرفت (`try / except`).
- **تابع (function) و متد (method):** متد تابعی است که به یک شیء یا نوع داده وابسته است، مثل `'abc'.upper()`.
- **ماژول (module) و کتابخانه (library):** ماژول یک فایل پایتون است و کتابخانه مجموعه‌ای از ماژول‌ها و بسته‌هاست.

- **عبارت (expression) و دستور (statement)**: عبارت مقدار دارد (`3 + 2`) و دستور کاری انجام می‌دهد (`x = 5`، `print(x)`).

پیوست د: منابع و مراجع

برای ادامه راه پس از این کتاب، منابع زیر پیشنهاد می‌شوند. بیشتر آنها رایگان و آنلاین هستند.

> **یادآوری:** نشانی‌های اینترنتی با گذشت زمان تغییر می‌کنند. اگر پیوندی کار نکرد نام منبع را جستجو کنید. پیش از چاپ نهایی، نشانی‌ها را یک بار بررسی کنید.

۱. مستندات رسمی

- **مستندات پایتون:** `https://docs.python.org/3/` شامل آموزش رسمی (Tutorial)، مرجع کتابخانه استاندارد و مرجع زبان.
- **راهنمای tkinter در مستندات پایتون:** `https://docs.python.org/3/library/tkinter.html`
- **PEP ۸ (شیوه‌نامه کدنویسی):** `https://peps.python.org/pep-0008/`
- **PEP ۲۰ (ذن پایتون):** `https://peps.python.org/pep-0020/` یا در پایتون بنویسید `import this`.
- **راهنمای پایتون در VS Code:** `https://code.visualstudio.com/docs/python/python-tutorial`
- **مستندات `pip` و محیط مجازی:** بخش Installing Python Modules و `venv` در مستندات پایتون.

۲. کتاب‌های پیشنهادی (انگلیسی)

- **Python Crash Course** نوشته Eric Matthes (No Starch Press): کتاب مقدماتی با پروژه‌های عملی. ساختار فصل‌های ریپوی `Python-1` تا حد زیادی همین کتاب را دنبال می‌کند.
- **Automate the Boring Stuff with Python** نوشته Al Sweigart (No Starch Press): پایتون برای خودکارسازی کارهای روزمره. متن آن رایگان آنلاین است.
- **Think Python** نوشته Allen B. Downey (O'Reilly): آموزش مفاهیم علوم کامپیوتر با پایتون؛ نسخه آنلاین رایگان دارد.
- **Fluent Python** نوشته Luciano Ramalho (O'Reilly): برای زمانی که پایه را خوب بلد هستید و می‌خواهید عمیق‌تر یاد بگیرید.

۳. وب‌سایت‌های آموزشی

- **Real Python** (`https://realpython.com`): مقاله‌ها و آموزش‌های عمیق.
- **TkDocs** (`https://tkdocs.com`): آموزش کامل Tkinter.

- **Python Tutor** (`https://pythontutor.com`): نمایش تصویری اجرای کد خط‌به‌خط؛ برای فهمیدن حلقه‌ها و تابع‌ها عالی است.
- **Stack Overflow** (`https://stackoverflow.com`): پرسش و پاسخ برنامه‌نویسی. پیش از پرسیدن بخش «چطور سؤال خوب بپرسیم» در پیوست ت را بخوانید.

۴. منابع آموزشگاه Tirotir

- ریپوی کلاسی ترم اول: `https://github.com/tirotir-ir/Python-1`
- برگه تقلب و نوت‌بوک‌های کودکان: `https://github.com/tirotir-ir/PythonKids_Lessons_ENG`
- اکستنشن‌های VS Code: `izeh` (اسنیپت‌های پایتون) و `Tirotir` (اسنیپت‌های Tkinter)
- ریپوی کدهای این کتاب: `https://github.com/worker2025/python-jozveh`
- وب‌سایت: `tirotir.ir` و کانال تلگرام `@AI_and_Philosophy`

۵. مسیرهای بعدی یادگیری

- **برنامه‌نویسی شی‌گرا:** پوشه `python_classes_examples` در ریپوی `Python-1`.
- **تحلیل داده:** کتابخانه‌های `numpy`، `pandas` و `matplotlib`.
- **وب:** `flask` یا `fastapi`.
- **یادگیری ماشین:** `scikit-learn` و سپس `pytorch` یا `tensorflow`.
- **خودکارسازی و ربات‌سازی:** `requests`، `selenium`، ربات تلگرام و `schedule`.

> **نکته:** بهترین راه یادگیری ادامه‌دار، ساختن پروژه‌ای است که واقعاً برایتان مهم است. ایده کوچکی پیدا کنید و آن را کامل بسازید.

پیوست ذ: نمایه

نمایه زیر اصطلاح‌های مهم را به ترتیب الفبای فارسی همراه محل بحث آنها فهرست می‌کند. ارجاع‌ها به **درس** یا **پیوست** است. برای جستجوی دقیق‌تر در فایل دیجیتال از `Ctrl+F` استفاده کنید.

> **نکته:** برای ساخت نمایه با شماره صفحه در ورد از تب References بخش Mark Entry و سپس Insert Index استفاده کنید.

آ تا پ

- آرگومان: درس ۸
- آرگومان نام‌دار: درس ۸
- اسنیپت: پیش از شروع؛ پیوست الف
- اشکال‌زدایی: درس ۱۱؛ پیوست ت
- اعشاری (`float`): درس ۲، ۳
- افزونه (اکستنشن): پیش از شروع؛ پیوست الف
- اندیس: درس ۴، ۷
- انواع داده: درس ۲
- ایندنت (فرورفتگی): درس ۵؛ پیوست چ
- برش: درس ۴، ۷
- بسته (`package`): پیوست پ
- بولی (`bool`): درس ۲، ۵
- بوم (`Canvas`): پیوست ث
- پارامتر: درس ۸
- پروژه: درس ۱۲؛ پیوست ح
- پنجره (`Tk`): درس ۱
- پوشه و مسیر فایل: درس ۱۰؛ پیوست ج

ت تا خ

- تابع: درس ۸
- تابع بازگشتی: درس ۸

- تاپل: درس ۹
- تبدیل نوع: درس ۲
- ترمینال: پیش از شروع؛ پیوست ج
- توان (`**`): درس ۳
- توضیح (کامنت): درس ۱؛ پیوست چ
- تکه‌کد: پیش از شروع
- جدول ضرب: درس ۶
- جمع و تفریق و ضرب: درس ۳
- چیدمان (`grid`، `pack`، `place`): پیوست ث
- حلقه `for`: درس ۶
- حلقه `while`: درس ۶
- حلقه بی‌پایان: درس ۶؛ پیوست ت
- خروجی (`print`): درس ۱
- خطا و انواع آن: درس ۱۱؛ پیوست ت، ج
- خطای زمان اجرا: درس ۱۱؛ پیوست ت
- خطای منطقی: پیوست ت
- خطای نحوی: پیوست ت

د تا س

- دستور `with`: درس ۱۰
- دکمه (`Button`): درس ۱، ۲
- دیکشنری: درس ۹
- ردگیری خطا (`Traceback`): پیوست ت
- رشته: درس ۴
- رمز عبور (فرم ورود): درس ۵؛ پیوست ث
- رویداد (`bind`): درس ۴؛ پیوست ث
- زوج و فرد: درس ۳، ۵
- زیرخط (نام‌گذاری): درس ۲؛ پیوست چ
- سراسری (`global`): درس ۸
- شرط (`if`، `elif`، `else`): درس ۵
- شمارنده: درس ۴، ۶

ف تا ل

- فایل (خواندن و نوشتن): درس ۱۰
- فایل اجرایی (`exe`، `pyinstaller`): درس ۱۲؛ پیوست ج
- فاصله‌گذاری و قالب کد: پیوست چ
- قالب‌بندی (`f-string`): درس ۲، ۴
- قاب (`Frame`): درس ۱۲؛ پیوست ث
- کتابخانه: درس ۱؛ پیوست پ
- کلاس: پیوست الف، ب
- کنسول: درس ۱؛ پیوست ج
- کیبورد و کلیدهای جهت‌نما: پیوست ث
- گرافیکی (`tkinter`): درس ۱ تا ۱۲
- لامبدا: درس ۴، ۸
- لغزنده (`Scale`): درس ۱۱
- لیست: درس ۷
- لاگ‌گیری (`logging`): پیوست ت

م تا ی

- ماژول: درس ۳؛ پیوست پ
- مجموعه (`set`): درس ۹
- متغیر: درس ۲
- متغیر محلی: درس ۸
- محیط مجازی (`venv`): پیوست پ، ج، ب
- مرتب‌سازی: درس ۷
- مقایسه و عملگرهای مقایسه: درس ۵
- منو (`Menu`): درس ۱۱؛ پیوست ث
- مدیریت خطا (`try`، `except`): درس ۱۱
- نام‌گذاری: درس ۲؛ پیوست چ
- نوع داده: درس ۲
- ورودی (`input`، `Entry`): درس ۲
- ویجت: درس ۱، ۲، ۶، ۷، ۹، ۱۰، ۱۱؛ پیوست ث

- `Checkbutton` و `Radiobutton`: درس ۹
- `Combobox`: درس ۹
- `Listbox`: درس ۶، ۷
- `Text` و `filedialog`: درس ۱۰
- `messagebox`: درس ۴، ۵
- `random`: درس ۷؛ پیوست پ
- `math`: درس ۳؛ پیوست پ
- `range`: درس ۶
- `import`: درس ۳؛ پیوست پ
- `PEP 8`: پیوست چ
- `break` و `continue`: درس ۶
- `after`: درس ۶، ۱۲؛ پیوست ث
- `enumerate`: درس ۷
- `raise`: درس ۱۱